ECHOCASE / ENGINEERING SERIES

企业级 .NET 全栈软件工程

ENTERPRISE .NET FULL-STACK SOFTWARE ENGINEERING

从真实实现理解架构、数据一致性与 React 状态

业务意图 领域边界 / 用例 / HTTP 合同

持久事实 EF 跟踪 / MVCC / 并发与幂等

交互承诺 确认缓存 / 草稿代次 / 冲突决策

2026 年 11 月 · 应用工程版

基于可审阅的 EchoCase 应用源码 · 中文技术教材

目录

序言：把一次保存解释清楚 8

如何阅读这本书 . . . 8

真实系统，也有明确边界 . . . 8

一条贯穿全书的判断标准 . . . 9

第 1 章 从用户任务到可执行的业务规则 10

1.1 案件定义与个人调查不是同一对象 . . . 10

1.2 把模块连成任务闭环 . . . 11

1.3 需求应包含失败后果 . . . 12

1.4 服务端投影也是需求实现 . . . 13

1.5 用反例辨认真实范围 . . . 13

1.6 以一段调查故事检查概念 . . . 13

1.7 可用性承诺同样需要边界 . . . 14

第 2 章 依赖方向、运行调用与架构取舍 15

2.1 两张不同的图 . . . 15

2.2 规则与机制分别放在哪里 . . . 16

2.3 与固定 CleanArchitecture 模板比较 . . . 16

2.4 依赖注入让选择成为运行事实 . . . 17

2.5 目录隔离之后还要检查什么 . . . 17

2.6 端口不是数据库可以随意替换的证明 . . . 18

2.7 业务域之外仍有应用工程 . . . 18

第 3 章 聚合、身份与业务状态机 19

3.1 为什么调查是聚合根 . . . 19

3.2 状态机表达允许的决定 . . . 20

3.3 无向关系的身份与规范形式 . . . 21

3.4 关联修订是一个完整行为 . . . 21

3.5 提交结论为何不只是更新状态 . . . 22

3.6 聚合没有替代数据库 . . . 23

3.7 不变量有不同的覆盖范围 . . . 24

3.8 Entity 与值语义不必制造额外类型 . . . 24

3.9 内存原子性与持久原子性 . . . 24

第 4 章 一次请求在 ASP.NET Core 中经历什么 25

4.1 从委托嵌套理解管道 . . . 25
4.2 端点参数从哪里来 . . . 26
4.3 严格 JSON 是可演进合同的一种选择 . . . 27
4.4 错误如何变成公开响应 . . . 27
4.5 TraceId 案例：结果执行发生在构造之后 . . . 28
4.6 用一条请求检查自己的理解 . . . 29
4.7 启动时构造处理器，请求时填入数据 . . . 29
4.8 await 维持调用关系，不提供业务原子性 . . . 29
4.9 返回值、响应状态和用户看到的结果 . . . 30

第 5 章 用例服务与逻辑 CQRS 31

5.1 一个服务协调什么 . . . 31
5.2 共享写路径是一段协议 . . . 32
5.3 校验分层的理由 . . . 32
5.4 查询模型与命令结果为什么相同 . . . 33
5.5 时间依赖也是用例输入 . . . 33
5.6 何时值得拆分更细 . . . 33
5.7 回调端口的能力与约束 . . . 34
5.8 规范化发生在指纹之前还是之后 . . . 34

第 6 章 HTTP 合同中的版本、意图与失败 35

6.1 URI 与方法表达什么 . . . 35
6.2 If-Match 是决定的前提 . . . 35
6.3 幂等键识别的是一次意图 . . . 36
6.4 错误分类应帮助下一步决定 . . . 37
6.5 读合同也需要边界 . . . 37
6.6 OpenAPI 与 TypeScript 提供怎样的保证 . . . 38
6.7 条件写入与条件读取不要混为一谈 . . . 38
6.8 状态码之后还需要可理解的正文 . . . 38

第 7 章 EF Core 跟踪器如何把对象变化变成 SQL 39

7.1 一个工作单元中的两种读取 . . . 39
7.2 身份解析先于变化比较 . . . 39

7.3 当前值已经改变，状态为何仍是 Unchanged . . . 40
7.4 原始 Version 如何进入条件 . . . 41
7.5 预分配 Guid 与 Added 状态 . . . 41
7.6 SaveChanges 与事务提交不是同一个事件 . . . 42
7.7 用五种状态阅读一项业务操作 . . . 43
7.8 三份值与一次真正的竞争 . . . 43
7.9 物化、变更检测、保存分别消耗什么 . . . 44

第 8 章 数据模型演进与时间精度 45

8.1 null 可以表达真实历史 . . . 45
8.2 约束之间怎样合作 . . . 46
8.3 旧回执是另一份持久合同 . . . 46
8.4 时间相等为何曾经失败 . . . 46
8.5 Down 能撤销结构，不能恢复丢失的解释 . . . 47
8.6 CHECK 与 null 的三值逻辑 . . . 47
8.7 schema 快照不是业务历史 . . . 48

第 9 章 PostgreSQL MVCC 与读取事实的时间 49

9.1 行更新留下版本，不是覆盖一张共享纸 . . . 49
9.2 Read Committed 的快照属于语句 . . . 50
9.3 单语句图读取的代价 . . . 51
9.4 可见性不会阻止丢失决定 . . . 51
9.5 跨行规则暴露更大的边界 . . . 51
9.6 事务生命周期影响读取成本 . . . 52
9.7 快照判断不只是比较大小 . . . 52
9.8 EF 身份缓存可能遮住新快照 . . . 53
9.9 更强隔离解决什么，又留下什么 . . . 53

第 10 章 根版本与事务如何共同拒绝过期写入 54

10.1 两次版本检查关闭两个不同窗口 . . . 54
10.2 真正同时读到旧版本时会发生什么 . . . 55
10.3 子实体没有版本列，为什么仍能保护 . . . 56
10.4 为什么还捕获一个唯一约束异常 . . . 56
10.5 回滚后的对象与数据库不是同一个状态 . . . 57
10.6 冲突处理为什么不能自动覆盖 . . . 57
10.7 审计唯一键为什么不能取代根条件 . . . 57

10.8 从失败结果反推验证强度 . . . 58

第 11 章 幂等回执与结果未知 59

11.1 幂等保护哪一段副作用 . . . 59
11.2 同键锁让回执检查串行发生 . . . 60
11.3 为什么先检查资源，再读取回执 . . . 60
11.4 历史结果不是最新资源 . . . 61
11.5 客户端怎样保留不确定性 . . . 61
11.6 当前保证的边界 . . . 63
11.7 三种重复要分开处理 . . . 63

第 12 章 可信身份、资源所有权与浏览器安全 64

12.1 Identity 与 Cookie 分别做什么 . . . 64
12.2 Owner 必须来自可信上下文 . . . 65
12.3 Cookie 会自动发送，所以需要防伪 . . . 65
12.4 不发送的内容，比隐藏的内容更可靠 . . . 66
12.5 安全头与频率约束的实际含义 . . . 66
12.6 安全评审回到业务事实 . . . 67
12.7 同源开发与文档安全策略 . . . 67
12.8 失败处理也有信息边界 . . . 67

第 13 章 React 渲染快照与状态所有者 68

13.1 事件回调看到哪一次渲染 . . . 68
13.2 Render 与 Effect 的职责边界 . . . 68
13.3 状态按所有权分开 . . . 69
13.4 组件身份决定状态保留多久 . . . 70
13.5 TypeScript 辅助哪些判断 . . . 70
13.6 组件库不是业务状态机 . . . 71
13.7 状态属于树中的位置，不属于函数局部变量 . . . 71
13.8 为什么函数式更新仍需业务条件 . . . 72

第 14 章 TanStack Query 与单调确认缓存 73

14.1 查询键属于资源身份 . . . 73
14.2 “后来到达”不代表“后来发生” . . . 74
14.3 mutation 是一段生命周期 . . . 75
14.4 为什么不把草稿乐观写入缓存 . . . 76

14.5 读取失败与未知写入是不同状态 . . . 76
14.6 确认缓存与草稿之间还差一层语义 . . . 76
14.7 失效刷新失败时，哪些事实仍然成立 . . . 76
14.8 操作完成边界影响组件行为 . . . 77

第 15 章 草稿基线、输入代次与显式冲突 78

15.1 一份草稿需要记录哪些事实 . . . 78
15.2 发出保存时冻结哪一份东西 . . . 79
15.3 成功确认必须对应代次 . . . 80
15.4 后台刷新不能静默替换基线 . . . 81
15.5 两种确认是两种决定 . . . 83
15.6 输入可继续，不代表所有操作都能继续 . . . 83
15.7 把确认逻辑读成状态转移表 . . . 83
15.8 自己保存成功，却又出现冲突 . . . 84
15.9 为什么不在 Effect 中自动“修复”冲突 . . . 84

第 16 章 React Flow：业务关系与本地布局 85

16.1 控件模型服务于交互 . . . 85
16.2 技术方向与业务方向分开 . . . 86
16.3 修订编辑器冻结打开时版本 . . . 87
16.4 等待实际完成，再关闭表单 . . . 87
16.5 窄屏与规模改变设计条件 . . . 89
16.6 派生边与局部节点采用不同更新方式 . . . 90
16.7 图形反馈要连接业务语言 . . . 90

第 17 章 从实际 EF 查询读懂 PostgreSQL 执行计划 91

17.1 查询形状先决定问题 . . . 91
17.2 三种路径各做了什么 . . . 92
17.3 根选择变快，外层工作仍存在 . . . 92
17.4 数值只在测量条件中有意义 . . . 93
17.5 为什么不继续增加所有可能的索引 . . . 93
17.6 从索引次序推导早停条件 . . . 94
17.7 计划是解释，不是固定模板 . . . 94

第 18 章 用反例验证承诺，而非累积测试数量 95

18.1 先确定要推翻的错误假设 . . . 95

18.2 单元边界检验规则与身份 . . . 96
18.3 集成验证必须触及真正的竞争窗口 . . . 97
18.4 浏览器反例要控制因果，少猜时间 . . . 97
18.5 测试依赖边界保护生产系统 . . . 98
18.6 回归保护与设计演进 . . . 98
18.7 有效反例需要独立的观察出口 . . . 98
18.8 小而明确的持续验证 . . . 98

第 19 章 把“修订关联”作为一次完整需求演进 99

19.1 先确定什么保持，什么改变 . . . 99
19.2 领域新增行为，应用复用协议 . . . 99
19.3 持久化兼容贯穿三条路径 . . . 99
19.4 新端点不改变旧端点的全部接受范围 . . . 100
19.5 前端拆分围绕变化原因 . . . 100
19.6 评审应从用户承诺逆向阅读 . . . 100
19.7 变更影响图为什么比文件清单有用 . . . 101
19.8 什么时候应该停止重构 . . . 101

第 20 章 设计推理题与解析 102

20.1 成功响应能否清掉新输入 . . . 102
20.2 幂等重试为什么不能换新版本 . . . 102
20.3 只给子表加版本能否保持原语义 . . . 102
20.4 DetectChanges 前 Unchanged 是否说明保存失效 . . . 103
20.5 复合索引消除了所有排序吗 . . . 103
20.6 新解释列能否全部补成迁移时间 . . . 103
20.7 把图坐标放进 Version 会怎样 . . . 103
20.8 发送通知能否放进 SavingChanges . . . 104

附录 术语、源码地图与一手参考 105

A.1 易混概念对照 . . . 105
A.2 按问题进入源码 . . . 106
A.3 框架与协议的一手资料 . . . 107
A.4 关于真实性与适用范围 . . . 107

序言：把一次保存解释清楚

调查员在笔记框里输入一句话，按下“保存”，然后继续补充另一条线索。网络稍慢，服务器先保存了第一句话，响应稍后才回来。此时，输入框应该显示什么？页面能否写“已保存”？如果另一个窗口又修改了档案，第一次保存的成功响应还能覆盖当前界面吗？

这些问题把企业应用的几个核心困难放在同一张桌上：业务对象如何表达允许的变化，数据库如何保证多项变化一起成立，HTTP 如何区别过期决定与重复尝试，前端如何保留用户尚未交付给服务器的意图。只有沿着整条路径追踪，才可能解释一个看似普通的按钮究竟承诺了什么。

本书以 EchoCase 的实际实现为主线。它是一个体量受控的调查档案应用，使用 C#、ASP.NET Core 10、EF Core 与 PostgreSQL，前端使用 React 19、TypeScript、TanStack Query 和 React Flow。案件题材让证据解锁、关系判断、调查复核和结论封存自然形成规则，读者不需要先学习复杂行业知识，就能进入一致性和交互语义。

如何阅读这本书

前六章从需求和领域进入请求、用例与 HTTP 合同。第七至十一章深入持久化：对象跟踪、兼容演进、MVCC、乐观并发和幂等。第十二章把可信身份、资源所有权与防伪校验接回同一条业务路径。第十三至十六章走到浏览器，解释渲染快照、确认缓存、草稿代次和图谱状态。最后几章讨论真实查询计划、验证边界、需求演进以及带解析的设计题。

每章的源码框来自明确固定的应用版本，显示真实路径和行号，链接可以直接定位上下文。为了在书页上阅读，超长行会折行，续行不生成新的源码行号。图中的时间轴强调因果次序，不按真实耗时比例绘制；示意事务编号和版本值会与真实观察明确区分。书中的运行摘录只保留能帮助推理的部分，不能用小段 SQL 代替整份实际查询。

正文不要求读者逐条执行命令。理解一次并发失败，并不以亲自建立数据库、收集日志和整理测试记录为前提。需要进一步研究时，可以从代码框进入实现，从章节给出的测试符号进入反例。本书让实现、图示和解释互相校正：当一句抽象原则无法说明实际源码的行为，就应缩小结论或重新理解机制。

真实系统，也有明确边界

EchoCase 只有一个固定案件定义、六条证据和至多十五条无向关系。读写共用一个 PostgreSQL 数据库，没有消息总线、独立读模型或外部通知。它使用逻辑 CQRS 与有行为的聚合，不以框架数量证明“企业级”。本书所说的企业应用工程，指对规则、身份、一致性、失败和可维护性的认真处理。

读者会看到仍需保留的限制：回执没有自动到期策略，草稿没有跨页面持久恢复，图谱位置只在当前视图中保留，账户功能也不构成完整的企业身份治理。这些限制有助于判断设计适用的条件。把样例中尚未实现的能力写成现有保证，会破坏理解，比承认范围更危险。

正文的源码固定于四目录整理后的应用提交 908811b95fb40cf14fa79b8eddf41796aa02cc04。界面与数据库观察选自 2026 年 11 月 11 日的真实运行；本轮对照确认，相关应用源文件在目录迁移后保持相同内容，只有路径与换行形式存在机械差异。这些是历史运行观察，不是对任意环境、负载或未来版本的承诺。详细素材出处保存在教材源的参考目录，读者不必在正文中穿行于记录清单。

一条贯穿全书的判断标准

理解框架，意味着能预测边界情况，而不仅能复述通常路径。读完 EF 章节，应能解释为什么修改子对象之后根版本也必须进入 SQL 条件；读完幂等章节，应能解释为何重试必须保留旧版本；读完 React 章节，应能解释为什么成功响应不能随意清空输入框的 dirty 标志。

本书会反复追问：当前掌握的事实来自哪个时刻，正在表达的是哪一次意图，哪一层有权作出这个决定，失败后还能相信什么。答案将具体落在对象、语句、事务、响应和屏幕上。把这些因果联系起来，才是一次保存真正完成的含义。

第 01 章

从用户任务到可执行的业务规则

一个案件管理应用很容易被写成五组增删改查接口：案件、证据、关联、进度和结论。EchoCase 的实际任务却有次序。调查员先选择案件，逐步调查证据，再根据事实建立联系；材料充分后进入复核，最后提交结论。若允许任意更新数据库字段，用户就可能在尚未调查证据时引用正文，或在结案后继续改写依据。功能清单没有表达这些限制，业务规则必须补上。

1.1 案件定义与个人调查不是同一对象

“雾港档案”是固定案件定义，包含题名、嫌疑人和六条证据。它类似一份可重复使用的教学材料。Investigation 则是某位用户针对该案件开启的一次调查，有自己的 Guid、状态、笔记、证据发现与关联。两个用户选择同一案件，不能因此共享已调查标记或结论。

这个区分直接影响数据模型和接口。案件列表提供材料目录，调查列表提供当前用户的档案。客户端不能把案件编号当成调查实例编号，更不能把本地选中案件等同于服务器上的持久进度。创建调查需要可信 owner；读取已有调查需要同时匹配 owner 和调查 Id。身份隔离不是界面筛选的一项便利，而是每次读取和写入都要成立的前提。



图 1.1 案件总览：①案件定义入口；②个人调查列表。固定案件与每位用户的调查实例是不同对象。

①固定案件不是共享调查进度。

②个人档案由 owner 隔离；列表统计来自已取得的调查快照。

总览中的固定案件入口与个人档案列表分开，正是这两个概念在界面中的投影。卡片标题相同不代表资源相同；真正的资源身份是调查 Guid。一次“重新开始”应建立新调查，而不是偷偷把同一个已提交对象恢复成 Active。

源码 backend/EchoCase.Domain/Investigation.cs · L33-38

```
33      public static Investigation Start(string ownerId, string caseId, DateTimeOffset now)
34      {
35          Require(!string.IsNullOrWhiteSpace(ownerId), "owner_required", "调查必须有所有者。");
36          CaseCatalog.Get(caseId);
37          return new() { Id = Guid.NewGuid(), OwnerId = ownerId, CaseId = caseId, Version = 1,
    CreatedAt = now, UpdatedAt = now };
38      }
```

Start 在入口检查 owner，并通过 CaseCatalog 校验案件存在。Version 从 1 开始，表示已经存在的第一个持久快照。CreatedAt 和 UpdatedAt 使用相同的受控时间，避免创建时内部出现两个彼此无关的时刻。构造器不接收客户端给出的状态或评分，因此客户端不能通过“创建时填好所有字段”跳过业务过程。

1.2 把模块连成任务闭环

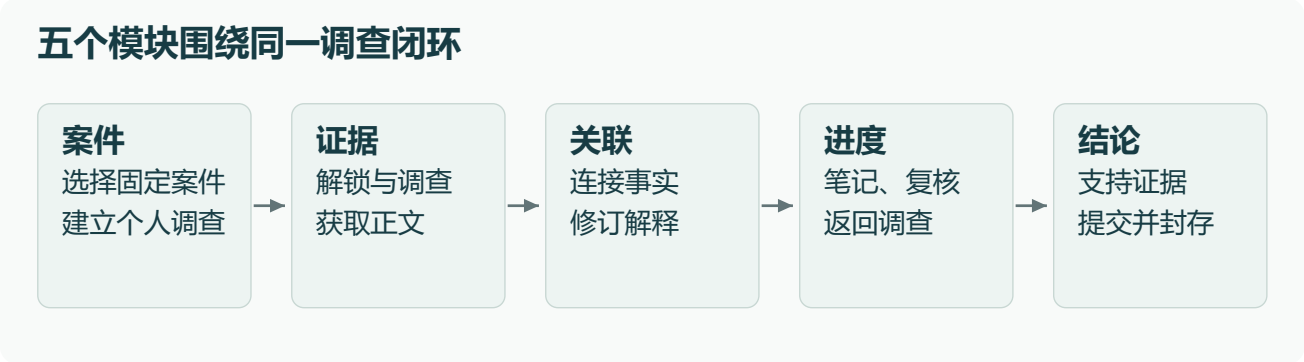


图 1.2 模块划分说明用户任务；一致性边界仍是调查聚合。

证据管理在这里不是任意上传附件。证据定义固定，一部分线索有前置条件：调查异常门禁记录后才看得到备用凭证领用册，调查领用册后再解锁调度消息。已解锁只说明可以调查，未调查时仍没有正文。界面显示“锁定”“待调查”“已调查”，对应不同事实，不能压成一个布尔值。

证据关联也不是给图上加一条装饰线。它记录调查员认为两条已调查事实互相印证或存在矛盾。关系两端必须属于本次调查已发现的证据，不能自连，也不能把 A-B 和 B-A 当成两条不同关系。关联修订允许判断改变，却保留原关系身份与端点，以便用户理解“同一判断被修正”而非“旧判断被删除后换了另一个对象”。

进度管理包含持久笔记和 Active、Reviewing、Submitted 状态。进入复核需要至少三条已调查证据和两条关联；复核中暂停编辑，允许返回调查补充材料；提交后封存。结论还要选择合法嫌疑人、提供长度合适的理由，并引用至少三条不同的已调查证据。进度条只是从这些事实计算出的呈现，不是可以随意修改的完成率字段。

1.3 需求应包含失败后果

“允许修订关联类型和解释”缺少重要语义：哪个状态允许修订，解释的长度单位是什么，其他窗口改动后怎么办，响应丢失时怎样重试。若等接口和数据库都写完后才补这些问题，客户端往往已经形成错误假设。



图 1.3 验收条件应描述事实与后果，按钮存在本身不能证明需求成立。

把这项需求写成完整句子，可以是：当前用户在 Active 调查中，针对一条已有关系，基于自己已读到的调查版本，修订类型并给出二十至五百个 Unicode 标量值的解释；成功保留关系 Id 与两端，根版本推进一次；失败不留下部分修订。这里同时出现用户目的、允许条件、结果和失败边界。后面各章将分别解释这些承诺如何落实，而不是让每一层都重复同一份校验清单。

“旧窗口得到错误”仍然不够。旧窗口应该保留用户刚输入的解释，并告诉他档案已经变化；服务器应该返回可区分的 412；数据库应该没有留下失败请求的审计或回执。错误处理因此成为功能的一部分。若失败后解释消失，即使服务器正确拒绝了过期请求，整个产品仍违背了用户的合理预期。

用户决定	成立的事实	被拒绝时应保留的东西
调查证据	前置已发现、状态可编辑	当前档案及尚未发出的输入
建立关联	两端已调查、关系未重复	用户的选择，允许重新核对
修订关联	Id 存在、版本一致、解释合法	原关系和本地解释草稿
保存笔记	状态可编辑、长度合法	未确认的新一代输入
提交结论	复核中、支持与说明完整	用户判断，不能默默改成别的结论

1.4 服务端投影也是需求实现

源码 backend/EchoCase.Application/Investigations.cs · L34-42

```
34  public static class SessionPresenter
35  {
36      public static SessionView Map(Investigation i) => new(i.Id, i.CaseId, CaseCatalog.Get(
    ·   i.CaseId).Title, i.Status.ToString(), i.Version, i.Notes, i.CreatedAt, i.UpdatedAt,
37          CaseCatalog.Get(i.CaseId).Evidence.Select(e => new EvidenceView(e.Id, e.Title, e.Kind,
38              i.IsVisible(e) ? e.Summary : "完成前置调查后解锁。", i.Discoveries.Any(d =>
    ·   d.EvidenceId == e.Id) ? e.Body : null,
39              !i.IsVisible(e), i.Discoveries.Any(d => d.EvidenceId == e.Id))).ToArray(),
40          i.Links.OrderBy(l => l.SourceId).ThenBy(l => l.TargetId).Select(l => new LinkView(
    ·   l.Id, l.SourceId, l.TargetId, l.Kind.ToString(), l.Rationale, l.RevisedAt)).ToArray(),
41          i.Score is null ? null : new(i.Suspect!, i.Reasoning!, i.SupportingEvidence!.Split(',
    ·   '), i.Score.Value));
42  }
```

SessionPresenter 没有把数据库实体原样序列化。它计算 Locked 和 Examined，只有已调查证据才映射 Body，并把领域枚举变成协议中的字符串。这个选择使未调查正文不进入普通读取结果。仅在浏览器中隐藏正文没有相同意义，因为用户可以查看网络响应。

投影也有代价。SessionView 包含完整调查所需的证据和关系，所以列表读取目前同样返回较宽的对象图。对一个小型固定案件，这能降低前端组合多个查询的复杂度；若档案数、正文或子项数量增长，就需要重新评估列表摘要。需求边界既决定规则，也决定哪种简化暂时合理。

1.5 用反例辨认真实范围

假设用户说“多人协同调查”。不能只把 owner 改为 teamId 就宣布支持协作。谁能邀请成员，已有回执归属于谁，成员被移除后是否还能重放，结论由谁封存，草稿如何提示他人编辑，都需要新的业务决定。现有根版本能够拒绝部分并发写入，却没有自动提供协作权限模型。

同样，固定规则产生的 100 分只评价本案支持证据的覆盖和指定嫌疑人，不是现实推理的置信度。教材中的小案件保留了清晰规则，帮助我们研究软件承诺；不能把题材的真实性与技术机制的真实性混为一谈。

需求分析在这里的成果不是更多文档页，而是一组能穿过代码、数据库和界面的判断。当后续设计让某项保证变得昂贵，应回到这些句子讨论取舍。这样，架构选择才有可以检验的理由。

1.6 以一段调查故事检查概念

调查员先打开异常门禁记录。此时它可见，但尚未调查，因此只有摘要。完成调查后，正文成为个人档案的一部分，领用册被解锁；领用册仍未调查，不能立即引用它的正文来建立关系。这个小过程要求三个概念分别成立：前置发现决定可见性，本次发现决定正文与可关联资格，调查状态决定能否继续修改。

如果另一窗口刚把调查送入复核，本窗口即使还显示“调查此证据”按钮，服务器仍应拒绝旧决定。需求不能写成“点击按钮后显示正文”，因为那没有说明状态变化之间的竞争。更准确的说法是：当操作提交时仍满足前提，成功发现证据并推进调查版本，随后读模型提供正文；否则保留现有事实并解释拒绝。

到结论阶段，三条支持证据也必须是不同且已调查的项目。这个条件没有要求图中所有关系组成一棵树，也没有验证自然语言推理是否逻辑完备。若把“形成完整证据链”的产品表达当成已经实现的图连通性算法，就会高估现有规则。需求语言中的形象表达，应继续落到明确计算条件。

1.7 可用性承诺同样需要边界

“不会丢输入”必须说明发生在哪里。当前工作台在切换页签时保留笔记，在晚到保存和脏刷新时保护文本；关闭页面、返回总览或切换调查则没有持久草稿恢复。若不说明生命周期，用户可能把有限保护理解成浏览器重启后仍可恢复。

“可以重试”也应指明重试对象。网络结果未知时，重试的是已冻结的原操作，不能把当前输入框中新补充的内容自动塞进旧键。当前文本可以继续编辑，但它属于下一项待决定意图。需求因此要分开讨论工作连续性和副作用连续性：前者允许继续写，后者要求先确认原操作。

这些边界并不减损功能价值。它们使文案、按钮状态、接口和恢复动作保持一致，避免界面作出后端从未承诺的保证。

第 02 章

依赖方向、运行调用与架构取舍

看到 Domain、Application、Infrastructure、API 四个项目，并不能确定系统已经获得清晰边界。真正的问题是：业务规则需要知道哪些技术细节，一次修改要经过哪些所有者，以及某项外部依赖改变时，哪一部分有理由跟着改变。EchoCase 用较少的抽象回答这些问题，也保留了不能被抽象消除的数据库语义。

2.1 两张不同的图

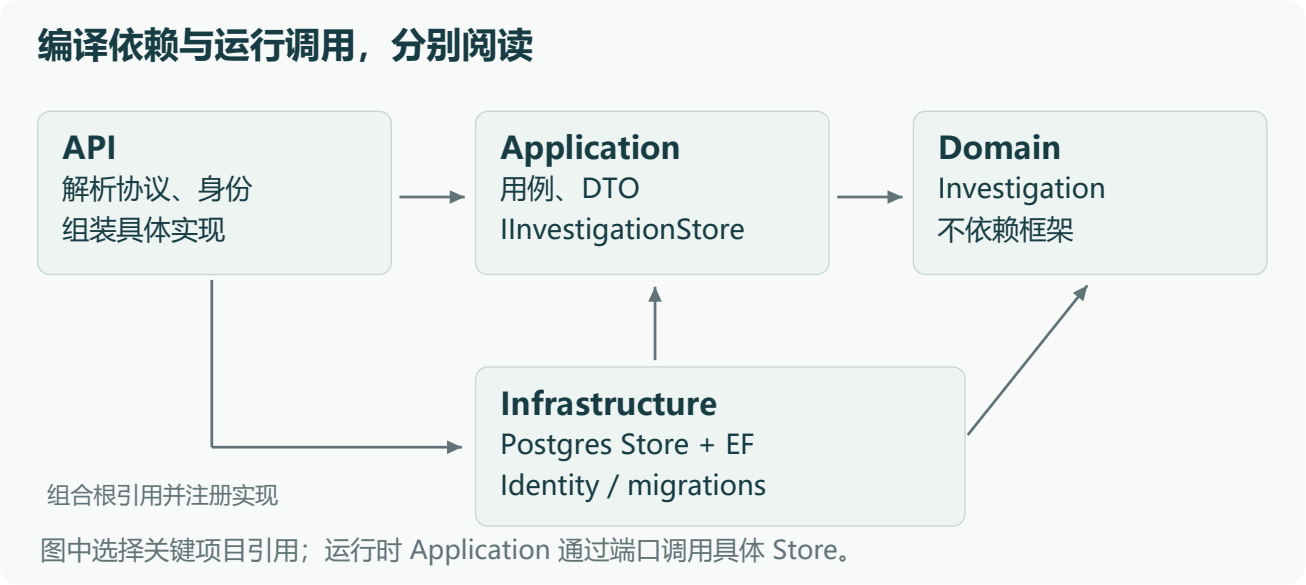


图 2.1 依赖倒置让应用层拥有端口；数据库实现依赖端口定义，而非反过来。

项目依赖图描述编译时谁可以引用谁。Domain 不引用 EF、ASP.NET 或 React；Application 引用 Domain，定义用例和存储端口；Infrastructure 引用应用端口并实现 PostgreSQL 持久化；API 作为组合根，知道具体实现和协议。箭头方向让内部规则不必引用外部框架。

运行调用图则从 HTTP 进入 InvestigationService，再通过 IInvestigationStore 调用 PostgresInvestigationStore，后者回调领域操作。调用当然会走到外部数据库。依赖倒置没有让执行方向反转；它改变的是接口定义由谁拥有，从而避免应用代码为了调用存储而编译依赖具体实现。

源码 backend/EchoCase.Application/Investigations.cs · L27-32

```
27  public interface IInvestigationStore
28  {
29      Task<IReadOnlyList<SessionView>> ListAsync(string ownerId, CancellationToken ct);
30      Task<SessionView> GetAsync(Guid id, string ownerId, CancellationToken ct);
31      Task<SessionView> WriteAsync(Change change, Func<Investigation?, Investigation> apply,
    ·       CancellationToken ct);
32  }
```

端口暴露有限的 List、Get 和 Write。它没有向 Application 暴露 DbSet 或 IQueryable，所以应用层无法随手选择 Include、跟踪策略或事务隔离。代价是查询形状被明确收在端口后面，新增复杂读需求可能

需要新增方法。Write 接收领域回调，因此它是适合本进程工作单元的端口，不能不加改造就变成远程 RPC 消息。

2.2 规则与机制分别放在哪里

“复核中不能修订关联”属于领域规则。无论 HTTP、后台任务还是未来桌面客户端调用，都应该受到它约束。“同一 owner/key 的重试先读取回执”属于当前持久化协议，需要数据库事务和锁。“If-Match 使用带引号的整数”属于 HTTP 合同，领域对象没有理由理解请求头。

这个划分防止一种常见退化：领域对象只是字段袋，应用服务承担所有 if，而 Store 又重复一遍状态判断。EchoCase 的领域根拥有状态转移和成员变更；应用服务命名用户意图、建立指纹并提供时间；Store 把一个意图变成原子持久事实。每层都有工作，边界并不意味着每层只能透传。

但“规则纯净”也有现实限度。Domain 里的端点排序使用 ordinal 规则，数据库 CHECK 用数据库文本比较。当前证据 Id 是固定的 ASCII 标识，两者在这个范围内一致；若开放任意多语言标识，应重新评估排序、排序规则与唯一性。编译依赖被隔离，并不意味着跨层语义不需要合作。

2.3 与固定 CleanArchitecture 模板比较

Jason Taylor 的 CleanArchitecture 在固定提交中提供 MediatR 请求处理器、行为管道、应用 DbContext 接口和保存拦截器。它是一组可研究的机制，不是必须逐项复制的目录规范。下面的判断来自具体文件，而非项目名称。固定上游版本

固定模板比较：机制与当前需要

上游请求管道	EchoCase 显式路径	改变选择的条件
MediatR + 多种 Behavior	Service 方法 → Write 端口	用例数量与横切逻辑增长
ApplicationDbContext 端口	领域根表达业务规则	跨聚合流程或外部通知
SavingChanges 拦截器	Store 组织本地事务	需可靠交付时再设计 outbox

图 2.2 比较固定提交中的机制；不从目录数量推导架构质量。

上游 UpdateTodoItem 处理器通过应用 DbContext 找到 Todo，再修改并保存。EchoCase 的相应用例通过根的 ReviseLink 修改关系，存储实现额外处理回执、版本和事务。这里的差异来自业务协议：一次调查修改需要稳定身份、重试确认和根版本仲裁。不能据此断言上游所有用例缺乏一致性，它的样例所表达的承诺不同。UpdateTodoItem 源码

EchoCase 没有引入通用调度器。当前命令数有限，显式服务方法让入口、操作名和回调可一起阅读。未来若出现大量独立用例与稳定复用的横切行为，可以重新评估行为管道；现在增加分派层只会多一个需要追踪的位置。判断标准是变更成本和可理解性，不是“没有 MediatR 就不算 CQRS”。

上游 SavingChanges 事件拦截器还揭示了更重要的时序问题：保存前发布事件，发生在 SQL 提交之前。若订阅者发送外部邮件，后续事务回滚不能撤回邮件。EchoCase 当前没有外部副作用，所以选择本地业务、审计和回执同事务，没有制造事件总线。将来需要可靠通知时，应明确提交记录和后续交付的关系，而不是把外部调用藏在保存钩子里。事件拦截器源码

2.4 依赖注入让选择成为运行事实

源码 backend/EchoCase.Api/Program.cs · L42-44

```
42  builder.Services.AddSingleton(TimeProvider.System);
43  builder.Services.AddScoped<IInvestigationStore, PostgresInvestigationStore>();
44  builder.Services.AddScoped<InvestigationService>();
```

API 在组合根注册接口到具体 Store 的映射，应用服务本身不 new 一个数据库实现。Scoped 表示同一请求解析到的服务链共享本次作用域中的实例；它不是进程级全局状态。DbContext 同样通过 AddDbContext 注册为请求作用域，跟踪本次工作单元。

TimeProvider 是单例，因为系统时钟本身不需要记录每个请求的可变业务状态，也便于测试替换。不能把这个生命周期选择推广给 Store 或 DbContext。若把跟踪上下文放进单例，两个请求就可能共享原始值和实体实例，破坏原本独立的工作单元；即使所有方法都是 async，也不会因此变成线程安全。



图 2.3 依赖注入只组装对象；数据库事务仍由 Store 显式开启和提交。

DI 只决定对象如何获得依赖，不决定数据库何时提交。事务在 Store 显式开始和提交，因此读者可以找到它的实际边界。把事务包装成不可见的通用属性，有时能减少重复，但也可能让某个用例是否跨多次保存变得难以辨认。此处选择明确，适合一次调查修改的规模。

2.5 目录隔离之后还要检查什么

生产项目引用测试项目，会让测试库进入真实依赖图；前端从测试目录导入固定数据，也可能把测试假设打进浏览器。目录命名不能阻止这种事情。EchoCase 的 test 独立拥有单元、集成、E2E 和检查工具，依赖方向是 Tests → Application。应用解决方案与测试入口分开，使生产构建有清晰边界。

这种架构检查不需要变成复杂治理平台。项目引用、生产包图和实际产物回答不同问题：源码没有显式引用，不代表传递依赖里没有测试库；包图正确，也不代表发布目录没有被复制进旧产物。验证应针对每种可能违反边界的路径。第十八章会进一步讨论如何让这些检查服务于可维护性。

架构的价值最终体现在可解释的变化上。关联修订增加领域行为、映射列和界面编辑器，却复用原有写入协议；应用层没有因为加两列而引用 EF。这样的结果比把每个操作机械拆成多个文件，更能说明依赖方向在起作用。

2.6 端口不是数据库可以随意替换的证明

IInvestigationStore 隐藏了 EF 类型，却承载了事务、回执和版本的业务保证。换一个实现时，仍需回答：同键操作是否串行，资源检查是否先于回执，根与子能否原子提交，旧回执如何恢复。只实现同样的方法签名，并不能获得同样语义。

这也说明为什么有些技术事实值得留在文档里。PostgreSQL 的事务级 advisory lock 和 READ COMMITTED 是当前保证的重要组成；应用层不编译引用它们，不代表维护者无需知道。架构隔离减少规则对技术 API 的耦合，技术推理仍必须进入真实实现。

一个完全内存的 Store 替身可以帮助研究用例如何构造 Change，却不能证明持久幂等和数据库竞争。测试可替换性有不同层次：接口替换便于组织测试，真实语义仍要在拥有该机制的边界验证。把“可 mock”当成架构目标，容易得到一套很好替换、却没有说明保证的接口。

2.7 业务域之外仍有应用工程

Identity、DbContext、迁移与 PostgreSQL Store 都位于 Infrastructure，因为它们是应用对持久化和身份的具体适配。这与云资源编排、镜像发行或环境部署是不同责任。按照名称删除 Infrastructure，会把合法的应用机制一并丢掉；按照责任阅读目录，才能保持边界清楚。

同样，前端不是后端的“最外层 DTO 展示器”。它拥有草稿与本地意图，是另一种需要认真建模的应用状态。后端聚合保证数据库事实，前端状态机保护未提交工作；两边通过 HTTP 合同合作，并没有一个通用实体类应该同时覆盖它们。

当团队变大，模块边界还会受协作和独立发布需求影响。当前四层规模足以让一条请求可追踪，新增更多项目不必然让并行开发更容易。先观察哪些决策经常一起改变，再选择拆分位置，比从组织架构直接复制目录更可靠。

第 03 章

聚合、身份与业务状态机

数据库可以保存任意一组字段，但业务只允许其中某些组合。调查在 Submitted 时必须有结论，关系的两端必须已经调查，复核中不能继续改笔记。DDD 在这里最有用的部分，是把这些约束交给一个能作出决定的对象，而不是把“领域”作为实体类所在的目录名。

3.1 为什么调查是聚合根

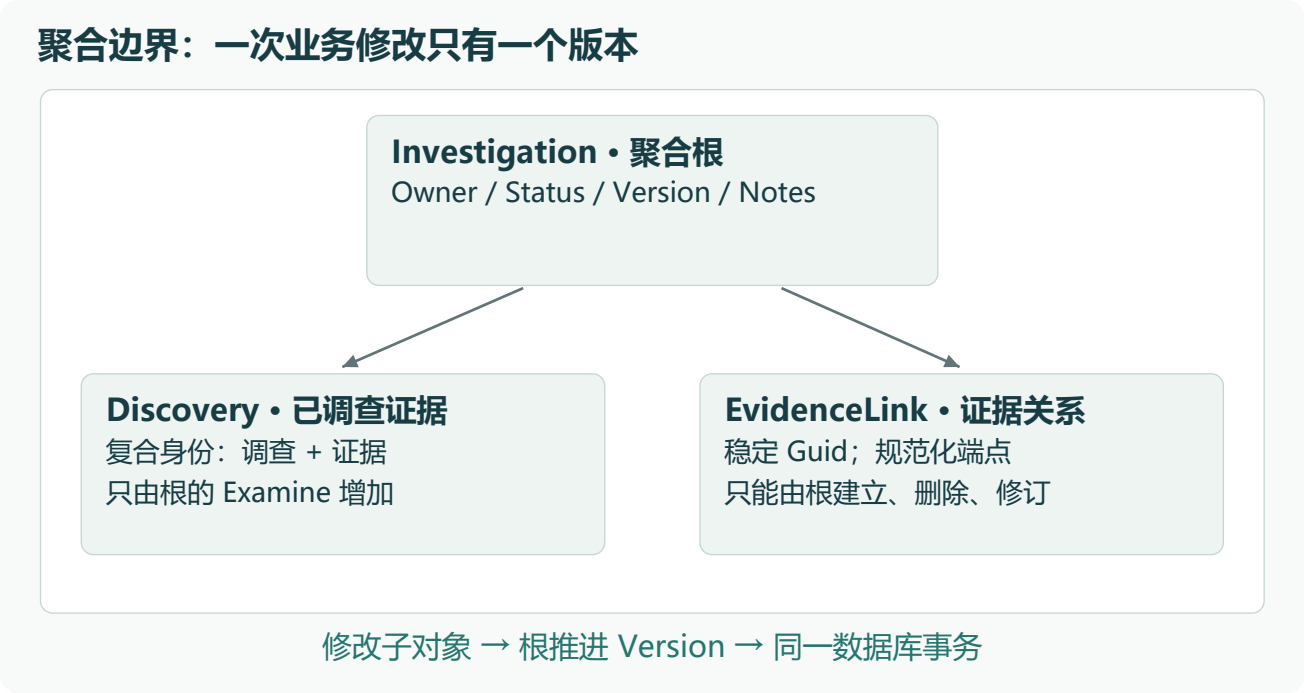


图 3.1 聚合不是数据库表的别名；根负责把跨成员规则组织成一次业务决定。

Investigation 的 Discoveries 和 Links 都只有只读集合入口，内部列表由根维护。外部可以观察成员，却不能直接 Add 或 Remove。EvidenceLink 的公开属性使用 private setter，修订方法也只在领域程序集内部可见。这个结构迫使业务调用表达“调查证据”“建立关联”“修订关系”，而不是任意设置字段。

聚合边界的依据是同一业务决定需要哪些事实一起判断。建立关系要同时看到两端是否已调查、是否已有该对关系，以及调查是否仍可编辑。若把关系做成一个完全独立的写入口，就必须把这些判断重新拼起来，还要解决检查之后其他修改发生的竞争。当前小型案件把它们放在一个根下，根版本成为统一的写入仲裁点。

这是有条件的选择。六条证据最多形成十五对无向关系，完整加载的成本受控。如果未来一个调查有几十万条证据，继续坚持所有成员必须完整加载，会把一致性边界变成性能瓶颈。那时应重新讨论哪些规则必须同步成立、哪些能由数据库约束或其他模型承担，不能只把 Include 改成分页后仍声称具有同样语义。

3.2 状态机表达允许的决定



图 3.2 复核需要至少三条已调查证据和两条关联；提交还必须满足支持证据与说明规则。

源码 backend/EchoCase.Domain/Investigation.cs · L96-109

```
96      public void Review(DateTimeOffset now)
97      {
98          Require(Status == InvestigationStatus.Active, "invalid_transition",
        "只有调查中的档案可以进入复核。");
99          Require(_discoveries.Count >= 3 && _links.Count >= 2, "insufficient_evidence",
        "复核需要至少三条已调查证据和两条关联。");
100         Status = InvestigationStatus.Reviewing;
101         Advance(now);
102     }
103
104     public void Resume(DateTimeOffset now)
105     {
106         Require(Status == InvestigationStatus.Reviewing, "invalid_transition",
        "只有复核中的档案可以返回调查。");
107         Status = InvestigationStatus.Active;
108         Advance(now);
109     }
```

Review 的条件不仅是当前 Active，还包括发现数与关系数。Resume 只允许从 Reviewing 返回 Active。两个方法都先校验，再改变状态，最后 Advance；因此规则拒绝时对象的版本与状态都不应变化。Submitted 没有返回路径，封存是当前业务中明确的终态。

界面可以根据同样事实禁用按钮，帮助用户避免无效操作；领域对象仍要自己检查。过期页面可能还保留一个可点击按钮，未来其他入口也可能直接调用服务。把校验仅放在组件里，会让规则依赖某个特定客户端是否诚实、是否及时刷新。

Advance 在每次成功修改后增加 Version 并更新 UpdatedAt。版本属于整个调查，不只属于笔记或某条关系。两个窗口即使修改不同成员，仍然竞争同一个根版本。这比较保守，却让“我基于哪个调查事实作出判断”具有简单含义。需要更细并发粒度时，必须同时说明不同修改为什么能够安全合并。

3.3 无向关系的身份与规范形式

源码 backend/EchoCase.Domain/Investigation.cs · L52-62

```
52      public void Link(string source, string target, LinkKind kind, DateTimeOffset now)
53      {
54          Editable();
55          Require(Enum.IsDefined(kind), "invalid_relation", "关联类型无效。");
56          Require(source != target, "self_link", "证据不能关联自身。");
57          Require(new[] { source, target }.All(id => _discoveries.Any(x => x.EvidenceId == id)),
 ·       "unexamined_link", "只能关联已调查的证据。");
58          if (string.CompareOrdinal(source, target) > 0) (source, target) = (target, source);
59          Require(!_links.Any(x => x.SourceId == source && x.TargetId == target),
 ·       "duplicate_link", "此对证据已有关联。");
60          _links.Add(new(Id, source, target, kind));
61          Advance(now);
62      }
```

函数先拒绝非法枚举、自连和未调查端点，再把 source、target 按 ordinal 顺序规范化。于是 A-B 与 B-A 都落到相同顺序，重复判断和数据库唯一索引可以使用同一对字段。技术 API 仍叫 SourceId、TargetId，但业务没有箭头方向。第十六章会说明这个差别如何影响 React Flow 的映射。

关系自身有 Guid。端点说明“连接哪两条事实”，Guid 说明“这是哪一条关系对象”。如果修订解释时删除旧关系再创建新关系，界面选中项、历史引用和客户端判断都可能失去连续性。保持身份让修订具有可追踪含义，也使测试能明确断言 Id 与端点不变。

3.4 关联修订是一个完整行为

源码 backend/EchoCase.Domain/Investigation.cs · L73-86

```
73      public void ReviseLink(Guid id, LinkKind kind, string rationale, DateTimeOffset now)
74      {
75          Editable();
76          Require(Enum.IsDefined(kind), "invalid_relation", "关联类型无效。");
77          var link = _links.SingleOrDefault(x => x.Id == id);
78          Require(link is not null, "link_not_found", "关联不存在。");
79          var explanation = rationale.Trim();
80          Require(explanation.EnumerateRunes().Count() is >= 20 and <= 500,
81              "rationale_length", "关联解释需要 20 至 500 个字符。");
82          Require(link!.Kind != kind || link.Rationale != explanation,
83              "link_unchanged", "关联类型和解释均未改变。");
84          link.Revise(kind, explanation, now);
85          Advance(now);
86      }
```

ReviseLink 的执行顺序值得逐项读。Editable 先保护状态；枚举检查防止构造出未定义值；随后按 Id 找到当前聚合成员。解释经 Trim 后使用 EnumerateRunes 计数，二十至五百的限制针对 Unicode 标量值。一个增补平面字符在 UTF-16 中可能占两个 code unit，这就是不能直接用 string.Length 代替此处语义的原因。

接着，方法拒绝类型和解释都没有变化的请求。这个规则避免“无意义修订”消耗一个新版本。它只适用于这个行为：当前 SaveNotes 即使内容相同，仍会推进版本；不能从一项规则推导所有命令都具有自动 no-op 识别。统一术语不等于各操作必须有相同策略。

最后才对子对象赋值并推进根。所有校验都发生在这之前，因此正常的业务拒绝不会先修改半个对象。数据库仍可能在保存时拒绝并发写入；那是持久化边界的失败，不能靠纯内存校验提前消除。领域对象保证单次决定的合法性，事务保证决定如何与其他请求共存。

3.5 提交结论为何不只是更新状态

源码 backend/EchoCase.Domain/Investigation.cs · L111-121

```
111     public void Submit(string suspect, string reasoning, IReadOnlyCollection<string> support,
        DateTimeOffset now)
112     {
113         Require(Status == InvestigationStatus.Reviewing, "invalid_transition",
        "先完成调查并进入复核。");
114         Require(CaseCatalog.Harbor.Suspects.Contains(suspect), "unknown_suspect",
        "请选择案件中的相关人员。");
115         Require(reasoning.Trim().Length is >= 20 and <= 4000, "reasoning_length",
        "结论说明需要 20 至 4000 字符。");
116         Require(support.Distinct().Count() >= 3 && support.All(id => _discoveries.Any(x =>
        x.EvidenceId == id)), "invalid_support", "结论需要至少三条不同的已调查证据。");
117         Suspect = suspect; Reasoning = reasoning.Trim(); SupportingEvidence = string.Join(',',
         support.Distinct().Order());
118         Score = CaseCatalog.Evaluate(suspect, support);
119         Status = InvestigationStatus.Submitted;
120         Advance(now);
121     }
```

Submit 将嫌疑人、说明和支持证据一并决定。support 需要至少三个不同值，而且每个值都必须属于已调查证据；重复同一证据三次不能满足数量。保存时去重排序，使持久表示具有稳定形式。评分来自固定 CaseCatalog 规则，随后状态改成 Submitted，版本前进。

注意说明长度这里使用 Trim 后的 UTF-16 长度，而关联解释使用标量值计数。这是当前合同的实际差异。本书不会把现有实现描述成统一的“用户可见字符数”规范；组合字符、表情序列与 grapheme cluster 又是另一层问题。工程师应首先识别实际计数单位，再判断未来是否需要一致化。



图 3.3 真实封存界面：结论、支持证据与固定规则评分一起展示。

①提交后的读模型已经包含 Verdict。

②100 分属于固定教学案件规则，并非现实调查结论的可信度。

封存界面从 VerdictView 读取已确认的结果，用户可以回看结论与支持材料。它不是在浏览器独立计算一个“看起来完成”的状态。状态机、持久事实和读模型保持同一语义，界面才有理由显示“结论已归档”。

3.6 聚合没有替代数据库

直接 SQL 可以绕过 Editable，数据库也没有完整复制每条状态转移规则。因此应用必须保持业务写入入口一致。与此同时，外键、唯一键、CHECK 和版本条件依然必要，它们防御数据竞争、映射缺陷与意外写入。领域与数据库的职责相互支撑，任何一层都不能仅凭另一层存在而省略自己的保证。

把领域建模做得恰当，意味着对象方法能够直接解释用户决定，状态转移可被推理，成员身份有稳定含义，失败不会留下难以理解的中间状态。在 EchoCase 中，这些性质比引入更多 DDD 类型名更重要。

3.7 不变量有不同的覆盖范围

“不能自连”只需要两端 Id；“两端已经调查”需要成员集合；“复核后禁止编辑”需要根状态；“结论支持必须来自已调查证据”同时涉及状态与多个成员。把这些判断交给根，可以在一个内存图中完成一致决定，也使方法名成为业务语言。

数据库局部 CHECK 能很好表达无自连和枚举值域，却不适合直接复制所有跨表状态判断。反过来，根可以在内存中检查重复，但面对真实并发仍需数据库唯一约束与版本。每条不变量都应标出其观察范围和仲裁位置，这比笼统说“所有校验都放领域”更可执行。

Version 大于零、Submitted 与 Score 非空的等价条件，是持久模型可以廉价保护的部分。但 Score 非空不能证明嫌疑人合法、说明充分或支持材料已调查。数据库中的有效行集合仍大于完整领域允许的行集合，正常业务入口承担两者之间的差额。

3.8 Entity 与值语义不必制造额外类型

EvidenceLink 是实体，因为同一关系会随时间被修订，业务关心身份连续性。端点对具有值语义，规范化后相同两端代表同一无向对；解释是一段受限制文本，当前没有独立生命周期。代码可以直接表达这些差别，不必为了术语齐全把每个 string 都包成一个类。

什么时候值得引入专门值对象？当一个概念有稳定且反复使用的校验、比较、规范化和组合规则，并且这些规则在多个地方容易漂移时。当前固定 ASCII 证据 Id 与有限枚举的复杂度受控；若未来开放自定义证据、跨案件引用或更复杂关系语义，应再评估类型边界。

无向图的上限也来自规则，而非调优参数：六个不同节点的无自连、无重复对最多是 6×5÷2，也就是十五条关系。这个业务上限随后支持完整聚合加载和简单画布呈现。领域分析因此能为数据库与前端提供可量化的设计条件。

3.9 内存原子性与持久原子性

ReviseLink 先完成所有可预知规则检查，再赋值。这样，DomainRule 拒绝时无需反向恢复一串已改属性。若在赋值之后又做可能失败的业务检查，就要认真考虑对象是否已经处于部分变化状态，即使数据库尚未保存。

这种内存顺序只覆盖本次对象操作。另一个请求可以同时作出不同决定，数据库也可能拒绝约束；此时需要事务回滚持久变化，并结束失败上下文。不要要求一个领域方法独自解决跨进程竞争，也不要因为存在事务而允许领域方法留下难以继续推理的半状态。

第 04 章

一次请求在 ASP.NET Core 中经历什么

修订关联的端点只有一行，却不是请求处理的全部。请求可能在端点执行前因身份、防伪或格式失败而停止；端点返回的错误对象还会经过结果执行器和 ProblemDetails writer；数据库异常则沿异步调用栈返回外围中间件。理解这些阶段，才能判断某段代码是否真的有机会执行。

4.1 从委托嵌套理解管道

ASP.NET Core 中间件可以在调用 next 之前工作，也可以等待 next 返回后继续工作，或者直接短路。它更像一层层嵌套的函数调用，而不是每个模块只执行一次的直线列表。EchoCase 的外围中间件先写安全头与 TraceId，再 try/await 后续处理，最后记录状态与耗时。中间件机制



图 4.1 此图按 Program.cs 的真实顺序绘制；授权短路时端点不会被执行。

UseRouting 选出端点，使后续组件能读取该端点的授权和限流元数据。Authentication 根据 Cookie 构造当前用户，限流才能按 NameIdentifier 分区；Authorization 再检查端点是否允许这个身份继续。若没有有效身份，受保护的业务端点不会执行。

防伪中间件位于授权之后，检查 /api 下除 GET、HEAD、OPTIONS 之外的请求。它也会保护登录、注册等匿名写入；匿名不是可以任意从其他站点发起状态变化的理由。路由元数据、当前身份和防伪令牌来自不同阶段，不能为了“看起来都是安全组件”随意交换顺序。

源码 backend/EchoCase.Api/Program.cs · L88-97

```
88  app.UseRouting();
89  app.UseAuthentication();
90  app.UseRateLimiter();
91  app.UseAuthorization();
92  app.Use(async (ctx, next) =>
93  {
94      if (ctx.Request.Path.StartsWithSegments("/api") && !HttpMethods.IsGet(ctx.Request.Method)
    .      && !HttpMethods.IsHead(ctx.Request.Method) && !HttpMethods.IsOptions(ctx.Request.Method))
95          await ctx.RequestServices.GetRequiredService<IAntiforgery>().ValidateRequestAsync(ctx)
    .      ;
96      await next();
97  });
```

静态文件与 SPA 回退随后处理界面资源。业务路由通过 MapGroup 要求授权，OpenAPI 只在开发或测试环境映射。API 未匹配路径有专门的 404 回退，避免把前端 index.html 当成 API 的成功 JSON。浏览器请求失败时，“收到 200 但正文是 HTML”往往意味着路由边界出了问题；状态码与媒体类型都应进入判断。

4.2 端点参数从哪里来



图 4.2 参数绑定组装调用；合法形状不等于业务合法，也不等于资源可见。

Minimal API 的处理器接收 Guid id、linkId、ReviseLinkRequest、InvestigationService、ClaimsPrincipal、HttpContext 和 CancellationToken。它们并非来自同一个地方：路由提供标识，JSON 提供正文，DI 提供服务，认证提供身份，宿主提供请求上下文和取消信号。编译器看到的只是参数列表，运行时绑定器负责把来源接起来。

源码 backend/EchoCase.Api/Program.cs · L137-137

```
137 api.MapPut("/investigations/{id:guid}/links/{linkId:guid}", async (Guid id, Guid linkId,
    .     ReviseLinkRequest body, InvestigationService s, ClaimsPrincipal p, HttpContext ctx,
    .     CancellationToken ct) => Respond(ctx, await s.ReviseLink(id, Owner(p), Key(ctx), Version(
    .     ctx), linkId, body, ct))).Produces<SessionView>().RequireRateLimiting("writes");
```

这一行的长处是协议到用例的映射可见：Owner 只取可信 claims，Key 从头中取，Version 解析 If-Match，再交给 Service。它的短处是可读宽度很大，适合源码导航，不适合把整个 Program 打印在书上。书中保留这一行是为了辨认边界，核心规则应继续读对应服务和根方法。

依赖由每次请求作用域解析。异步等待不会自动结束作用域，DbContext 仍属于同一次请求。CancellationToken 向下传递，让被放弃的读取有机会停止；但取消不是撤销已提交事务的协议。客户端断开连接后，数据库可能已经提交，这正是第十一章讨论结果未知的起点。

4.3 严格 JSON 是可演进合同的一种选择

源码 backend/EchoCase.Api/Program.cs · L41-41

```
41  builder.Services.ConfigureHttpJsonOptions(o => { o.SerializerOptions.RespectNullableAnnotation
    s = true; o.SerializerOptions.RespectRequiredConstructorParameters = true;
    o.SerializerOptions.NumberHandling = System.Text.Json.Serialization.JsonNumberHandling.Str
    ict; o.SerializerOptions.UnmappedMemberHandling =
    System.Text.Json.Serialization.JsonUnmappedMemberHandling.Disallow; });
```

配置启用必需构造参数与可空注解约束，拒绝未知成员，并保持数字处理严格。这样，拼错的字段不易被默默忽略，传 null 给非空参数也不会顺利流入后面的领域代码。它改善的是请求形状的可靠性，不能证明字符串内容符合业务规则。

严格未知字段也会收紧兼容范围。一个新客户端向旧服务发送新字段时，旧服务可能拒绝请求。因此演进需要明确客户端与服务端的先后兼容，而不是笼统承诺“加字段永远安全”。旧回执反序列化则走另一套默认 JsonSerializer 选项和可选 DTO 参数；不能把 HTTP 绑定配置直接套到所有 JSON 场景。

正文还受到 Kestrel 的 16 KiB 限制。这个上限限制的是请求字节，不是 Unicode 字符数；一个字符在 UTF-8 中可能占多个字节。域层的笔记长度限制和传输上限分别约束不同资源。满足字符限制的输入，仍可能在传输层被拒绝，界面需要有合理反馈。

4.4 错误如何变成公开响应

源码 backend/EchoCase.Api/Program.cs · L70-80

```
70          var (status, code, detail) = e switch
71          {
72              ApplicationFault f => (f.Status, f.Code, f.Message),
73              DomainRule d => (422, d.Code, d.Message),
74              AntiforgeryValidationException => (400, "csrf_invalid",
    "防伪令牌缺失或失效。请刷新页面。"),
75              BadHttpRequestException b => (b.StatusCode, "invalid_request", "请求格式不正确。"),
76              NpgsqlException => (503, "database_unavailable", "数据服务暂不可用。请稍后重试。"),
77              InvalidOperationException { InnerException: NpgsqlException } => (503,
    "database_unavailable", "数据服务暂不可用。请稍后重试。"),
78              DbUpdateException => (503, "write_failed", "保存未完成。请刷新后重试。"),
79              _ => (500, "internal_error", "服务发生错误，请使用追踪编号排查。")
80          };
```

ApplicationFault 携带已定义的 HTTP 语义，DomainRule 映射为 422。数据库连接失败转成 503，未识别异常则返回通用 500。对外 detail 不直接拼入数据库异常原文，减少泄露 SQL、连接信息或内部类型的风险；服务端仍保留异常用于定位。

错误边界只在 Response.HasStarted 为 false 时重新生成正文。如果响应已开始，再把前面写出的字节当作不存在，会产生另一份格式错误的响应。当前应用主要返回有限 JSON，不涉及复杂流式输出，所以这个限制容易理解；若未来加入文件流或 SSE，就需要重新分析发送过程与失败语义。

错误映射也不能证明所有取消和提交确认问题都被穷尽处理。当前客户端对 status 0 的网络和解析失败有专门保护，但一般 5xx 没有统一的持久恢复协议。工程师要区分“内部异常被转成友好文字”和“用户意图已得到确定结论”。前者是错误呈现，后者需要事务与回执协同。

4.5 TraceId 案例：结果执行发生在构造之后



图 4.3 ASP.NET Core v10.0.9 的真实顺序；最终值的所有权决定修复应放在哪一阶段。

一次真实的业务拒绝返回 422，响应头的 X-Trace-Id 为 de914ead377f15db9909ac49e0dfa6ee，正文却曾出现 00-de914ead377f15db9909ac49e0dfa6ee-3070f240a46434ed-00。两者共享同一 trace 部分，但不是同一个字符串。对于要求用户复制编号的界面，这种差异足以让错误检索变得困惑。

原因不在数据库，也不在浏览器展示。ASP.NET Core 的默认 writer 在执行 ProblemDetails 时先设置 Activity.Id，它是完整 W3C 活动标识；应用头部使用 Activity.TraceId。先在 Results.Problem 的 extensions 中填一次 traceId，仍可能被后续 writer 覆盖。固定版本源码中可以看到“先写默认扩展，再调用定制钩子”的顺序。默认 writer 源码

源码 backend/EchoCase.Api/Program.cs · L46-51

```
46  builder.Services.AddProblemDetails(o => o.CustomizeProblemDetails = context =>
47  {
48      // The default writer sets Activity.Id (the full W3C parent string) first.
49      // Keep the public error identifier equal to our response/log TraceId.
50      context.ProblemDetails.Extensions["traceId"] = Activity.Current?.TraceId.ToString() ??
    .     context.HttpContext.TraceIdentifier;
51  });
```

修复放在 CustomizeProblemDetails，让最终正文采用与响应头相同的 TraceId。这个案例说明，框架调用的生命周期比“我已经给对象赋值”更重要。构造结果、选择 writer 和真正写入响应是不同事件；应在拥有最终值的阶段统一合同。

4.6 用一条请求检查自己的理解

若一个有效登录用户提交旧版本的修订，管道先确认身份和防伪，再绑定正文，端点解析版本，Store 加载当前调查后拒绝为 412，外围错误边界生成 ProblemDetails。若缺少登录，授权就会先停止处理，领域方法没有机会拒绝版本。两个错误不会随意“同时返回”，先到达的有效边界决定公开结果。

理解这种短路和回返关系，能避免把每个故障都归咎于“接口调用失败”。请求管道给业务路径建立外部条件，领域与事务再决定业务事实是否成立。接着还需辨认端点的构造时刻，以及异步交付的边界。

4.7 启动时构造处理器，请求时填入数据



图 4.4 处理器尚未进入，错误也可能已经形成；返回结果之后，字节仍需写出并被解析。

MapPut 在应用构建阶段登记路由模式、处理委托和元数据。框架据此形成请求处理路径；请求到达后才匹配具体地址、取得路由值、读取正文并解析参数。登记了一个端点，不意味着它在启动时执行了领域代码，也不意味着所有路由值都能成功绑定。Minimal API 参数绑定

路由上的 guid 约束可使某些地址不匹配该端点；JSON 绑定又可能在处理器调用前失败；处理器里的 Version 则是应用显式编写的头解析。这三个阶段都可能涉及“输入不合法”，但发生位置和公开结果不一定相同。调试时应先确定端点是否匹配、处理器是否进入，再向内追踪，而不是只在领域方法入口打断点。

DI 参数也需要区分定义与实例。注册 IInvestigationStore 是启动配置，当前请求实际解析哪个对象由作用域决定；请求中的 CancellationToken 来自宿主，不是请求正文里的一个字段。将这些来源拆开，能避免把客户端输入、可信上下文和程序依赖混为一谈。

4.8 await 维持调用关系，不提供业务原子性

当端点 await Service，Service await Store，Store await 数据库，线程可以在等待期间服务其他工作。局部调用的逻辑先后仍保持，但不能从“都在一条 async 链”推导没有竞争。另一个请求拥有独立上下文，可以在同一等待时间内推进相同调查。

同样，取消信号只是协作请求。查询开始前观察取消、数据库执行中响应取消、提交之后连接中断，具有不同后果。HTTP 客户端不能从自己的等待结束推断远端事务一定回滚。API 需要在成功路径等到提交；客户端需要在没有确定结果时保留原意图。异步模型与幂等协议在这里接上。

DbContext 也不能因 async 就同时执行多个查询。当前 Store 顺序 await 锁、加载、回执和保存，保持一个上下文中的操作次序。为了“节约一点时间”把这些依赖任务放入 Task.WhenAll，不仅可能违反上下文并发使用要求，还会破坏回执检查与资源授权的先后语义。

4.9 返回值、响应状态和用户看到的结果

端点返回 IResult 后，结果执行器才真正写入头与正文。Respond 设置 ETag，并构造 Ok 结果；成功对象的存在不等于浏览器已经收到全部字节。ProblemDetails 案例也显示，结果对象在写出阶段仍可能被框架补充字段。

可以把一次成功拆成三个承诺：数据库提交成功，服务端开始交付成功结果，浏览器解析并接纳结果。前两者发生后，第三者仍可能失败。服务器无法在同一个 PostgreSQL 事务里强制浏览器完成解析；客户端也不能因为看不到最终结果就否认服务器提交。回执把这段不可消除的缺口转成可以再次询问的确定记录。

这条因果链让请求管道不再只是中间件顺序图。它解释了为什么错误映射要在最终 writer 统一编号，为什么响应已开始不能重写正文，以及为什么可靠保存需要一直考虑到浏览器确认。

第 05 章

用例服务与逻辑 CQRS

CQRS 最直接的价值，是让“想知道什么”和“想改变什么”有不同语义。查询可以被取消、缓存和重复读取；命令携带用户决定，需要判断合法性、版本和重复尝试。EchoCase 让两者使用不同方法和请求类型，但读写仍共用数据库与返回视图。理解这个具体范围，比先决定引入哪种分派库更有帮助。

5.1 一个服务协调什么

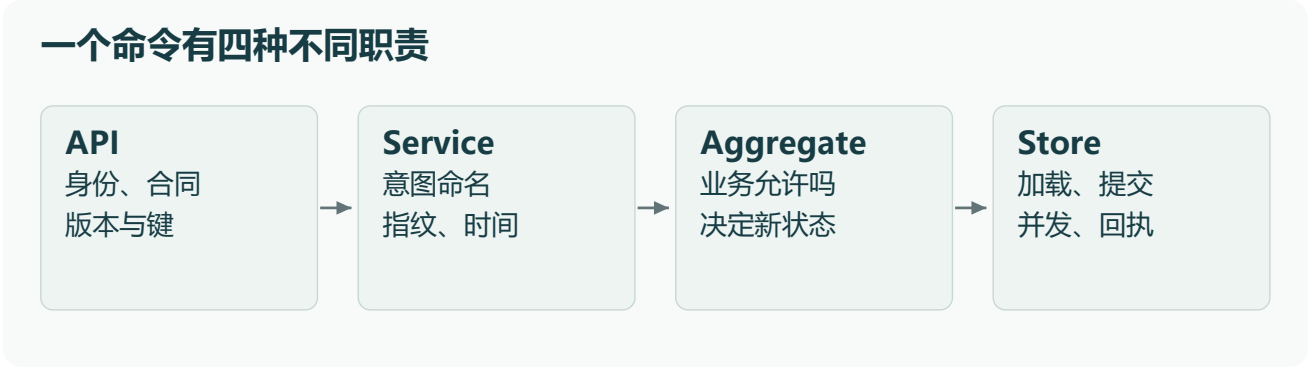


图 5.1 逻辑 CQRS 分离读写意图，读写仍然使用同一 PostgreSQL 数据库。

InvestigationService 的方法名对应用户意图：Start、Examine、Link、ReviseLink、Notes、Transition、Verdict。每个方法构造所需参数，再通过共享 Write 建立变更上下文。它没有直接保存数据库，也不在自身复制完整领域状态机。

源码 backend/EchoCase.Application/Investigations.cs · L62-69

```
62      public Task<SessionView> ReviseLink(Guid id, string owner, string key, int version, Guid
        linkId, ReviseLinkRequest body, CancellationToken ct) =>
63          Write(owner, key, id, version, "revise-link", new { linkId, body }, i =>
64          {
65              if (!Enum.TryParse<LinkKind>(body.Kind, false, out var kind) || !Enum.IsDefined(
        kind) || kind.ToString() != body.Kind)
66                  throw new DomainRule("invalid_relation", "关联类型无效。");
67              i!.ReviseLink(linkId, kind, body.Rationale, Now());
68              return i;
69          }, ct);
```

ReviseLink 把 linkId 与 body 一起纳入操作内容，操作名固定为 revise-link。回调在 Store 完成加载和协议检查后才执行；此时 i 应当是可信 owner 范围内的调查。回调把字符串枚举转成领域枚举，并调用根方法。这个端口让应用服务表达要做什么，把事务何时成立交给持久层。

PUT 修订端点要求 Kind 精确等于枚举名称。Enum.TryParse 可能接受数值形式字符串，所以额外的 ToString 比较具有实际意义。现有 POST 建联路径没有这一项比较，仍保留原合同的较宽接受范围。本书把差异写出来，因为维护系统常常需要兼容已有行为；不能为了描述整齐而假装两个入口已经完全相同。

5.2 共享写路径是一段协议

源码 backend/EchoCase.Application/Investigations.cs · L91-98

```
91      private Task<SessionView> Write<T>(string owner, string key, Guid? id, int? version,
        string operation, T body, Func<Investigation?, Investigation> apply, CancellationToken ct)
92      {
93          if (!Guid.TryParseExact(key, "D", out _)) throw new ApplicationFault(400,
        "idempotency_key_required", "写操作需要 UUID 格式的 Idempotency-Key。");
94          var canonical = JsonSerializer.Serialize(new { id, version, operation, body });
95          var hash = Convert.ToHexString(SHA256.HashData(Encoding.UTF8.GetBytes(canonical)));
96          return store.WriteAsync(new(owner, key, hash, id, version, operation), apply, ct);
97      }
98  }
```

Write 首先要求标准 D 格式的 UUID。随后对包含 id、version、operation、body 的应用对象序列化，再计算 SHA-256。指纹不包含任意原始请求字节，JSON 属性空白或 HTTP 头顺序不决定意图身份；但版本、路由语义和正文内容都进入比较。这是当前 DTO 和序列化路径中的规范形式，不是对任意 JSON 的跨语言规范化标准。

Key 的字符串本身会按收到的形式持久化，并没有先统一 UUID 大小写。新客户端生成的新键具有稳定形式，原样重试不会出问题；但不能据此声称所有文字形式相同的 UUID 都自动折叠为一个业务键。要改变这项行为，需要一起考虑旧回执查找和兼容策略。

Change 把 owner、key、fingerprint、sessionId、expectedVersion 和 operation 传给 Store。它不只是一个“保存参数集合”，而是一次用户意图的身份与前提。第十一章会解释，为何重试必须保留旧 expectedVersion，而不能为了通过检查偷偷改用最新版本。

5.3 校验分层的理由



图 5.2 各层守住自己的假设；界面校验改善反馈，不能替代服务端规则。

API 负责请求能否绑定；应用层负责用例合同的类型转换；领域根负责状态与内容规则；数据库负责持久事实的关系约束及竞争仲裁。它们可以共同约束一条请求，但不必都写相同的 if。客户端的二十字提示是及时反馈，领域中的计数是业务保证，数据库 CHECK 是最终持久约束，各自应与对应单位保持一致。

分层不意味着所有错误都提前发现。例如即使领域已检查无重复关系，另一个请求仍可能竞争相同数据。Store 的事务和根版本承担这一窗口里的判断。若为了避免数据库错误而在应用层先做一次额外查询，仍然没有消除查询与保存之间的竞争，反而可能引入另一个过期观察。

5.4 查询模型与命令结果为什么相同

SessionView 是完整调查视图，查询和成功命令都返回它。这样，命令成功后界面可以立即获得新的版本、状态和关系，不必凭请求正文自行猜测服务器规范化后的内容。Trim 后解释、评分和解锁状态，都以服务端视图为准。

相同形状不代表相同时间语义。Get 描述一次读取快照；回执重放返回过去某次命令成功时的 SessionView。这个差别会直接影响前端缓存：旧回执即使 HTTP 200，也不能使确认缓存从版本 8 回到版本 5。CQRS 的主要区分在意图与时间，而不只是 DTO 是否使用不同类名。

当前没有异步投影或读写库分离，因此命令提交后针对同一数据库的新查询可以读到已提交事实，不存在额外消息队列引起的投影延迟。仍然存在网络乱序、并发新写和语句快照差异；“没有最终一致读模型”不等于“所有读取永远最新”。

5.5 时间依赖也是用例输入

源码 backend/EchoCase.Application/Investigations.cs · L85-89

```
85      private DateTimeOffset Now()
86      {
87          var utc = clock.GetUtcNow();
88          return utc.AddTicks(-(utc.Ticks % TimeSpan.TicksPerMicrosecond));
89      }
```

Service 从 TimeProvider 获取 UTC 时间，并把不足微秒的 tick 尾数去掉，再交给领域。这样，业务对象、命令回执和数据库重读使用可一致表达的时刻。测试可以替换时钟，讨论规则时不必依赖实际等待。

审计和回执自己的 CreatedAt 由 Store 的时钟获取，并非完全同一个局部 now。它们记录相邻的持久操作时刻，不能被当作领域 UpdatedAt 的逐 tick 同义字段。这也是为何数据建模应说明时间字段各自代表哪个事件，而不是只给所有字段起一个“时间戳”的统称。

5.6 何时值得拆分更细

如果每个新用例都在服务中复制授权、验证、日志和事务，维护成本会发出信号。可以引入独立处理器或组合行为，但要先确定需要复用的语义。当前共同部分是一个数据库写协议，显式 Write 足以集中处理；领域规则仍各自清晰。

相反，若把每个三行方法拆成请求、处理器、验证器、映射器和注册器，却没有独立变化原因，读者追踪一项业务决定就要跳更多文件。架构粒度应随职责和复杂度增长，不应由术语预先规定。这个小服务展示的是一种有边界的协调方式，而非所有企业项目都必须采用的最终模板。

5.7 回调端口的能力与约束

WriteAsync 在完成锁、资源加载和回执检查后调用 apply。回调接收的是本次上下文已加载的聚合，新建调查时才是 null；应用方法在既定路径下使用 i!。这个非空断言不是对任意调用的运行时验证，正确性依赖 Change 与用例入口共同建立的前提。

回调是同步函数，领域决定不在里面等待远程 API。这样，Store 能清楚包住一次有限业务修改，回执重放时也能完全跳过它。如果未来把外部调用或不可重复副作用塞进回调，锁与事务会被拖长，重试语义也会失效。接口签名允许传递代码，不表示任何代码都适合进入事务。

这种设计换来的可读性，是每个用例的领域调用与操作名相邻；代价是应用端口理解一个特定工作单元形状。大量跨聚合操作或复杂异步编排出现时，它可能不再合适。架构评审应保留这些改变选择的信号，而不是把现有简化永久规则化。

5.8 规范化发生在指纹之前还是之后

解释的 Trim 在领域内部执行，指纹却根据应用请求对象建立。因此同键请求从“前后有空格的解释”改成“去掉空格的解释”，即使最终领域文本可能相同，也会被视为不同请求而拒绝。这个行为保护了原意图的精确重放，不能被描述成语义等价去重。

若希望把某些差异规范化后再建立指纹，需要明确哪些差异被忽略，以及已有回执如何比较。比如更改枚举大小写接受规则，会同时改变绑定、领域转换和指纹含义。这样的调整属于合同演进，不能当成无关紧要的字符串清理。

共享 Write 的价值正是在一个地方暴露这类决定。所有写方法都携带相同的意图坐标，便于分析重放与冲突；业务规则仍留在根中，避免协议复用演变成通用数据修改器。

第 06 章

HTTP 合同中的版本、意图与失败

HTTP 把一次业务交互变成跨进程协议。客户端看到的是请求、响应和超时，不会自动知道事务是否提交。一个可靠的写接口因此需要表达三件事：操作哪个资源，基于哪个已知事实，以及这是否为同一次尝试。EchoCase 用资源路径、If-Match 和 Idempotency-Key 分别承载它们。

6.1 URI 与方法表达什么

调查是主资源，证据调查和关系修订围绕它组织。读取单个调查返回 SessionView，成功写入同样返回完整视图及 ETag。PUT 到已有 linkId 修改关系类型和解释，不替换端点；DELETE 删除关系；POST 表达建立调查、调查证据和状态动作。选择方法不能单独决定重试安全，实际业务协议仍要说明命令是否可能重复产生新版本。

EchoCase 不把整个实体暴露成一个通用 PATCH。客户端不能直接写 Version、OwnerId、Score 或 Status。进入复核、返回调查和提交结论有不同前置条件，因此需要语义明确的用例。这样的接口使“允许哪些变化”可从路径与请求类型读出，也减少了批量属性赋值绕过规则的机会。

6.2 If-Match 是决定的前提

源码 backend/EchoCase.Api/Program.cs · L147-154

```
147 static int Version(HttpContext ctx)
148 {
149     var value = ctx.Request.Headers.IfMatch.ToString();
150     if (value.Length < 3 || value[0] != '"' || value[^1] != '"' || !int.TryParse(value[1..^1],
      .      out var version) || version < 1)
151         throw new ApplicationFault(428, "precondition_required", "需要 If-Match: \"版本号\"。")
      .     ;
152     return version;
153 }
154 static IResult Respond(HttpContext ctx, SessionView value) { ctx.Response.Headers.ETag = $"\"{
      .     value.Version}\""; return Results.Ok(value); }
```

Version 要求一个带双引号的正整数。当前实现接受单个强样式版本标签，不支持弱标签、标签列表或星号。解析失败使用 428 表达缺少可接受前提。ETag 在成功响应中采用同样形式，因此客户端可以把已确认版本带回写请求。

这个版本是调查聚合的应用版本，不是数据库 xmin，不是修改时间，也不是 JSON 字节哈希。服务端先比较客户端前提与已加载版本，EF 保存时还会把跟踪原值放入 UPDATE 条件。前一次比较提供及时反馈，后一次比较关闭真实竞争窗口，两者缺一都不能完整说明并发行为。

运行摘录

```
If-Match: "4"
Idempotency-Key: <本次意图的 UUID>
```

上面是协议形状说明，尖括号不是实际发送值。如果用户基于版本 4 修订关系，而另一个窗口已提交版本 5，服务器拒绝此次新意图。它不能自动把请求“升级”为基于 5 的判断，因为用户从未见过版本 5 中的事实。

6.3 幂等键识别的是一次意图

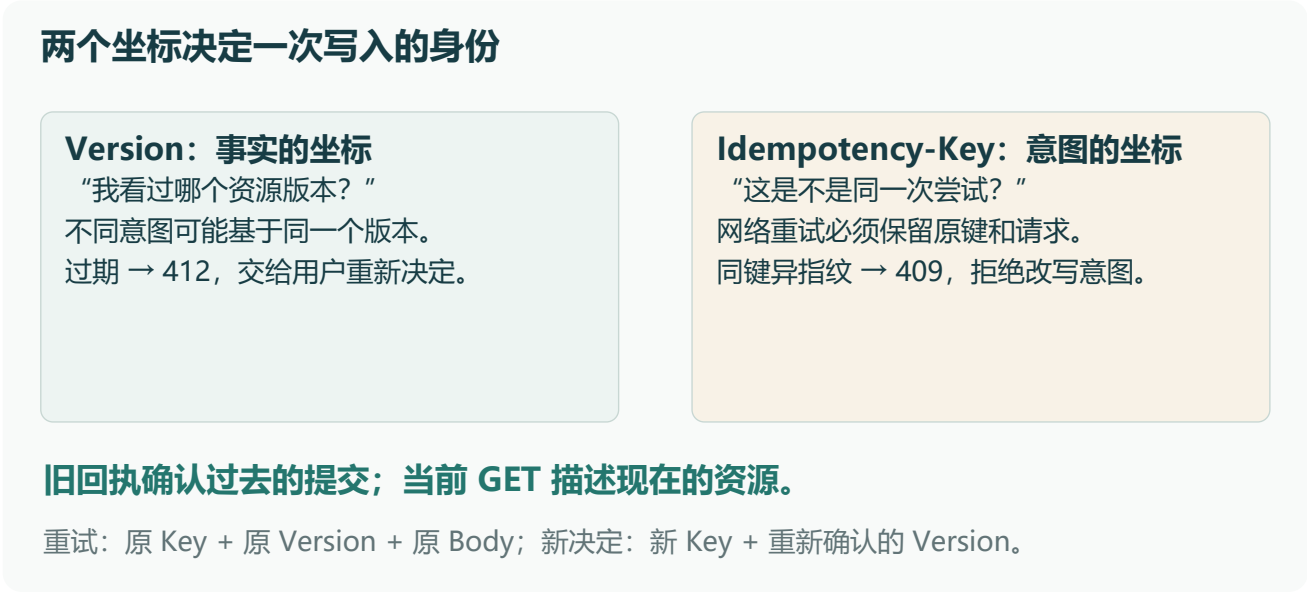


图 6.1 条件写入和幂等分别解决丢失更新与不确定重试；它们不能互相替代。

相同内容的两次点击未必是同一次意图。用户第一次保存后再次确认相同内容，系统可能把它当作另一次命令；网络重试则应保留原键。当前 SaveNotes 并非天然 no-op：正常成功会推进版本。因此，即使方法使用 PUT，客户端仍需使用应用幂等协议，才能在响应丢失时确认第一次提交而不额外执行。

同一 owner/key 的回执已存在且指纹相同，服务器返回历史结果。指纹不同则 409，表示客户端把一个键拿去表达另一项请求。若原键不存在但资源版本过期，则 412。409 关心意图身份，412 关心资源前提，二者不应被压成同一个“保存失败”。

6.4 错误分类应帮助下一步决定



图 6.2 状态码与客户端的“结果未知”属于不同观察层次。

400 表示请求形状或防伪令牌等问题，401 要求认证，404 统一资源不存在与不可见。422 说明请求进入业务后违反了规则；412 要求用户核对资源变化；409 要求修复幂等键使用。429 是请求频率约束，503 表示暂不可用的服务条件。客户端展示应保留有助判断的 code、detail 和 traceId，而不是根据所有非 200 结果统一清空表单。

“结果未知”却不是服务器返回的某一个状态码。fetch 失败时客户端没有完整响应，成功正文无法解析时也没有可使用的结果。当前客户端把这些情况包装为 status 0，以阻止继续发出不同写入，并允许原样重试。这是浏览器本地的分类，不是合法 HTTP 状态码，更不能在 API 文档里写成服务器会返回 HTTP 0。

6.5 读合同也需要边界

单个调查的查询同时筛选 Id 与 OwnerId，完整视图反映一个查询语句的快照。API 对 /api 设置 no-store，避免普通 HTTP 缓存把私人数据当成可共享资源。浏览器内 TanStack Query 仍可以管理应用层缓存，因为那是有身份和生命周期的 JavaScript 状态，不等同于共享代理缓存。

列表最多取五十个调查，按 UpdatedAt 倒序。它没有游标，也没有在相同时间戳时显式增加 Id 的稳定分页次序。当前界面因此是有限总览，而不是可遍历任意历史数据的完整分页合同。如果未来要求审阅全部旧档案，需要新增明确的分页语义，不能仅把 Take 的数字调大。

源码 backend/EchoCase.Infrastructure/PostgresInvestigationStore.cs · L18-23

```
18      private IQueryable<Investigation> Graph => db.Investigations.Include(x => x.Discoveries)
        .Include(x => x.Links).AsSingleQuery();
19
20      public async Task<IReadOnlyList<SessionView>> ListAsync(string ownerId, CancellationToken
        ct) =>
21          (await Graph.AsNoTracking().Where(x => x.OwnerId == ownerId).OrderByDescending(x =>
        x.UpdatedAt).Take(50).ToListAsync(ct)).Select(SessionPresenter.Map).ToArray();
22      public async Task<SessionView> GetAsync(Guid id, string ownerId, CancellationToken ct) =>
23          SessionPresenter.Map(await Graph.AsNoTracking().SingleOrDefaultAsync(x => x.Id == id
        && x.OwnerId == ownerId, ct) ?? throw Missing());
```

Get 与 List 使用 AsNoTracking，避免为了展示而维护写入快照。Graph 选择单语句加载，使根、发现和关系共享同一语句快照。一次读取仍可能在返回浏览器前就被新写入超越，所以界面显示的“当前版本”应理解为当前已确认快照，而不是所有客户端共同认可的绝对现在。

6.6 OpenAPI 与 TypeScript 提供怎样的保证

schema.d.ts 从 API 合同生成，前端通过 components 的 schema 类型引用 Session 和 ReviseLinkRequest，减少手写字段漂移。它能在构建时发现字段名或可空类型不匹配，却不会在运行时验证 response.json 的每个值。泛型 request 是类型承诺，不是 JSON schema 校验器。

同样，自动文档不能完整表达 Idempotency-Key 的持久回执语义，也不能替代对 If-Match 接受子集的说明。机器可读合同处理形状，人类可读合同解释时间和决定。二者配合，才能让一个客户端在失败后知道该保留原意图、重新认证，还是请求用户重新确认。

6.7 条件写入与条件读取不要混为一谈

响应带 ETag，不代表应用已经实现 If-None-Match 和 304 条件读取。当前代码在写端解析 If-Match，在成功响应中返回版本，并对 API 设置 no-store。它主要把标签作为写入前提，不是一套完整 HTTP 缓存协商实现。

同样，这份解析器是项目公开接受的有限形式，不是完整实体标签语法库。维护者若打算支持列表、星号或弱标签，需要先定义它们如何对应单个聚合版本，再调整解析和验证。不能只让字符串“能被 int.Parse”就假装支持全部标准条件头。

版本还不能跨资源比较。调查 X 的 8 与调查 Y 的 4 没有谁更新的全局含义；键和缓存必须同时包含资源身份。时间戳也不能替代这种身份范围，尤其当同一时刻创建多个独立调查时。

6.8 状态码之后还需要可理解的正文

412 告诉客户端前提失败，却没有自动完成用户的比较决定。界面仍需取得当前调查，保留本地草稿并解释基线差异。422 告诉客户端业务拒绝，却不能仅显示数字；稳定 code 适合程序分类，面向人的 detail 解释如何调整。

TraceId 的作用是定位本次请求，不是业务错误类别，也不是授权令牌。用户可以复制它帮助排查，但它不能替代明确 code，更不能据此访问另一个用户的资源。将诊断编号、错误语义和业务身份分别建模，接口才不会成为一团“字符串约定”。

HTTP 层的教学重点因此不止是方法和路径。它把用户决定的条件、服务器的可公开结论以及客户端下一步行动接起来。协议越明确，前端越少需要猜测，失败恢复也越容易保留正确意图。

第 07 章

EF Core 跟踪器如何把对象变化变成 SQL

关联修订在领域中只改变一个子对象，数据库却必须同时更新关系行和调查根版本。中间的桥梁不是 SaveChanges 这个方法名，而是 DbContext 所维护的实体身份、原始值、当前值、状态和关系。若无法解释这些内容，就很难分辨真正的并发冲突、错误映射和漏存修改。

7.1 一个工作单元中的两种读取

EchoCase 的 Get 和 List 是只读路径，采用 AsNoTracking；Write 则通过 Graph 加载带跟踪聚合。写路径需要记住根原来的 Version，后续 UPDATE 才能知道此次修改基于什么。只读展示不需要这些快照，所以选择减少跟踪负担。

DbContext 适合短生命周期工作单元。它不是所有数据的持续镜像，跟踪中的对象也不会因为其他连接提交而自动刷新。把同一个上下文长期缓存，可能一直复用旧实例和旧原值。当前请求作用域使读取、领域决定和保存局限在一条可见链中。跟踪概念

源码 backend/EchoCase.Infrastructure/EchoDbContext.cs · L51-54

```
51          i.HasMany(x => x.Discoveries).WithOne().HasForeignKey(x => x.InvestigationId)
    .OnDelete(DeleteBehavior.Cascade);
52          i.Navigation(x => x.Discoveries).UsePropertyAccessMode(PropertyAccessMode.Field);
53          i.HasMany(x => x.Links).WithOne().HasForeignKey(x => x.InvestigationId).OnDelete(
    DeleteBehavior.Cascade);
54          i.Navigation(x => x.Links).UsePropertyAccessMode(PropertyAccessMode.Field);
```

私有列表不妨碍持久化。模型告诉 EF 使用字段访问导航，框架在实体物化和关系修复时能填充内部集合；业务调用仍只能从只读视图观察成员。持久化可以理解对象结构，不要求领域为数据库开放公共 setter。这里的 Discovery 和 EvidenceLink 是普通实体关系，并非 EF OwnsMany 的 owned 类型，二者不要混淆。

7.2 身份解析先于变化比较

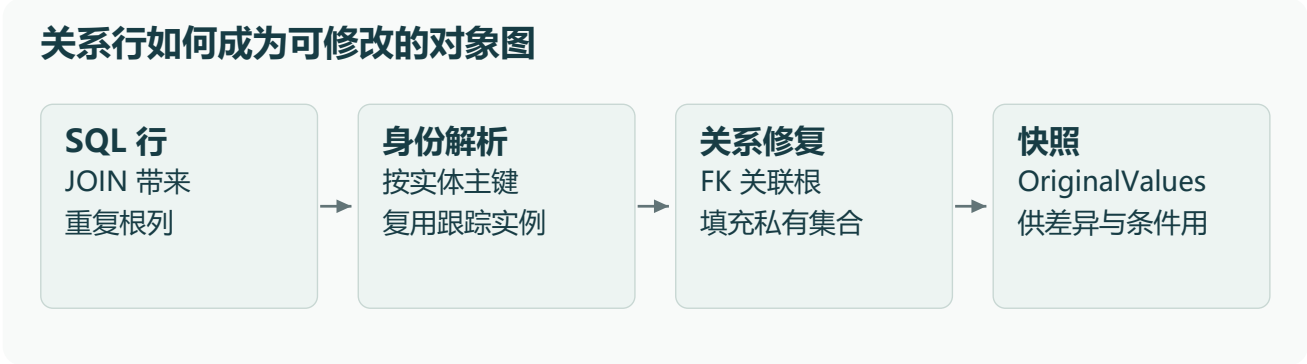


图 7.1 写路径加载带跟踪图；只读 Get/List 使用 AsNoTracking。

单语句 JOIN 可能让同一个调查的根列在多行中重复出现。带跟踪查询通过实体主键解析身份，同一个键在当前上下文中对应同一个实例；导航与外键再把发现和关系接回根。若没有这种区分，重复根行可能变成几个相互矛盾的对象，修改哪一个都不明确。身份解析

实体身份不等于对象引用地址。Guid 标识关系在持久世界中的连续性，引用标识本工作单元里正在修改的实例。将一个 DTO 重新 new 成同键实体，再把它附加到已经跟踪该键的上下文，会引入身份冲突，也容易丢失原值。EchoCase 选择先按 owner/id 加载真实图，再调用领域方法，避免让客户端提供一个任意“待附加实体图”。

关系修复也不负责业务规则。EF 能根据外键把某条 Link 放到根集合，却不知道复核中禁止修订，或解释为什么至少二十个字符。能物化某个对象图，只说明映射允许表达它，不说明这组事实已经通过所有领域判断。

7.3 当前值已经改变，状态为何仍是 Unchanged

默认快照跟踪在实体进入上下文时记录值，之后在变更检测中比较当前值。普通 C# 属性赋值并不必然立即通知跟踪器。领域方法返回后，CLR 中的 Version 已从 4 变成 5，关系类型也已变化；若尚未扫描，跟踪条目仍可能显示 Unchanged。快照与检测

同一对象，三个观察时刻

观察点	根 Investigation	子 EvidenceLink
刚加载	Unchanged Original Version = 4	Unchanged Original Kind = Corroborates
领域方法返回	CLR Version = 5 状态尚未扫描	CLR Kind = Contradicts 解释、时间已赋值
DetectChanges 后	Modified UpdatedAt、Version	Modified Kind、Rationale、RevisedAt

真实记录：保存后 Version = 5；旧版本对子实体的修改随失败事务回滚。

图 7.2 变更检测把 CLR 属性差异转为 EF 状态；实际记录保留了扫描前后的区别。

图中的状态来自真实关联修订观察。扫描之前，根与关系均为 Unchanged；显式 DetectChanges 后，根的 UpdatedAt、Version 标为 Modified，子关系的 Kind、Rationale、RevisedAt 标为 Modified。根其他属性没有因“保存整个聚合”就全部标脏。

这解释了一个容易误判的调试现场：看到 Unchanged 不能立刻说领域赋值失败，应同时看当前 CLR 属性、是否执行过检测，以及调试观察本身会不会触发检测。反过来，Modified 也不表示事务已经提交，它只说明 EF 准备产生持久化修改。对象变化、状态变化、SQL 执行和提交是四个时刻。

7.4 原始 Version 如何进入条件

源码 backend/EchoCase.Infrastructure/EchoDbContext.cs · L45-50

```
45          i.Property(x => x.Version).IsConcurrencyToken();
46          i.Property(x => x.Notes).HasMaxLength(4000);
47          i.Property(x => x.Reasoning).HasMaxLength(4000);
48          i.Property(x => x.Suspect).HasMaxLength(100);
49          i.Property(x => x.SupportingEvidence).HasMaxLength(1000);
50          i.HasIndex(x => new { x.OwnerId, x.UpdatedAt }).IsDescending(false, true);
```

IsConcurrencyToken 告诉 EF 在更新和删除时比较加载得到的原始值。它不会自动把 Version 加一；真正推进版本的是根的 Advance。当前值决定 SET 中的新值，原始值决定 WHERE 中的预期值。这两个数字即使属于同一属性，也在不同位置承担不同职责。

真实生成语句中的核心部分如下，参数名称保留，值由驱动绑定。此次观察的根原值为 4，成功后为 5。

运行摘录

```
UPDATE "Investigations" SET "UpdatedAt" = @p0, "Version" = @p1
WHERE "Id" = @p2 AND "Version" = @p3;
UPDATE "Links" SET "Kind" = @p4, "Rationale" = @p5, "RevisedAt" = @p6
WHERE "Id" = @p7;
```

子关系 UPDATE 的 WHERE 只有 Id，没有它自己的版本条件。它之所以受到并发保护，是因为同一事务还必须成功更新根。如果有人另写一条绕过根版本的子表更新路径，这份保护就不再成立。聚合的设计承诺必须由所有写路径共同遵守，映射中的单个属性配置不能替代它。

7.5 预分配 Guid 与 Added 状态

源码 backend/EchoCase.Infrastructure/EchoDbContext.cs · L66-72

```
66          l.HasKey(x => x.Id); l.Property(x => x.Id).ValueGeneratedNever();
67          l.Property(x => x.SourceId).HasMaxLength(64); l.Property(x => x.TargetId)
    .HasMaxLength(64);
68          l.Property(x => x.Kind).HasConversion<string>().HasMaxLength(24);
69          l.Property(x => x.Rationale).HasMaxLength(500);
70          l.HasIndex(x => new { x.InvestigationId, x.SourceId, x.TargetId }).IsUnique();
71          l.HasOne<Discovery>().WithMany().HasForeignKey(x => new { x.InvestigationId,
    x.SourceId }).OnDelete(DeleteBehavior.Restrict);
72          l.HasOne<Discovery>().WithMany().HasForeignKey(x => new { x.InvestigationId,
    x.TargetId }).OnDelete(DeleteBehavior.Restrict);
```

EvidenceLink 构造器自己生成 Guid，因此映射显式采用 ValueGeneratedNever。这个配置声明“键值已经由应用分配”，而不是把非默认键当成数据库已存在成员的信号。对已跟踪根中新发现的子实体，生成策略会影响状态推断；错误状态可能产生 UPDATE，影响零行之后表现为并发异常。值生成配置



图 7.3 生成策略影响新成员发现；看实体状态和实际 SQL，才能区分映射错误与并发。

所以 DbUpdateConcurrencyException 的名字不等于“另一个用户一定改过数据”。它也可能说明框架更新了原本不存在的行。定位时应先问：这个成员本应新增还是更新，状态是 Added 还是 Modified，键由谁分配，生成 SQL 是否符合意图。若把映射错误一律解释成真实竞争并无限重试，只会重复错误语句。

这里没有把所有 Guid 属性都改成 ValueGeneratedNever 的必要。规则针对实际由领域预先分配的关系身份，其他实体和键类型应按自己的创建路径判断。框架约定可以减少配置，但一旦领域身份策略与约定不同，就应明确表达。

7.6 SaveChanges 与事务提交不是同一个事件

源码 backend/EchoCase.Infrastructure/PostgresInvestigationStore.cs · L47-56

```
47          var result = apply(aggregate);
48          if (aggregate is null) db.Investigations.Add(result);
49          var response = SessionPresenter.Map(result);
50          db.Receipts.Add(new() { OwnerId = c.OwnerId, Key = c.Key, Fingerprint = c.Fingerprint,
        Response = JsonSerializer.Serialize(response), CreatedAt = clock.GetUtcNow() });
51          db.Audit.Add(new() { OwnerId = c.OwnerId, InvestigationId = result.Id, Operation =
    c.Operation, Version = result.Version, OccurredAt = clock.GetUtcNow() });
52          try
53          {
54              await db.SaveChangesAsync(ct);
55              await tx.CommitAsync(ct);
56          }
```

Store 先把命令结果投影成视图，再把回执与审计加进上下文，然后 SaveChanges 并显式 Commit。投影对象在内存中已经存在，并不意味着它可以提前交给客户端。只有提交成功，返回值才成为服务器对外确认的业务结果。

关系提供者通常让单次 SaveChanges 的多条修改具有事务性，但当前协议的锁、加载与回执读取发生在保存之前，因此 Store 需要显式事务把这些阶段包括进来。尚未提交的事务在离开作用域时回滚；若丢失的是提交确认，则还需按结果未知分析。当前代码没有把一个可能已损坏跟踪状态的上下文拿去继续服务下一次请求。事务边界

实际旧版本写入的观察显示，根的并发条件失败后，子关系尝试修改没有留存。这才是聚合保护的关键结果：不是捕获到某个异常就算完成，而是失败决定没有留下部分业务事实。下面再将状态种类、原始值与工作单元成本分别辨认清楚。

7.7 用五种状态阅读一项业务操作

跟踪器的状态是一套持久化语言。Detached 表示当前上下文不管理这个实例；Unchanged 表示已跟踪且尚未识别需写入的差异；Added 表示准备插入；Modified 表示已有实体的部分值需要更新；Deleted 表示准备删除。状态属于本上下文中的条目，不是业务 Status 的另一种写法。Submitted 调查在读取后也可以是 Unchanged，Active 调查在本次保存中可能是 Modified。



图 7.4 状态可按操作进入或转换；本图按语义并列展示，不表示固定的状态转换链。

把 Examine 放入这个模型，根已经被加载为 Unchanged，新 Discovery 进入集合，检测后新成员准备 INSERT，根的 Version 与 UpdatedAt 准备 UPDATE。把 ReviseLink 放入模型，成员已经存在，因此是属性更新而非新增。把删除关系放入模型，则应关注集合移除如何在已配置关系中变成删除，而不是看到集合数量减少就断言数据库行已经消失。

状态的价值在于让预期变成可检查问题。新增关系失败时，首先应期待 Added 和 INSERT；修订关系失败时，期待稳定 Id 对应的 UPDATE；根没有 Modified 时，应追问哪个行为漏掉了 Advance。这样，诊断从业务意图走向状态，再走向 SQL，不会被单个异常名牵着走。

7.8 三份值与一次真正的竞争

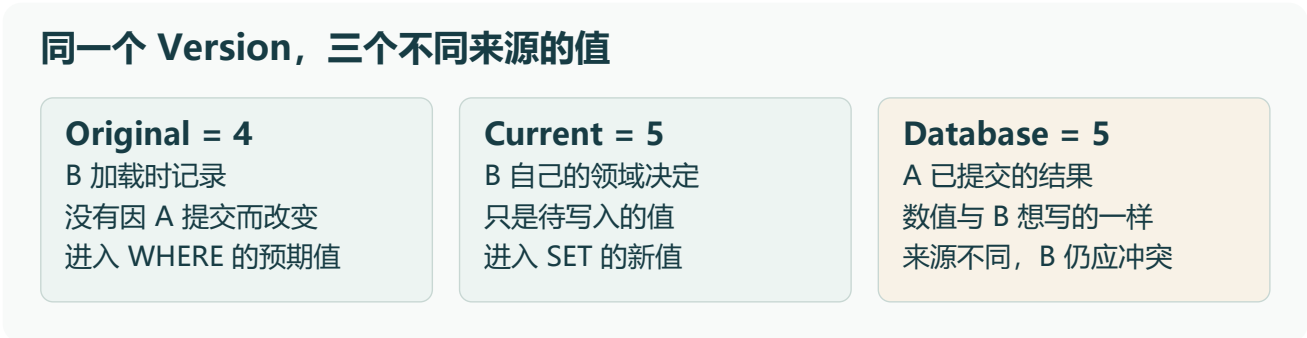


图 7.5 示意推演：Current 等于数据库数值，不能证明本次操作已经成功。

同一个 Version 可以有三个观察值：上下文加载时的 original、领域改完后的 current、另一条数据库读取看到的 database。设 A 与 B 都加载 4，A 保存 5；B 的 original 仍是 4，current 也可能已被自己的决定推进到 5，数据库则是 A 的 5。这三个数字表面只出现 4 和 5，语义却完全不同。

B 不能因为 current 等于数据库的 5 就认为自己的决定已经提交。current 表示它想写出的值，数据库的 5 属于 A；WHERE 应比较 original 4，正是为了识别这种同值不同来源。若用客户端传来的最新数字覆盖 OriginalValues，便可能抹掉检测竞争的依据。

业务中常用时间戳猜测“谁更新得更晚”，但精度、时钟和并发来源都会增加歧义。应用版本在同一聚合内提供明确序列；原始值则把该序列与本工作单元的读取联系起来。两者合作，才能将一段跨时间的用户判断变成数据库可仲裁的条件。

7.9 物化、变更检测、保存分别消耗什么

物化把结果行转换成对象，身份解析与关系修复要处理重复根列；跟踪器为条目和原值保留内存；DetectChanges 要比较被跟踪图中的值与关系；保存再把变更组织成命令。减少 SQL 次数并不一定减少全部应用成本，因为一个很宽的 JOIN 仍可能让客户端处理大量重复数据。

当前图上限小，所以完整加载与默认检测容易解释；只读路径不需要保留修改快照，故用 AsNoTracking。未来若把大量历史调查长期放入一个上下文，扫描和内存成本会增长，即使每次实际只改一条笔记。此时先缩小工作单元与查询形状，通常比盲目关闭自动检测更容易保证正确性。

关闭检测是改变框架前提的选择，不是普通“性能开关”。漏掉扫描可能让 CLR 变化没有进入 SQL；手工批量设置状态又可能扩大 UPDATE 范围。任何这类优化都应能解释受影响实体的完整生命周期，并由实际生成命令校正，不能只比较某一次 SaveChanges 的耗时。

最后，SaveChanges 默认接受跟踪变化与外部事务最终提交仍有先后。即使保存阶段已经把条目恢复为 Unchanged，提交确认依然可能失败。EchoCase 只在 CommitAsync 成功之后返回结果，并结束请求作用域，避免把跟踪状态误当成持久提交证明。这再次说明，同一工作单元里必须辨认每个观察时刻。

第 08 章

数据模型演进与时间精度

增加“关系解释”看似只需要两列：Rationale 和 RevisedAt。真正困难在于旧数据没有这两个事实，旧成功回执中也没有对应字段，而新界面却必须能区分“从未修订”和“已经修订”。安全演进首先需要定义缺失值的含义，再讨论迁移语法。

8.1 null 可以表达真实历史

旧关系只包含端点和类型，没有说明何时以及为何修订。迁移若给所有旧行填“历史记录”并写入迁移时间，就制造了没有发生过的业务行为。EchoCase 选择两列都可空：二者均为 null 表示没有可记录的修订解释；新修订则同时写入解释与业务时间。



图 8.1 迁移成功只覆盖表结构；对象反序列化与领域语义还需独立兼容。

源码 backend/EchoCase.Infrastructure/Migrations/20261111060117_LinkRevisionExplanation.cs · L14-25

```
14              migrationBuilder.AddColumn<string>(
15                  name: "Rationale",
16                  table: "Links",
17                  type: "character varying(500)",
18                  maxLength: 500,
19                  nullable: true);
20
21              migrationBuilder.AddColumn<DateTimeOffset>(
22                  name: "RevisedAt",
23                  table: "Links",
24                  type: "timestamp with time zone",
25                  nullable: true);
```

这是加法式演进，原 Id、端点和 Kind 不变。数据库 CHECK 进一步要求两列要么同时为空，要么同时非空且解释长度在二十至五百之间。只加一个 nullable 字符串而不建立配对规则，会允许“有解释却不知道何时修订”等没有明确定义的状态进入持久层。

可空并不是宽松到任何内容都合法。领域的 Trim 和标量值计数仍处理新业务输入；数据库的 varchar 与 char_length 保护持久边界。当前规则使用 Unicode 标量值，与 PostgreSQL UTF-8 数据库中字符计数相配。若业务改成“用户感知字数”，组合字符和表情簇就需要重新定义，不能只改界面上的提示文字。

8.2 约束之间怎样合作

关系 Id 是主键；(InvestigationId, SourceId, TargetId) 唯一索引防止重复无向对；两个复合外键把端点连接到同一调查的 Discoveries；CHECK 拒绝自连、非法 Kind 和非规范顺序。不同约束分别保护身份、唯一性、成员归属和局部值域。

它们没有完整实现 Active 才能修改，也没有证明结论理由充分。将所有领域规则都复制到数据库触发器并非当前选择。日常写入走聚合，数据库兜住适合关系模型的事实。评审时应逐条判断保证来自哪里，不能用“有数据库约束”笼统覆盖所有规则。

唯一索引还具有竞争语义。两个事务即使都在各自快照中看不到冲突，最终也不能同时保存同一唯一键。第十章会看到，(InvestigationId, Version) 审计唯一索引甚至可能在根 UPDATE 之前发现竞争。因此约束不只是静态 schema 文本，也是运行时仲裁的一部分。

8.3 旧回执是另一份持久合同

源码 backend/EchoCase.Application/Investigations.cs · L11-11

```
11  public sealed record LinkView(Guid Id, string SourceId, string TargetId, string Kind, string?
    ·     Rationale = null, DateTimeOffset? RevisedAt = null);
```

LinkView 的新增参数提供 null 默认值，让旧 JSON 缺少字段时仍能恢复视图。回执保存的是序列化成功响应，不会随着表结构升级自动重新生成。如果只验证新数据库查询能返回两列，却忘记重放旧 JSON，网络重试就可能在升级后失败。

旧回执重放后仍应表达旧操作成功时的事实，而不是给它补成当前调查。否则幂等键不再确认原操作，变成了另一种隐藏查询。返回旧形状兼容恢复的目的，是维持这份历史语义；最新资源由正常 Get 再取得，前端负责不让旧回执倒退当前确认缓存。

这也是“向后兼容”需要具体对象的原因。表行兼容、HTTP 请求兼容、响应消费者兼容和存储 JSON 兼容是四个问题。一个字段可空只回答其中一部分。严谨的改动应沿着所有实际持久表示追踪，而不是只看迁移能否执行。

8.4 时间相等为何曾经失败

一次真实保存的即时响应与随后重读，tick 值分别为 639299746962572167 和 639299746962572160，相差七个 tick，也就是 0.7 微秒。用户界面几乎看不见这个差异，但逐值合同和回执比较能看到。原因是 .NET 的时刻分辨率比 PostgreSQL timestamp 的微秒精度更细。PostgreSQL 时间类型



图 8.2 实际修复后两个 ticks 都为 639299753895425590；不把显示格式当作时刻一致。

源码 backend/EchoCase.Application/Investigations.cs · L85-89

```
85      private DateTimeOffset Now()
86      {
87          var utc = clock.GetUtcNow();
88          return utc.AddTicks(-(utc.Ticks % TimeSpan.TicksPerMicrosecond));
89      }
```

修复在业务时间进入领域之前截去不足微秒的尾数，而不是等读取时再用模糊容差隐藏差异。这样，即时响应的领域时间、序列化回执和数据库重读有相同可表达精度。实际修复后观察中的两个值都为 639299753895425590。

时间精度与时间格式不同。把同一时刻格式化成不同小数位，或者以不同 UTC 偏移显示，未必改变时刻；这里原先的值确实有精度损失。当前实现统一传入 UTC，而不是要求 PostgreSQL 替我们保留原始显示时区。需要保存“用户当时所在时区”时，应增加独立业务字段，不能指望 timestamptz 自动记住所有输入形式。

这项修复不会把历史回执中的高精度时间重新写一遍。旧回执仍是旧操作记录。对未来保证和历史行为应分开描述，避免在兼容处理中改写事实。

8.5 Down 能撤销结构，不能恢复丢失的解释

迁移的 Down 会删除约束和两列。若这些列已经保存新解释，删除它们就会丢失业务信息；因此“存在 Down 方法”不等于无损回退能力。本书不把 schema 的逆操作包装成完整的数据恢复承诺。

对于这次加法演进，旧列集中的身份、端点和类型可以保留，新代码理解 null，新输入填入成对新字段。若未来改为重命名、拆列或改变含义，则需要更细的读取兼容与数据转换设计。演进安全取决于语义和数据流，不由 EF 生成文件的长度决定。

8.6 CHECK 与 null 的三值逻辑

SQL 表达式可能得到 true、false 或 unknown。一个简单的 char_length(Rationale) 范围条件，在 Rationale 为 null 时不会像普通 C# bool 那样直接得到 false；CHECK 也不能被当作隐式 NOT NULL。当前约束显式组合两种合法情况，避免让半对空值滑过。PostgreSQL 约束语义

Rationale	RevisedAt	当前配对规则
null	null	合法旧状态，未记录修订
有效解释	非空时间	合法新修订
null	非空时间	拒绝，缺少解释事实
有解释	null	拒绝，缺少修订时间

数据库仍可能接受二十个空格，因为 char_length 计算长度而不自动执行领域 Trim。领域会把它清理后拒绝。这再次说明相近校验的边界不同：数据库保护持久形状，领域保护新行为语义。直接写表不是合法业务 API。

8.7 schema 快照不是业务历史

EF 的模型快照描述当前映射，用于后续迁移差异；迁移文件描述结构变化；Audit 描述应用操作；Receipt 描述某次成功结果。这四份材料都可能随时间增长，却不是同一种历史。修改模型快照不能补出旧关系的解释，审计版本也不能替代真实字段迁移。

关联修订的 additive 设计让旧列集继续有意义，新列含义可被准确区分。若以后需要保留每次解释全文，当前只保留最新值和操作审计还不够，需要单独的修订历史模型。现有 Audit 没有完整前后值，也没有防篡改保证，不能在需求未实现时称作完整审计账本。

第 09 章

PostgreSQL MVCC 与读取事实的时间

两个请求都读到 Version 4，随后一个成功、另一个失败，并不矛盾。读取回答“在这个快照中看见什么”，写入回答“现在能否让这项决定成立”。MVCC 为并发读取提供版本可见性，应用仍需建立自己的业务写入条件。

9.1 行更新留下版本，不是覆盖一张共享纸

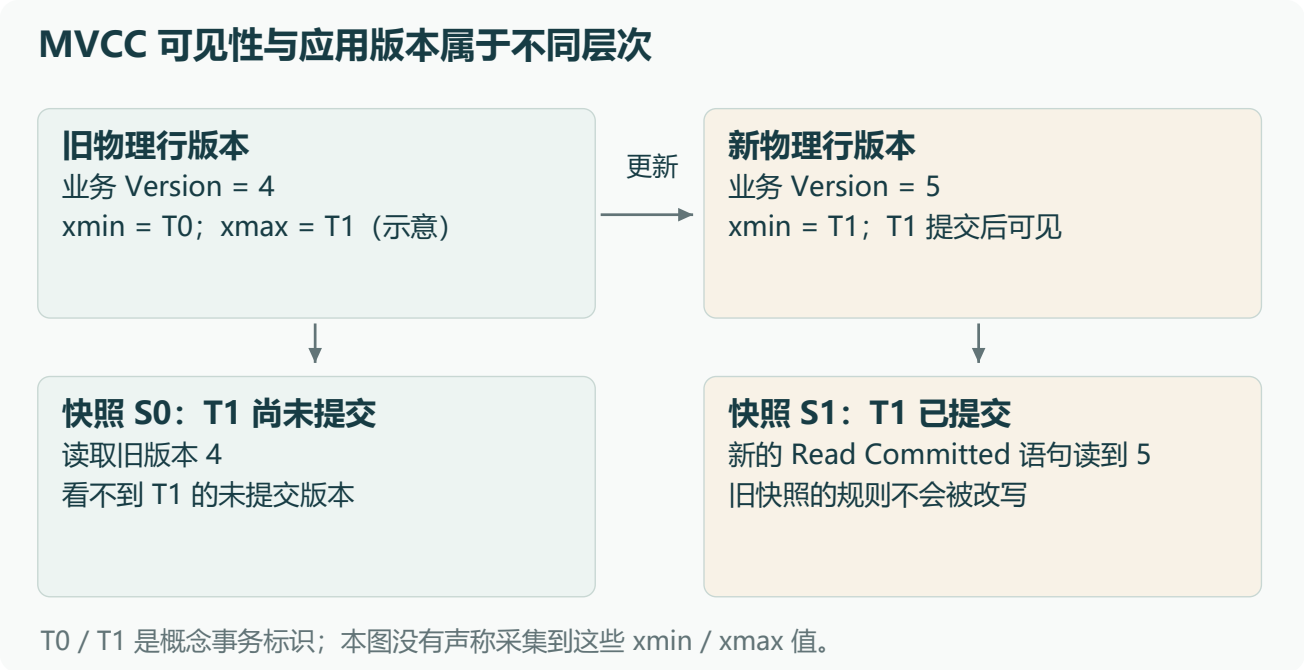


图 9.1 行版本解释“读到哪个事实”；应用 Version 解释“此次写入基于哪个事实”。

可以把一次 UPDATE 理解为创建新的物理行版本，并让可见性规则决定不同快照应看到旧版还是新版。图中的 xmin、xmax 和事务 T0、T1 是概念标识，用来说明版本来源与失效关系，没有声称实际采集了这些系统列。真实事务状态、快照与可见性判断比单纯比较两个数字更复杂。MVCC 概念

旧快照不能看到尚未提交的 T1 新版本，新语句在 T1 提交之后可能看见它。这样，读者不需要等待每一个写者完成才能读取一个一致的已提交世界。并发写入同一行仍可能等待锁，DDL 也可能与读取发生锁冲突；“普通读写并发更好”不等于数据库没有锁。

应用 Version 则是 Investigation 的业务坐标。Version 从 4 到 5，由根方法显式推进；数据库事务编号属于数据库内部，不与它一一对应。一次事务可能修改许多实体，一次业务版本也可能伴随多个表写入。把 xmin 当作当前应用 Version，会混淆持久实现与公开合同。

9.2 Read Committed 的快照属于语句

PostgreSQL 的默认 Read Committed 中，一条普通查询看到语句开始前已提交的数据以及自身事务的相应变化。下一条语句开始时可以取得更新快照。因此把两次 SELECT 放进同一事务，并不能保证它们返回完全相同的事实。隔离级别



图 9.2 语句级快照允许不重复读；事务包裹并不自动意味着整个事务只有一个读取快照。

这对 EchoCase 有两个直接影响。首先，Graph 通过 AsSingleQuery 把根、发现和关系作为一条 JOIN 查询加载，避免在多个读取语句之间拼出不同提交时刻的图。其次，同键重试等待 advisory lock 后，再执行回执查询，可以看到先前事务已经提交的回执；事务是在等待之前开始的，并不因此永久停在旧快照。

源码 backend/EchoCase.Infrastructure/PostgresInvestigationStore.cs · L28-34

```
28          await using var tx = await db.Database.BeginTransactionAsync(ct);
29          // A database-scoped lock, released on commit/rollback; unrelated keys proceed
    ·   independently.
30          var lockName = c.OwnerId + ":" + c.Key;
31          await db.Database.ExecuteSqlInterpolatedAsync($"SELECT pg_advisory_xact_lock(
    ·   hashtextextended({lockName}, 0))", ct);
32          Investigation? aggregate = null;
33          if (c.SessionId is { } id)
34              aggregate = await Graph.SingleOrDefaultAsync(x => x.Id == id && x.OwnerId ==
    ·   c.OwnerId, ct) ?? throw Missing();
```

BeginTransactionAsync 没有显式指定更强隔离，实际 Store 竞争观察记录两方都为 ReadCommitted。理解这项事实后，才能解释为何锁等待之后的查询能恢复成功结果，也才能知道图读取与后面的回执读取并非整个事务共享的永恒快照。

9.3 单语句图读取的代价

AsSingleQuery 让同级集合通过 JOIN 返回。若一个根有 d 条发现和 l 条关系，连接可能为该根产生 d×l 行，根列在其中重复。当前案件最多六条发现、十五条关系，所以最坏数量受控；这种已知上限支持了选择，而不是让笛卡尔放大现象消失。单查询与拆分查询

若改为 split query，通常能减少重复根列，但多个语句在 Read Committed 下可能观察到不同提交阶段。要保持同样读取语义，可能需要更强隔离或其他设计。不能只根据某个性能提示统一改成拆分，然后认为业务快照保证完全不变。查询性能和一致性经常由同一个选择共同决定。

在实际查询探针中，每个样本根有两条发现和一条关系，五十个根最终形成一百条连接行。这个数字是给定数据形状的结果，不能推广成所有调查永远只放大两倍。第十七章会沿着真实计划解释根选择和子表连接分别花费什么。

9.4 可见性不会阻止丢失决定

设 A、B 都读取版本 4，各自计算版本 5。若最后只是按 Id 更新，后提交者可能覆盖先提交者的结果。两个事务都只读了已提交数据，仍然会丢掉一个用户决定。MVCC 防止看见半提交世界，不会自动知道这两个决定是否可以合并。

EchoCase 把加载时的 Version 作为 UPDATE 条件，并让任何成员修改都推进根。第十章将说明数据库等待之后如何重新判定条件，以及 EF 如何根据影响行数报告竞争。这里先建立边界：读取隔离提供观察基础，业务并发协议保护决定，二者解决的问题不同。

9.5 跨行规则暴露更大的边界

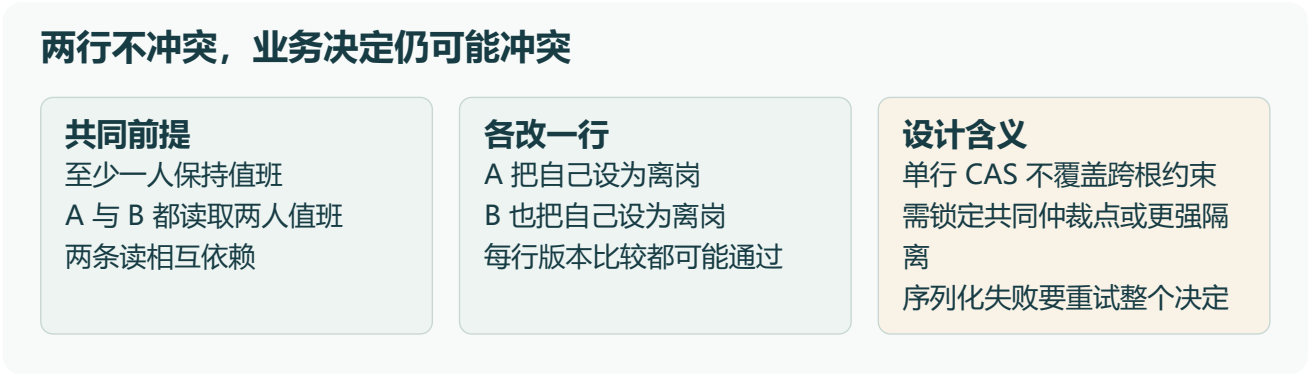


图 9.3 概念反例：展示根版本策略的边界，不声称 EchoCase 已实现值班业务。

值班反例不是 EchoCase 新增功能，而是帮助识别当前策略的适用条件。A 和 B 都看到两人值班，各自只更新自己的行；如果每行版本都未改变，各自 CAS 可能通过，最后却没有任何人在岗。每行更新都合法，并不推出跨行约束仍成立。

如果规则跨越多个聚合，可以让决定竞争一个共同锁定点，或在适当设计中使用 Serializable，并处理整个事务重新执行。提高隔离不能省略失败处理：序列化失败意味着这次决定需要在新事实下重做，而不是只重试最后一条 UPDATE。是否可以自动重做，还要看其中是否包含外部副作用和用户选择。

当前调查把需要同步判断的发现与关系收在一个根版本下，缩小了这种风险；它没有宣称解决任意跨调查规则。清楚认识边界，才能在新需求出现时知道什么时候需要改变模型。

9.6 事务生命周期影响读取成本

旧快照需要旧行版本继续存在，长事务可能让旧版本的清理推迟。读事务看起来没有改数据，仍可能对系统产生持续成本。本书不把它扩展成运维教程，只保留应用设计含义：不要把等待用户输入、远程交互或长时间计算藏进数据库事务。

EchoCase 的事务在收到完整命令后才开始，数据库读取、领域决定和保存处于有限请求链中。用户在浏览器编辑数分钟，不会持有一个同步数据库事务；她只持有 baseline Version。乐观并发把这段思考时间移出数据库，把重新确认放到冲突发生时；继续理解快照和对象缓存，才能看清这项选择的条件。

9.7 快照判断不只是比较大小

一个容易误导的心智模型是“xmin 小于当前事务号就可见”。事务可以交错开始和提交：较早编号的事务仍在执行，较晚编号的事务却已经提交。快照需要识别在它建立时仍活跃的事务，并结合提交状态判断行版本；仅看事务号先后不足以恢复可见世界。

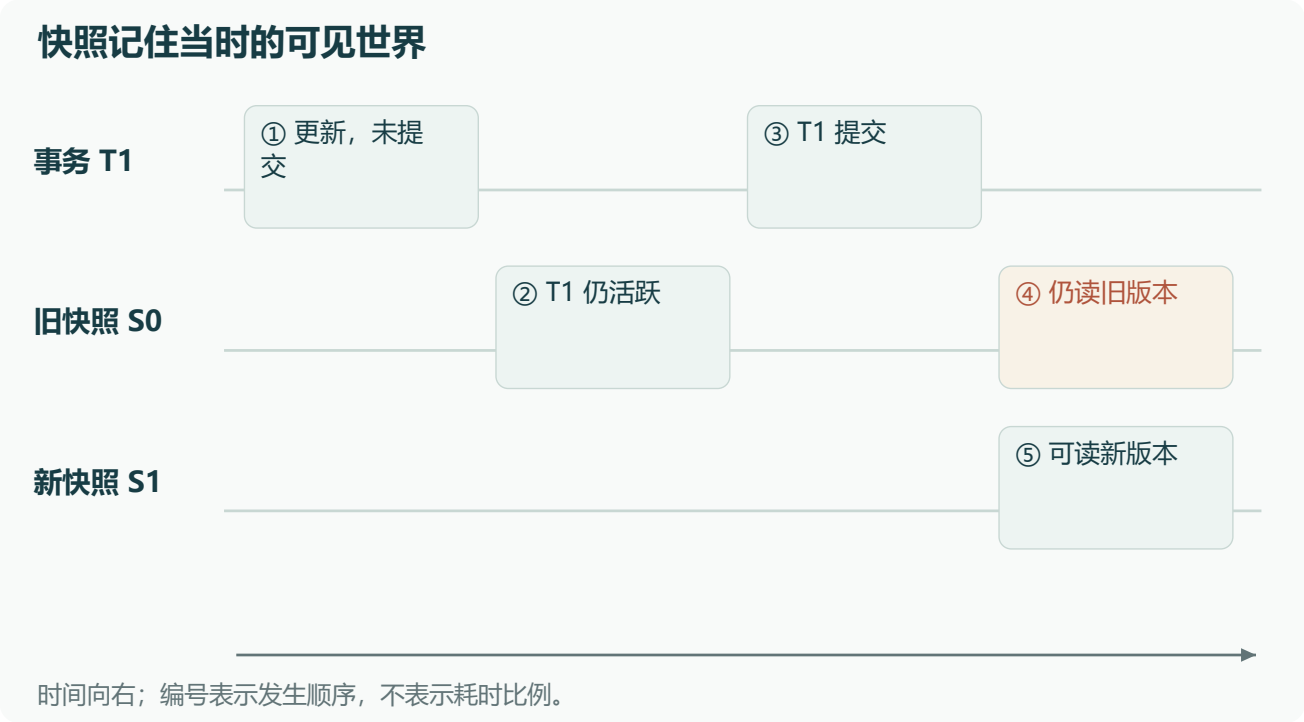


图 9.4 示意可见性：后来的提交不会把旧快照改写成新快照；实际规则还结合事务状态。

图中 T1 和 T2 都是示意，重点是提交边界。快照建立时 T1 尚未提交，即使稍后真的提交，这个旧快照也不会因此改写成新快照。下一条 Read Committed 语句可以取得新快照，从而看到 T1 的结果。于是“不读未提交”与“可重复读”成为不同保证，不能混用。

数据库语句内部也不要求在整个执行期间停止其他事务。它依据确定的可见性规则遍历版本，其他事务可以继续提交。应用得到的是一个自洽读取视角，而非暂停世界后的拍照。这种理解比“先锁全表再读”更接近 MVCC 的目的，也更能解释长快照为何仍需要旧版本存在。

9.8 EF 身份缓存可能遮住新快照

即使数据库下一次查询使用新快照，跟踪上下文仍可能复用已经管理的同键实例。数据库可见性和 EF 对象身份是两层机制：前者决定语句能读到的行，后者决定结果如何进入当前对象图。不能拿一个长期跟踪的对象一直不变，直接证明 PostgreSQL 提供了事务级重复读。

EchoCase 的 Get 使用无跟踪查询，Write 每个请求使用新上下文，因此当前路径减少了这种混淆。在研究隔离级别时，要明确观察的是原始 SQL 结果、重新物化的对象，还是同一个跟踪实例。否则实验可能无意中研究了身份映射，却把结论写成数据库隔离。

同样，前端 Query 的最后确认缓存也不自动等于数据库的“最新”。从数据库快照到 EF 对象，再到 HTTP 正文和浏览器缓存，每层都可能保留一个有限时刻的事实。版本使这些时刻可以比较，却不会让传播延迟消失。

9.9 更强隔离解决什么，又留下什么

Repeatable Read 让事务中的后续读取保持更稳定的快照，因此适合需要同一观察世界的分析，但不能仅凭它就宣称所有跨行业务约束安全。前面的值班写偏差正说明，两个事务在同样稳定的旧世界中分别改不同的行，也可能共同违反规则。

Serializable 追求与某种串行顺序等价的成功结果，代价之一是部分事务必须失败重试。失败不是数据库“不稳定”，而是系统拒绝让不可能串行成立的决定共同提交。应用需要重新读取并重新计算整个决定；若其中包含已执行的外部副作用或不可重复用户选择，自动重试又需要额外设计。

EchoCase 当前选择默认隔离、单语句图读取、同键事务锁和根条件写入。这些机制各自负责一个边界，并没有把事务级别作为万能替代。评审时，应沿实际可能的竞争列出保护点，再判断是否存在未被任何一点仲裁的规则。

第 10 章

根版本与事务如何共同拒绝过期写入

在页面上禁用保存按钮，只能阻止这个页面再次点击。另一个标签页、另一个请求或同一用户的另一台设备仍可能修改调查。真正的仲裁必须发生在持久层，而且必须保护根与子对象形成的完整决定。

10.1 两次版本检查关闭两个不同窗口

Store 加载调查后，先比较 aggregate.Version 与 ExpectedVersion。若用户发来版本 4，而当前已是 5，就立即返回 412。这个检查给常见过期窗口提供清晰反馈，却不是最终并发保证：比较完成后，到 SQL 保存之前，另一个事务仍可能提交。

源码 backend/EchoCase.Infrastructure/PostgresInvestigationStore.cs · L35-46

```
35          var previous = await db.Receipts.AsNoTracking().SingleOrDefaultAsync(x => x.OwnerId
    == c.OwnerId && x.Key == c.Key, ct);
36          if (previous is not null)
37          {
38              if (previous.Fingerprint != c.Fingerprint) throw new ApplicationFault(409,
    "key_reused", "此幂等键已用于其他请求。");
39              Replays.Add(1);
40              return JsonSerializer.Deserialize<SessionView>(previous.Response)!;
41          }
42          if (aggregate is not null && aggregate.Version != c.ExpectedVersion)
43          {
44              Conflicts.Add(1);
45              throw new ApplicationFault(412, "version_conflict",
    "档案已被其他窗口修改。请刷新后重新确认操作。");
46          }
```

因此模型又把 Version 标为 concurrency token。EF 保存使用加载时原值作为 UPDATE 条件；若另一事务已推进根版本，旧条件不再匹配。前面的 if 是应用层快速判断，后面的 WHERE 是数据库中的原子条件。只保留前者会有检查与使用之间的空隙，只保留后者则把容易识别的旧请求也拖到保存阶段。

10.2 真正同时读到旧版本时会发生什么

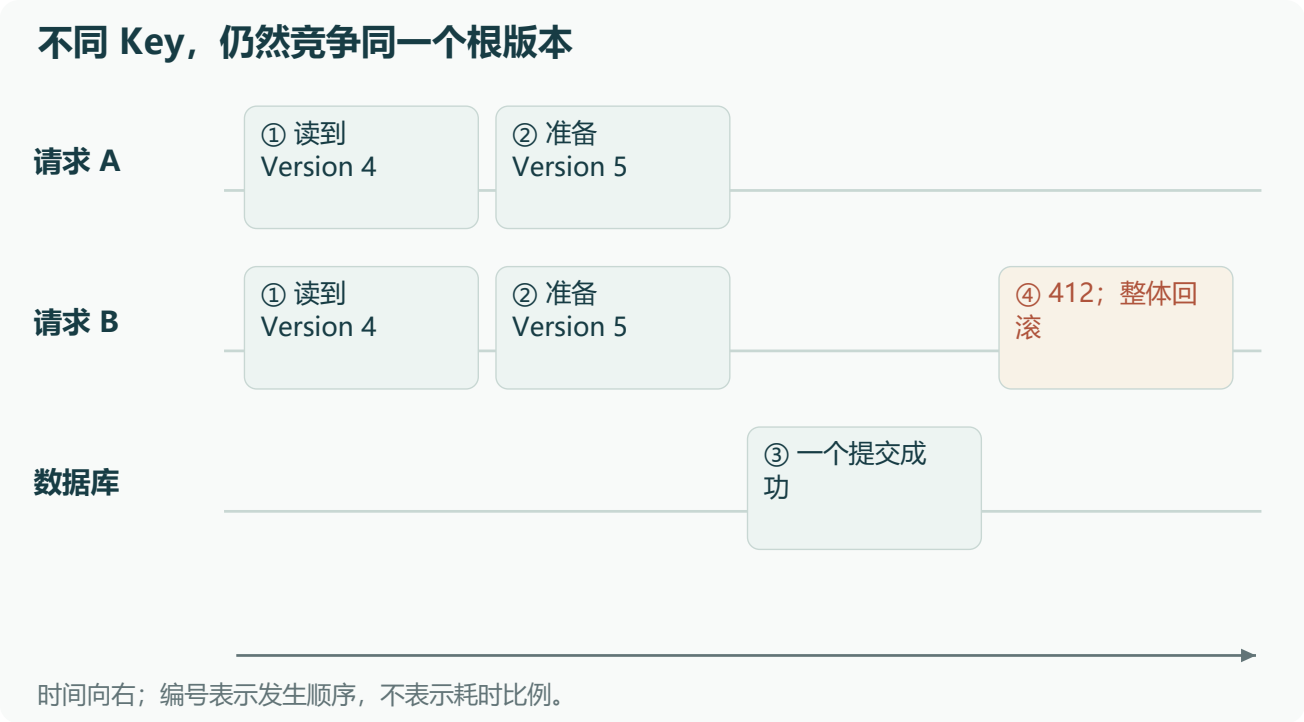


图 10.1 真实双 Store 竞争结果为 200 / 412；最终版本 5，修订审计和回执各保留一条。

真实 Store 竞争使用两个独立 DbContext、两个不同幂等键，让双方在领域回调的屏障前都完成加载。这个同步条件很重要：若只是依次发两个请求，后者可能在读取时就看到新版本，只验证了前置 if，没有触及数据库竞争。

两方实际隔离均为 ReadCommitted，最终结果是一个 200、一个 412，根停在版本 5；成功修订的审计和回执各保留一条。这些观察合在一起，说明仲裁不仅返回了不同状态，还保护了持久结果。不同幂等键没有让操作互相重放，它们确实在竞争同一个根。

对于根 UPDATE 的条件写路径，后来的写者可能先等待前一个写者。前者提交后，条件会针对可更新的行重新判断；旧 Version 不再匹配时，更新不能假装仍然成功。EF 根据应更新但未更新的结果抛出并发异常。EF 并发令牌

10.3 子实体没有版本列，为什么仍能保护



图 10.2 EF 批次中 Audit 可能先写；只有整体事务结果可以作为成功依据。

关联修订改变 Links 行，根只改变 UpdatedAt 和 Version。两条修改必须在同一个事务。只要根条件失败，子修改就不能单独留下；如果先把子表提交，再尝试推进根版本，就会出现“接口返回冲突，但关系已被改写”的破坏性结果。

当前领域保证每项成功修改推进根一次，Store 把根、子、审计和回执放入同一次 SaveChanges 和同一显式事务。这是可推理的协议：一次用户决定对应一个新根版本，所有附属事实共享成败。数据库没有魔法识别 DDD 聚合，保证来自这条具体写入路径。

10.4 为什么还捕获一个唯一约束异常

源码 backend/EchoCase.Infrastructure/PostgresInvestigationStore.cs · L57-67

```
57          catch (DbUpdateConcurrencyException)
58          {
59              Conflicts.Add(1);
60              throw new ApplicationFault(412, "version_conflict",
    "并发写入冲突。请刷新后重新确认操作。");
61          }
62          catch (DbUpdateException e) when (e.InnerException is PostgresException { SqlState:
    "23505", ConstraintName: "IX_Audit_InvestigationId_Version" })
63          {
64              // Audit can be inserted before the aggregate UPDATE in EF's batch.
65              Conflicts.Add(1);
66              throw new ApplicationFault(412, "version_conflict",
    "并发写入冲突。请刷新后重新确认操作。");
67          }
```

实际批次里，Audit 的 INSERT 可能排在根 UPDATE 前面。Audit 对 (InvestigationId, Version) 有唯一索引；两个基于版本 4 的决定都会尝试插入版本 5 的审计。后来的事务可能先在该唯一键上等待并失败，尚未到达根 UPDATE。

因此 Store 只对指定约束 IX_Audit_InvestigationId_Version 的 PostgreSQL 23505 映射为 412。不能把全部唯一约束失败都叫版本冲突：关系对重复、邮箱重复或回执键错误都有不同语义。精确匹配约束名，是避免误分类的关键。

运行摘录

```
INSERT INTO "Audit" (..., "Version") VALUES (..., @p4);
UPDATE "Investigations" SET "UpdatedAt" = @p5, "Version" = @p6
WHERE "Id" = @p7 AND "Version" = @p8;
```

这是实际批次顺序的压缩展示，省略号仅省去审计其他列，不能作为可执行 SQL。它提醒读者：代码中先改变根属性，并不决定 EF 批次一定先发送根 UPDATE。要解释异常来源，必须知道真正执行的语句与约束。

10.5 回滚后的对象与数据库不是同一个状态

事务失败后，当前内存对象可能仍带着尝试过的新属性值。数据库回滚不会把每个 CLR 对象自动恢复成旧值。EchoCase 抛出错误并结束本请求作用域，不把这套对象当作成功资源返回，也不继续在同一上下文中处理下一项业务。

客户端同样不应该用失败请求正文替换确认缓存。用户的草稿可以保留，因为它是尚未成立的意图；服务器确认状态则需要读取实际持久事实。把这两类状态混在一个对象中，往往导致“回滚”时误删新输入，或把失败值显示成已保存。前后端在这里必须共享相同的成败语义。

10.6 冲突处理为什么不能自动覆盖

假设 A 修改笔记，B 修订关系，两者都基于版本 4。保守的根版本策略会拒绝一个决定，即使它们改动不同字段。是否可以自动合并，需要业务判断：B 的解释可能依赖 A 刚纠正的笔记事实，单纯“字段不重叠”并不足以证明独立。

当前应用要求用户看到变化后重新确认。若未来要支持字段级版本或自动合并，应明确哪些决定能够交换顺序、如何检测语义相关性，以及审计版本如何解释。更高并发不是免费改进，它会改变用户承诺与推理复杂度。

这里的乐观来自用户编辑期间不持有数据库锁，而不是完全没有锁。真正保存时数据库仍需要行和索引层面的同步。同样，它也不意味着冲突可以忽略。合理的界面应保留意图、展示当前事实，并让用户作出新的决定。

10.7 审计唯一键为什么不能取代根条件

版本审计唯一键会阻止两个事务同时记录同一调查版本，但它保护的是审计事实。若某条写路径不写审计，或错误地产生不同版本号，它就不再覆盖竞争。根 UPDATE 的 WHERE 则直接把修改与读取原值联系起来，属于资源本身的保护。

保留两者不是重复设计同一个锁。根条件表达“我基于这个版本修改”；审计唯一性表达“一个调查版本只对应一个操作记录”。它们碰巧能在部分竞争中较早发现同一冲突，错误分类才需要识别不同数据库表面的失败来源。业务含义仍各自清楚。

如果未来一次操作对根推进两次，审计只记录最终版本，唯一索引也不会自动发现中间版本被跳过。因此“每个成功行为只推进一次”仍是领域与用例的合同，需要有针对性验证。数据库保证和程序保证的组合才构成完整模型。

10.8 从失败结果反推验证强度

两个请求返回 200/412，还可能只是后者很晚才读到新版本。只有确认双方都加载旧版，才能说明数据库竞争路径被触及；只有检查子关系、回执和审计，才能说明失败没有部分落库。每增加一个观察，排除的错误解释就更具体。

反过来，一次竞争测试不能证明所有调度顺序都已穷尽。它验证的是被同步条件明确构造的关键窗口，源代码和数据库机制帮助将结论推广到同类情况。工程论证由模型与观察共同组成，任何一边单独都不够。

第 11 章

幂等回执与结果未知

客户端没有收到成功，不代表服务器没有成功。事务可能已经提交，响应却在网络中丢失；也可能 HTTP 200 已经到达，但正文无法解析。此时若客户端生成新键、换成最新版本并再次保存，就把“确认原操作”变成了“再做一次新操作”。

11.1 幂等保护哪一段副作用

EchoCase 的命令只改变本地 PostgreSQL 里的业务、审计和回执，没有发送邮件或调用外部支付。幂等协议因此能把这些副作用放进同一个事务：要么全部提交，要么都不成立。回执中的 Response 是成功时投影的 SessionView。先提交业务再补回执，会留下已成功却无法重放的缺口；先提交回执，则可能为失败业务制造虚假的成功证明。

源码 backend/EchoCase.Infrastructure/EchoDbContext.cs · L73-78

```
73          var r = b.Entity<Receipt>();
74          r.HasKey(x => x.Id); r.Property(x => x.OwnerId).HasMaxLength(450); r.Property(x =>
    x.Key).HasMaxLength(36); r.Property(x => x.Fingerprint).HasMaxLength(64);
75          r.HasIndex(x => new { x.OwnerId, x.Key }).IsUnique(); r.HasIndex(x => x.CreatedAt);
76          var a = b.Entity<AuditEntry>();
77          a.HasKey(x => x.Id); a.Property(x => x.Operation).HasMaxLength(32); a.Property(x =>
    x.OwnerId).HasMaxLength(450);
78          a.HasIndex(x => new { x.InvestigationId, x.Version }).IsUnique();
```

Receipts 以 (OwnerId, Key) 唯一，审计以 (InvestigationId, Version) 唯一。前者识别用户意图，后者约束调查版本事实。它们不能互相替代：两个不同键仍会竞争同一版本；同一个键的重试则不应再次推进版本。

“幂等”在这里不表示网络只发送一次。网络可以多次传送相同请求，服务器在既定范围内确认同一提交。若未来加入外部副作用，本地回执无法让远程系统自动参与回滚，需要新的交付设计。把当前保证称作任意系统的 exactly-once，会越过实现边界。

11.2 同键锁让回执检查串行发生

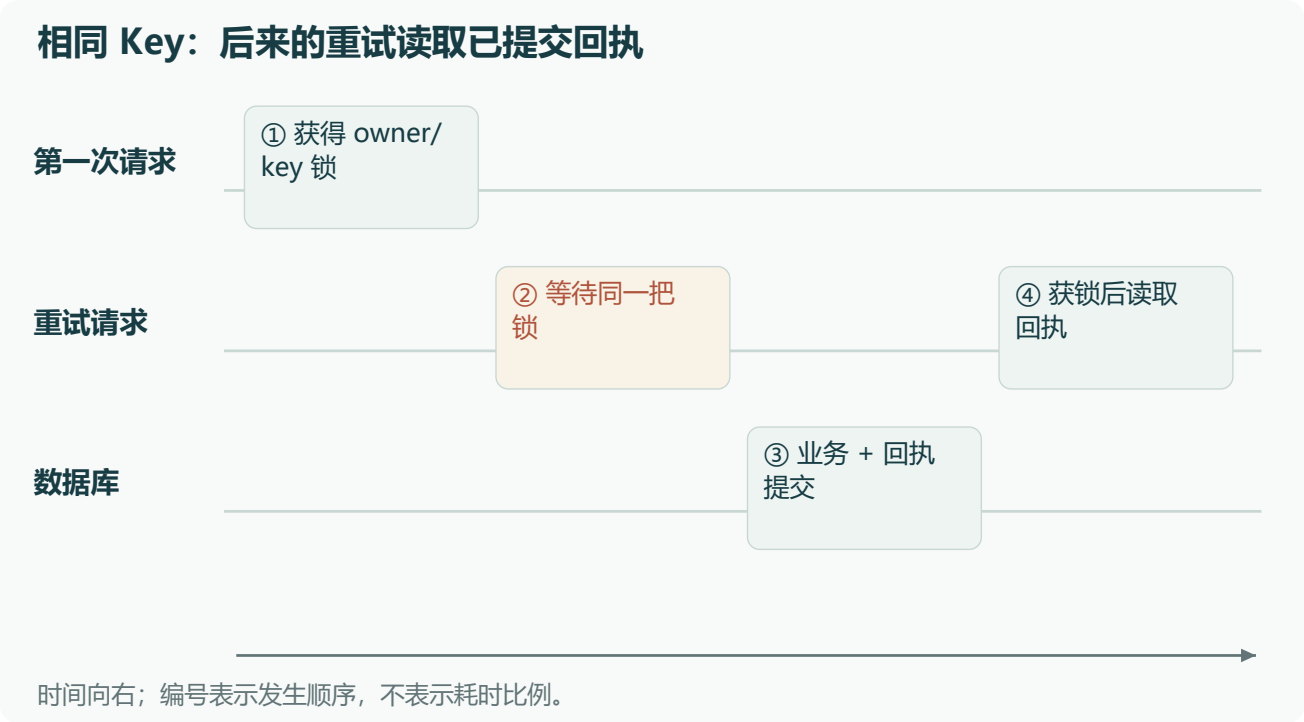


图 11.1 事务级 advisory lock 在提交或回滚后释放；锁名哈希碰撞只会增加无谓等待。

Store 在事务内对 owner 与 key 拼接后的名字哈希加事务级 advisory lock。第一次请求获锁后加载、判断、修改并提交；同键重试等待。获锁后再读回执，Read Committed 的新语句可以看见先前提交结果，便不再执行领域回调。事务级 advisory lock

事务级锁在提交或回滚时释放，不依赖连接池恰好关掉连接。64 位哈希理论上可能碰撞，两个不相关键因此多等待一会儿；真实回执仍按 owner/key 精确查询和指纹比较，所以碰撞不会把一个用户的响应当成另一个键的结果。锁解决排队，精确键解决身份。

该锁不是调查级锁。不同键可以同时推进到同一根的版本仲裁，前章真实竞争正是这种情况。若误以为 advisory lock 已经序列化所有调查修改，就可能删掉根版本条件，从而破坏另一条竞争路径。

11.3 为什么先检查资源，再读取回执

写路径先按 id 与 owner 加载调查，再查 owner/key 回执。这样，即使调用者持有一个可猜测的键，也不能借回执绕过对目标调查的可见性判断。回执属于意图记录，不是免授权凭证。

找到回执后，指纹不同返回 409，相同则直接恢复历史 SessionView；版本前置比较发生在没有回执的路径。这一顺序允许第一次已成功、现在资源版本更高的原样重试仍被确认。若把版本比较放在重放之前，原请求使用的旧版本会使每次重试都被拒绝，回执就失去意义。重放分支不执行 apply、不增加审计，读取事务离开作用域时释放锁，不再为确认过去制造新业务变化。

同时，重试不能只保留键却更换 body 或 version，因为这些内容进入指纹。一个键只能表达一个确定意图。客户端若希望采纳最新服务器状态重新保存，需要让用户确认并生成新键，那属于下一次业务决定。

11.4 历史结果不是最新资源

设命令 K 在版本 4 上成功，产生版本 5。随后另一个命令产生版本 6。K 的重试应返回版本 5 的历史成功结果，不应伪造为版本 6，因为版本 6 不属于 K 的决定。

前端仍需要知道两件事：K 确实成功了，当前资源却已经更晚。Query 缓存的单调版本规则保留 6，草稿确认逻辑则拒绝用旧回执清掉仍需用户判断的输入。一个 HTTP 200 因此可以确认历史操作，却不能授权界面回到旧状态。

这种区分在业务系统中很普遍。订单创建回执确认订单曾建立，不代表它仍未取消；审批回执确认某步曾通过，不代表整个流程仍停在那一步。回执具有操作时间语义，资源查询具有读取时间语义。

11.5 客户端怎样保留不确定性

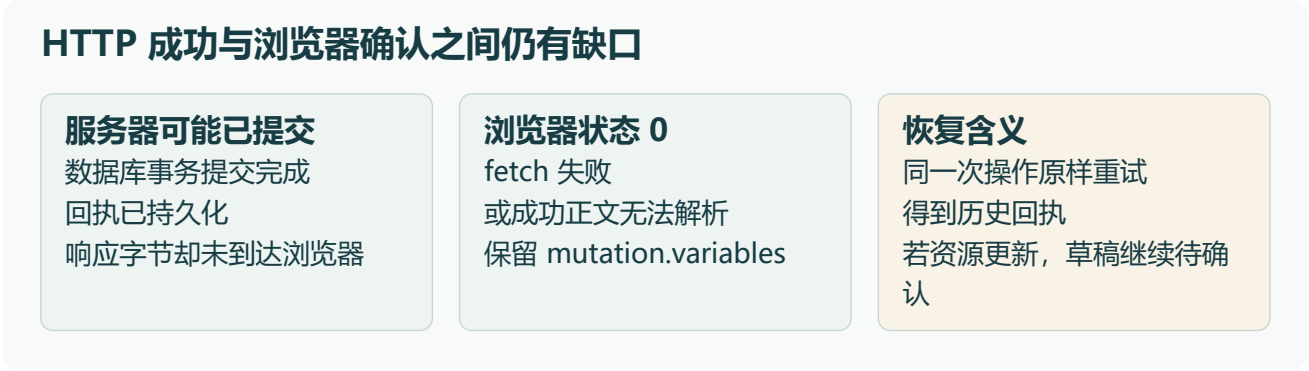


图 11.2 当前实现覆盖特定未知结果；没有声明通用断线持久队列或 exactly-once 网络传输。

源码 frontend/src/api/client.ts · L44-60

```
    response = await fetch(url, {
      method,
      headers,
      credentials: "same-origin",
      signal: options.signal,
      body:
        options.body === undefined ? undefined : JSON.stringify(options.body),
    });
  } catch (error) {
    // Query cancellation is intentional; it is not an ambiguous write failure.
    if (options.signal?.aborted) throw error;
    throw new ApiError(
      0,
      "network_error",
      "连接中断，保存结果尚未确认。恢复连接后可重试同一次操作。",
    );
  }
```

fetch 抛错时，若不是主动 AbortSignal 取消，客户端生成 status 0 的 ApiError。取消查询是有意停止，不应该被展示成写入结果未知；真实断线则不能断言事务失败。这个细分避免每次查询取消都触发恢复流程。

源码 frontend/src/api/client.ts · L71-84

```
71    if (response.status === 204) return undefined as T;
72    try {
73      return await response.json();
74    } catch (error) {
75      if (options.signal?.aborted) throw error;
76      throw new ApiError(
77        0,
78        "response_unreadable",
79        method === "GET"
80          ? "响应内容无法读取，请重新加载。"
81          : "响应内容无法读取，保存结果尚未确认。可重试同一次操作。",
82      );
83    }
84  }
```

HTTP 成功正文的解析失败也进入结果未知分类。真实测试在服务器提交后截断 JSON，浏览器无法构造可信 Session。错误文字保留原操作重试的含义，而不是把 200 当作足够的确认。

源码 frontend/src/workspace/useInvestigation.ts · L56-58

```
56    const uncertain =
57      mutation.error instanceof ApiError && mutation.error.status === 0;
58    const blocked = mutation.isPending || uncertain;
```

blocked 在 pending 或 uncertain 时阻止发出新的不同写入。笔记输入仍能继续，因为输入是本地草稿，不立即产生服务器副作用。工作台的“重试同一次操作”使用 mutation.variables 原样再次调用，因此 route、body、method、version、key 和草稿代次均保持不变。



图 11.3 成功正文不可读后，以原意图重试并恢复确认的真实界面。

①截图记录恢复后的文本，不把它伪称为错误发生瞬间。

②恢复确认依赖历史回执，而不是重新执行一次新的保存。

截图记录的是正文不可读之后，经原意图重试恢复确认的真实界面，而非错误刚发生的瞬间。它与受控网络反例一起说明：恢复不是把输入重新包装成任意新请求，而是从持久回执取得第一次操作的确定结果。

11.6 当前保证的边界

回执没有自动过期或清理策略，幂等保护因此依赖数据持续保留。它保留完整 SessionView，随规模增长会增加存储成本；新 DTO 演进也必须考虑旧回执。清理时限若未来引入，就会改变“多久以内可安全重试”的公开合同，需要产品和客户端一起理解。

当前状态 0 分支不覆盖所有可能的提交确认丢失形式，也没有把未知操作持久化到浏览器以供重启恢复。刷新或关闭页面可能失去本地变量。一般 5xx 也没有统一纳入恢复队列。这些限制不会否定现有保护，却决定了不能宣传成通用离线写入框架。

一个好的幂等设计，既要知道如何重放，也要知道哪些事实不能从重放推导。它确认一次已记录决定，不替用户制造下一次决定。

11.7 三种重复要分开处理

情形	请求身份	正确含义
网络重传原请求	同 owner、同 key、同指纹	恢复原成功，不再次应用行为
用户再次作决定	新 key，可有相同正文	按当前前提执行新的命令
错误复用旧键	同 key，不同版本或正文	409，拒绝把历史身份改成新意图

相同正文并不足以识别重复。用户可能在新事实下再次保存相同文字；也可能只是某次响应没有收到。若服务器只按 body 去重，会把正常新决定与恢复混淆；若只按版本去重，不同操作也可能共享同一前提。当前协议把这些维度分别保存，客户端明确选择属于哪种情况。

没有回执的失败请求不构成已成功记录。领域校验在持久保存前抛错时，不会留下成功 Response；事务失败也不保留半条回执。客户端仍应保持意图使用清晰，不能把“未占用某键”当作随意变换同键正文的恢复协议。

第 12 章

可信身份、资源所有权与浏览器安全

调查是私人资源。保护它不仅要知道“用户是否登录”，还要知道“这个用户是否能访问这一份调查”，以及“这个浏览器写请求是否来自可信交互”。认证、授权、防伪和内容投影对应不同边界，任何一个都不能凭另一个存在而省略。

12.1 Identity 与 Cookie 分别做什么

Identity 的 UserManager 负责账户存储、密码检查和失败次数。登录成功后，应用构造包含 NameIdentifier 与 Name 的 ClaimsPrincipal，再通过指定 Cookie scheme 签入。后续请求由 Cookie 认证恢复身份，而不是重新把邮箱和密码放进每个业务正文。

源码 backend/EchoCase.Api/Program.cs · L124-126

```
124     var principal = new ClaimsPrincipal(new ClaimsIdentity([new(ClaimTypes.NameIdentifier,
        user.Id), new(ClaimTypes.Name, user.Email!)], "EchoCase"));
125     await ctx.SignInAsync("EchoCase", principal);
126     return Results.Ok(new UserView(user.Id, user.Email!));
```

源码 backend/EchoCase.Api/Program.cs · L25-32

```
25  builder.Services.AddAuthentication("EchoCase").AddCookie("EchoCase", o =>
26  {
27      o.Cookie.Name = "echocase.session"; o.Cookie.HttpOnly = true; o.Cookie.SameSite =
        SameSiteMode.Strict;
28      o.Cookie.SecurePolicy = builder.Environment.IsDevelopment() ||
        builder.Environment.IsEnvironment("Testing") ? CookieSecurePolicy.SameAsRequest :
        CookieSecurePolicy.Always;
29      o.ExpireTimeSpan = TimeSpan.FromHours(2); o.SlidingExpiration = false;
30      o.Events.OnRedirectToLogin = ctx => { ctx.Response.StatusCode = 401; return
        Task.CompletedTask; };
31      o.Events.OnRedirectToAccessDenied = ctx => { ctx.Response.StatusCode = 403; return
        Task.CompletedTask; };
32  });
```

Cookie 使用 HttpOnly 与 SameSite Strict；非开发、非测试环境要求 Secure。会话有效期两小时，不滑动延期。API 未登录时返回 401，避免把 JSON 客户端重定向到一个 HTML 登录页。浏览器可读的邮箱用于展示，资源访问使用稳定用户 Id。

这些配置并不等同完整企业账户生命周期。当前手工 Cookie principal 没有完整实现安全戳持续验证、全设备撤销、多因素认证、确认邮箱和密码重置流程。Identity 提供相关能力，不代表只引用它就自动接上所有能力。评审应读实际注册和验证路径，区分已用机制与框架潜力。

12.2 Owner 必须来自可信上下文

源码 backend/EchoCase.Api/Program.cs · L145-146

```
145 static string Owner(ClaimsPrincipal p) => p.FindFirstValue(ClaimTypes.NameIdentifier)!;
146 static string Key(HttpContext ctx) => ctx.Request.Headers["Idempotency-Key"].ToString();
```

业务请求不接收 OwnerId。端点从已认证 claims 取得 owner，再向下传递。Store 在 SQL 中同时筛选调查 Id 与 OwnerId，未知或不可见统一为 404。它既避免把其他用户的数据读进内存后再判断，也减少通过响应差异探测某个 Id 是否存在的机会。



图 12.1 这些边界共同保护会话应用；Identity 的存在不等于完整账户安全生命周期。

资源过滤要覆盖 Get、List 和 Write，不能只在页面加载时检查一次。即使客户端隐藏了某条按钮，用户仍能直接发请求；即使 Guid 难以猜测，也不构成授权。幂等回执路径同样先验证资源，避免重放绕开所有权。

没有团队或管理员角色是当前业务范围的决定。将来新增共享调查，需要定义成员、权限变化和回执可见性。不能仅删除 owner 过滤，就把私人应用改造成安全协作系统。

12.3 Cookie 会自动发送，所以需要防伪

浏览器会根据站点和 Cookie 规则自动携带会话凭证。服务端看到有效 Cookie，只能确认浏览器拥有会话，不能单凭它证明这次写入是用户在本应用中作出的决定。CSRF 令牌把浏览器可取得的请求令牌与对应 Cookie 联系起来，写 API 要求 X-CSRF-TOKEN。ASP.NET Core 防伪

客户端先从 /api/auth/csrf 获取令牌，再在写请求中发送。登录、注册也经过防伪中间件。登出后客户端重新取得令牌，避免身份切换后沿用旧交互上下文。SameSite Strict 减少跨站发送，但不能代替明确的服务器防伪规则。

CSRF 与 XSS 不同。若攻击者已能在本应用同源脚本中执行任意代码，往往可以读取请求令牌并发起同源请求。HttpOnly 限制脚本直接读取会话 Cookie，不能阻止恶意同源脚本利用现有会话。因此还需要内容输出和脚本来源边界。

12.4 不发送的内容，比隐藏的内容更可靠

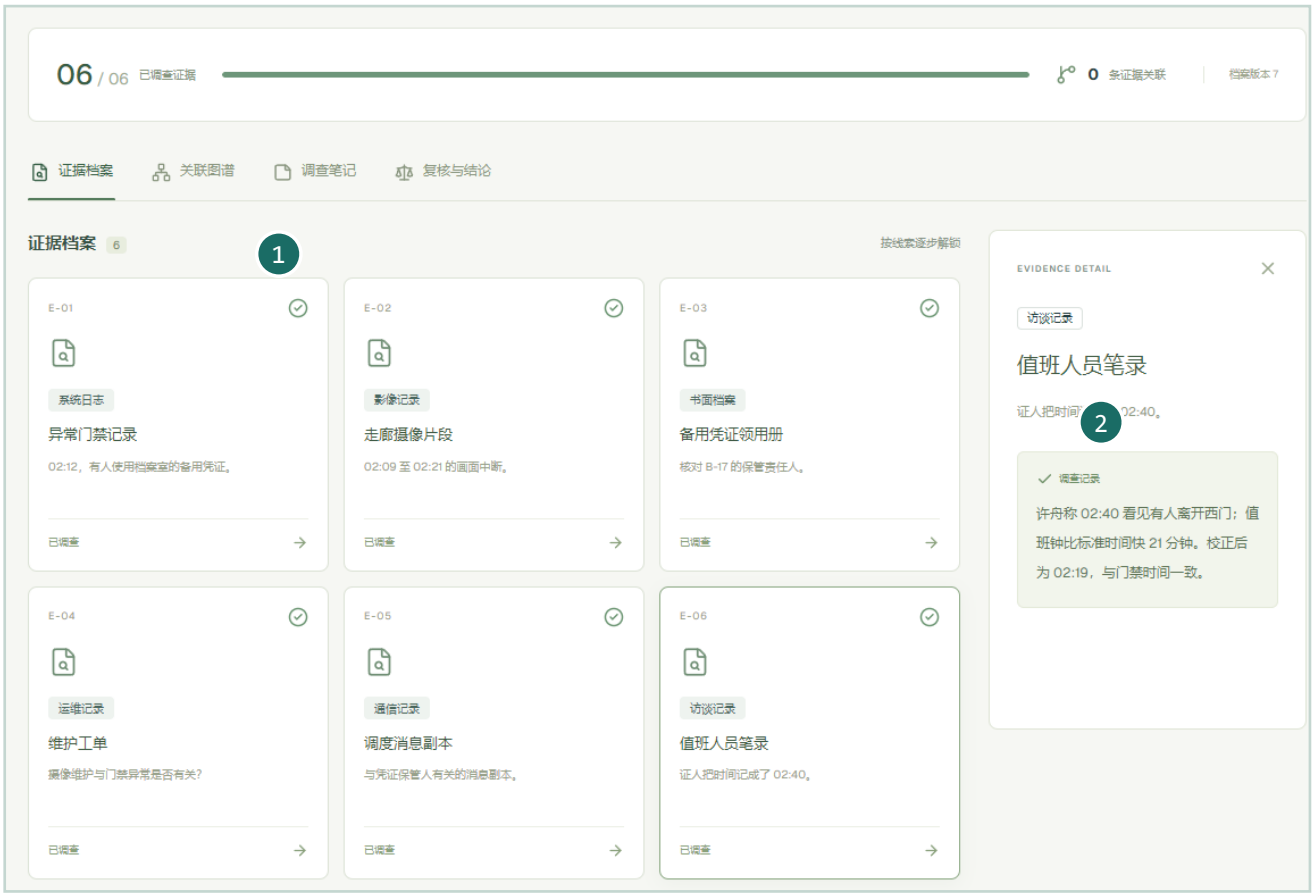


图 12.2 真实证据工作台：①卡片显示已调查状态；②详情正文由调查结果提供。

①可见、已调查、可编辑属于不同条件。

②正文没有在未调查时预先发送给浏览器。

SessionPresenter 只有在证据已调查后才发送 Body。锁定证据的摘要也被替换为统一提示。这个边界成立在服务器投影中，浏览器无需先获得全部秘密再用 CSS 隐藏。界面的锁图标只是解释当前状态，不能独自承担内容保护。

React 默认把普通文本作为文本节点输出，当前笔记与解释没有作为任意 HTML 注入。若未来允许富文本，需要重新定义可接受标记、清理与渲染过程；把字符串类型改成 HTML 并不会自动继承现有安全性。

API 的 no-store 限制 HTTP 缓存行为。Query 缓存则按 owner 与 id 分区，登录和登出时清除非 me 查询，避免同一浏览器切换账户后看到上一人的数据。服务端授权仍然是最终保证，客户端清缓存保护的是界面隔离和合理体验。

12.5 安全头与频率约束的实际含义

CSP 主要限制脚本、连接、图片等来源，并禁止被其他页面作为 frame 嵌入。当前 style-src 允许 unsafe-inline，以兼容实际样式和图谱内联呈现，不能把这份策略形容成完全禁止内联内容。nosniff 与 no-referrer 降低类型猜测和外链信息暴露。

认证端点按远端 IP 固定窗口限流，每分钟十五次；业务写入按用户 Id，每分钟一百二十次。固定窗口在边界附近可能允许短时突发，不是平滑流量保证；进程内限流器也不自动形成多实例全局配额。若入口经过代理，可信转发地址配置会影响 IP 身份，当前代码没有因此获得任意部署下准确客户端地址的保证。

密码失败锁定与限流针对不同成本。前者记录账户失败，后者限制请求速度。错误正文应避免不必要的账户存在性泄露，内部定位则通过 TraceId 关联。把这些行为连成威胁与边界的解释，比罗列“使用了 Cookie、CSP、Identity”更能评估实际保护。

12.6 安全评审回到业务事实

一条修订请求要同时满足：有可信身份，目标属于该身份，写入来自有效防伪上下文，请求形状可接受，领域允许，版本没有过期，意图没有被改写。每一层拒绝的理由不同，客户端的下一步也不同。

安全不是独立放在工程末尾的一页配置。它决定了 owner 如何进入所有层、哪些数据允许进入读模型、缓存如何分区，以及重试能否越过资源边界。后面的 React 设计会继续使用这些事实，而不是把浏览器当作天然可信环境。

12.7 同源开发与文档安全策略

生产静态界面由 API 宿主提供时，页面响应上的 CSP 才约束该页面后续脚本和连接。开发时前端由 Vite 提供，API 的 JSON 响应上带 CSP，并不会自动成为 Vite 页面自己的文档策略。检查某个 API 头存在，不能直接证明所有开发页面执行相同安全边界。

前端 request 使用相对路径与 same-origin 凭证，通过既定开发代理或同源宿主访问 API。浏览器的同源、SameSite 和服务端的身份判断分别有不同范围。它们协作构成会话路径，不能用“后端有 CORS”一类泛泛说法代替真实部署和请求来源分析。

对教材而言，关键是能把每项配置连接到它实际保护的对象：HttpOnly 保护 Cookie 的脚本可读性，CSP 约束文档执行，CSRF 保护写请求意图来源，owner 过滤保护资源可见性。保护对象明确，配置才不只是术语清单。

12.8 失败处理也有信息边界

404 合并不可见与不存在，有助于减少资源探测信息；登录失败用统一文字，不把“用户不存在”与“密码错误”直接展示成两个业务提示。它们不会消除所有时序差异，也不构成完整抗枚举证明，但体现了公开合同对内部事实的选择。

数据库异常则在服务端保留，客户端看到稳定错误语义和编号。若把全部异常堆栈直接发送，调试虽然方便，却会把实现细节变成公开接口的一部分。若只发“失败”两个字，又让用户无法决定下一步。良好的错误合同在可行动解释与内部细节之间建立明确边界。

第 13 章

React 渲染快照与状态所有者

后端已经有明确的状态机，前端仍可能丢数据。原因是浏览器同时处理服务器事实、网络中的操作、用户输入和局部交互。把它们都叫 state，只解决了语法命名，没有解决谁能覆盖谁。React 的渲染模型为这些变化提供执行基础，业务仍要定义各类状态的含义。

13.1 事件回调看到哪一次渲染

React 调用组件函数，依据当前 props 和 state 计算 JSX，再把必要变化提交到 DOM。事件回调与局部变量属于那次渲染的快照。调用 setter 会安排下一次计算，不会把已经执行中的闭包变量改造成未来值。渲染与提交、状态快照

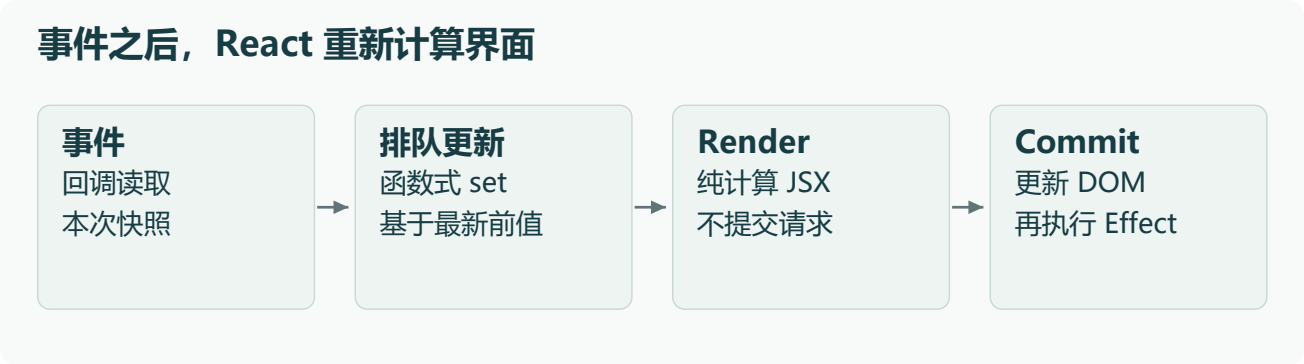


图 13.1 事件闭包不会自动变成下一次渲染的变量；网络回调必须明确携带原操作快照。

若保存按钮回调捕获文本 A，发出请求后用户输入 A+B，晚到回调仍然对应发送 A 的那次操作。它不应因为组件此时显示 A+B，就把 A+B 当成已经提交的正文。反过来，也不该把输入框强行恢复成响应中的 A。网络操作需要保存明确的意图快照，不能依赖“执行回调时随便读取哪个变量”。

函数式 setDraft(d => ...) 接收更新队列中的前一个状态，适合在异步确认时比较当前代次。它解决的是读取最新本地前值的问题，不自动知道这份前值是否对应原操作。操作中仍需携带 draftRevision，业务比较才能成立。更新队列

13.2 Render 与 Effect 的职责边界

渲染应该计算界面，不在组件函数体中直接发起写入。用户按下保存属于事件；浏览器地址变化监听与服务器快照到本地状态的同步属于 Effect 的使用范围。把请求藏进“某字段变了就保存”的 Effect，容易让初始化、重渲染或后台刷新意外产生新命令。Effect 同步

EchoCase 在事件中构造 Operation，交给 mutation；草稿钩子的 Effect 只处理快照到达与已知成功确认。两个 Effect 的前提不同：服务器新快照能更新干净草稿，却不能覆盖脏草稿；保存确认还需检查对应代次。Effect 并不天然比事件更“响应式”，关键是变化由哪个原因触发。

源码 frontend/src/main.tsx · L7-19

```tsx
const queryClient = new QueryClient({
  defaultOptions: {
    queries: { retry: 1, staleTime: 10_000, refetchOnWindowFocus: true },
    mutations: { retry: false },
  },
});
ReactDOM.createRoot(document.getElementById("root")!).render(
  <React.StrictMode>
    <QueryClientProvider client={queryClient}>
      <App />
    </QueryClientProvider>
  </React.StrictMode>,
);
```

QueryClient 在组件外创建，避免每次渲染都新建缓存容器。应用启用 StrictMode，开发时会增加渲染与 Effect 检查，帮助暴露不纯计算和缺少清理的问题；它不表示生产环境每次用户点击都必然执行两次写入。StrictMode

13.3 状态按所有权分开

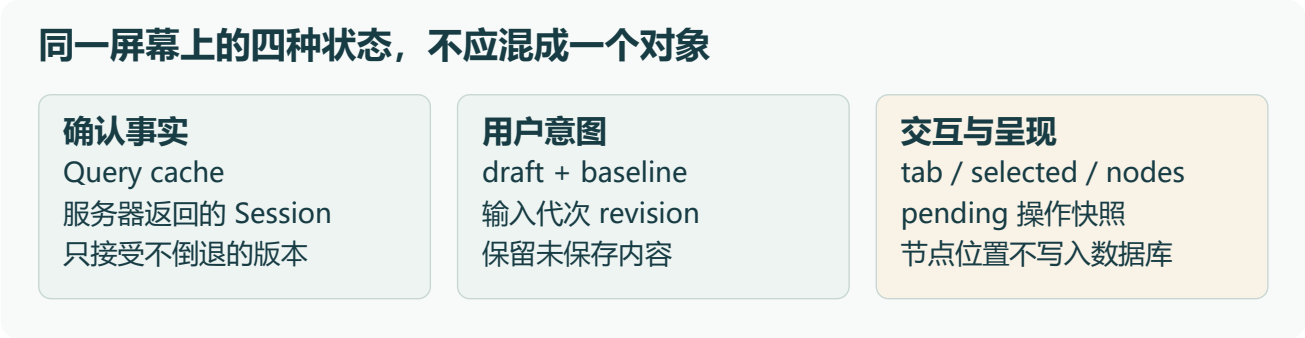


图 13.2 React 负责重绘，Query 负责服务器状态生命周期，草稿钩子负责输入语义。

Query 缓存只保存服务器确认的 Session。Operation 保存本次发送的 route、method、body、version、key 和可选草稿代次。useNotesDraft 保存 text、baseline、version、revision、dirty 与 saved。tab、selected、图节点坐标则只表达当前交互。它们可以出现在同一屏，但更新权限不同。

例如 pending 的保存内容可作为“正在保存此快照”提示，却不进入确认缓存。用户继续输入会改变 draft.text，不会改变 operation.body。后台 refetch 更新 Query，若草稿已脏，不能因此改写 baseline。把这些值分开后，一句“保存成功”才能精确指向其中一个操作，而不是含混地宣布整个页面都已同步。

这种分离也让错误恢复更简单。服务器错误不要求“回滚所有 React state”，因为确认缓存从未写入未经确认的文本；只需保留草稿并展示错误。局部乐观呈现仍然存在，但它明确标注为 pending，不冒充持久事实。

13.4 组件身份决定状态保留多久



图 13.3 切换页签隐藏笔记面板，不卸载草稿；切换调查则使用新组件身份。

源码 frontend/src/App.tsx · L142-147

```
142            <Workspace
143              key={id}
144              id={id}
145              user={user.data}
146              onBack={() => navigate(null)}
147            />
```

App 用调查 Id 作为 Workspace 的 key。切换调查时，React 将它视为不同组件身份，本地草稿和图坐标不会串到另一份档案。相同调查内部切换页签，NotesEditor 保持挂载，只使用 hidden 改变可见性，草稿因此继续存在。状态保留与重置

这是有范围的保留：返回总览使工作台卸载，刷新页面也会重建本地状态。当前实现没有浏览器持久草稿或离开确认。不能把“切换页签不丢输入”扩大成“任何导航与重启都能恢复”。状态生命周期应与产品提示一致。

App 负责身份和导航，Workspace 负责本次调查装配，钩子负责请求与草稿，编辑器负责字段交互。拆分的收益在于不同变化原因被放到不同所有者中，而不是让所有文件行数一样。前端曾把请求、草稿、图谱和页面结构混在一个大组件里；需要修复的正是这些状态之间隐含的覆盖关系。

13.5 TypeScript 辅助哪些判断

Session 和 ReviseLinkRequest 来自生成合同，Operation 则是前端自己拥有的意图类型。两者分开，使 API 字段变化和本地操作生命周期可以分别检查。可选 draftRevision 说明并非所有命令都与笔记代次有关；成功处理必须先判断 route，不能把任意业务成功当成笔记已保存。

unknown 出现在网络和泛型缓存边界，具体使用处再缩小为 Session。当前实现主要依赖生成合同和服务端行为，没有完整运行时 schema 验证。因此类型系统能约束前端源码，却不能对任意失真的成功 JSON 提供证明。第十一章的正文不可读保护只覆盖解析失败，合法 JSON 但字段错误还需要其他策略。

13.6 组件库不是业务状态机

源码 frontend/src/components/ui/button.tsx · L42-55

```
42  function Button({
43    className,
44    variant = "default",
45    size = "default",
46    ...props
47  }: ButtonPrimitive.Props & VariantProps<typeof buttonVariants>) {
48    return (
49      <ButtonPrimitive
50        data-slot="button"
51        className={cn(buttonVariants({ variant, size, className }))}
52        {...props}
53      />
54    );
55  }
```

Button 组合 Base UI 行为、变体样式和原始 props，让 disabled、可访问属性与事件继续由使用者传入。它没有决定“何时允许修订”。NotesEditor 和 LinkEditor 依据业务状态、冲突、长度与 blocked 计算 disabled，再把结果交给控件。

统一视觉样式可以降低重复，也可能掩盖实际组件后端。本实现使用 Base UI 版本的按钮原语，不能因为目录叫 shadcn/ui 就假定内部一定是另一套组件库。阅读真实依赖和组件封装，才能解释焦点、禁用和属性透传如何发生。

React 19 提供许多可选机制，本应用没有为展示版本号而引入所有新 API。这里的核心难点是异步事实与用户意图的分离，基础 Hook 配合明确协议已能表达。下面把组件身份与更新队列接回实际代码，再进入 Query 生命周期。

13.7 状态属于树中的位置，不属于函数局部变量

组件每次调用都会创建新的局部变量，React 仍能保留输入，是因为它将 Hook 状态关联到组件在树中的身份和 Hook 调用顺序。函数不是一个一直挂在栈上的窗口对象；下一次渲染会重新执行函数，再取回相应状态。

这解释了两个实际选择。useNotesDraft 总在 NotesEditor 的固定调用位置运行，不能按某个临时 tab 条件决定是否调用 Hook；Workspace 的 key 随调查 Id 改变，表示应该建立另一组状态。把 key 随每次服务器 version 改变则会把每次刷新变成卸载重建，草稿也会随之丢失。

渲染过程中不应该修改已有 state 对象。setDraft 返回新对象，让 React 和使用方能够把变化识别为新状态；旧操作快照也不会因为共享可变引用被悄悄改写。当前 Operation 的 body 在构造时来自文本值，后续输入不会变动这个字符串快照。

13.8 为什么函数式更新仍需业务条件

setDraft(d => ...) 只保证更新函数获得队列中的适当前值。它不会推断 current Session 属于哪个请求，也不会自动知道某个 success 是否已被处理。代码因此同时使用操作 key、draftRevision 和 result.version，分别检查重复确认、输入代次和服务器先后。

若只使用函数式更新却无条件 dirty=false，晚到响应仍会丢掉新编辑状态；若只比较版本却不比较代次，同一次保存等待期间的新输入仍可能被误确认。React 提供正确组合状态更新的机制，领域化的客户端模型提供判断条件。

开发检查增加 Effect 执行机会，也推动我们让确认逻辑具有明确身份。acknowledged ref 不是为了掩盖所有重复 Effect，而是记录“这个成功键已经用于确认”，让后续重渲染不重复产生可见状态变化。其他 Effect 仍需按自身资源正确清理，例如地址监听的移除。

第 14 章

TanStack Query 与单调确认缓存

服务器状态缓存并非把最后到达的 JSON 存起来。网络可能让旧查询晚到，幂等重放可能返回旧成功，后台刷新可能在用户编辑时取得新版本。Query 管理请求生命周期，EchoCase 还要定义哪些结果有权更新当前事实。

14.1 查询键属于资源身份

源码 frontend/src/workspace/useInvestigation.ts · L24-31

```
24  const queryKey = ["session", owner, id];
25  const query = useQuery({
26    queryKey,
27    queryFn: ({ signal }) =>
28      request<Session>(`/api/investigations/${id}`, { signal }),
29    structuralSharing: latest,
30    refetchInterval: 30_000,
31  });
```

键为 ["session", owner, id]，包含用户与调查标识。查询函数消费 AbortSignal，再把它传给 request/fetch。三十秒轮询与窗口聚焦刷新让页面有机会得知其他窗口修改，但不是实时推送；用户仍可能在两次读取之间基于旧快照作决定。查询键

缓存不能替代服务端授权。owner 进入键是为了隔离浏览器中的快照，服务端仍按可信身份筛选资源。登录和登出还会清除非 me 缓存，避免同一浏览器账户切换后的展示泄露。两个边界相互配合，而非互相替代。

14.2 “后来到达”不代表“后来发生”

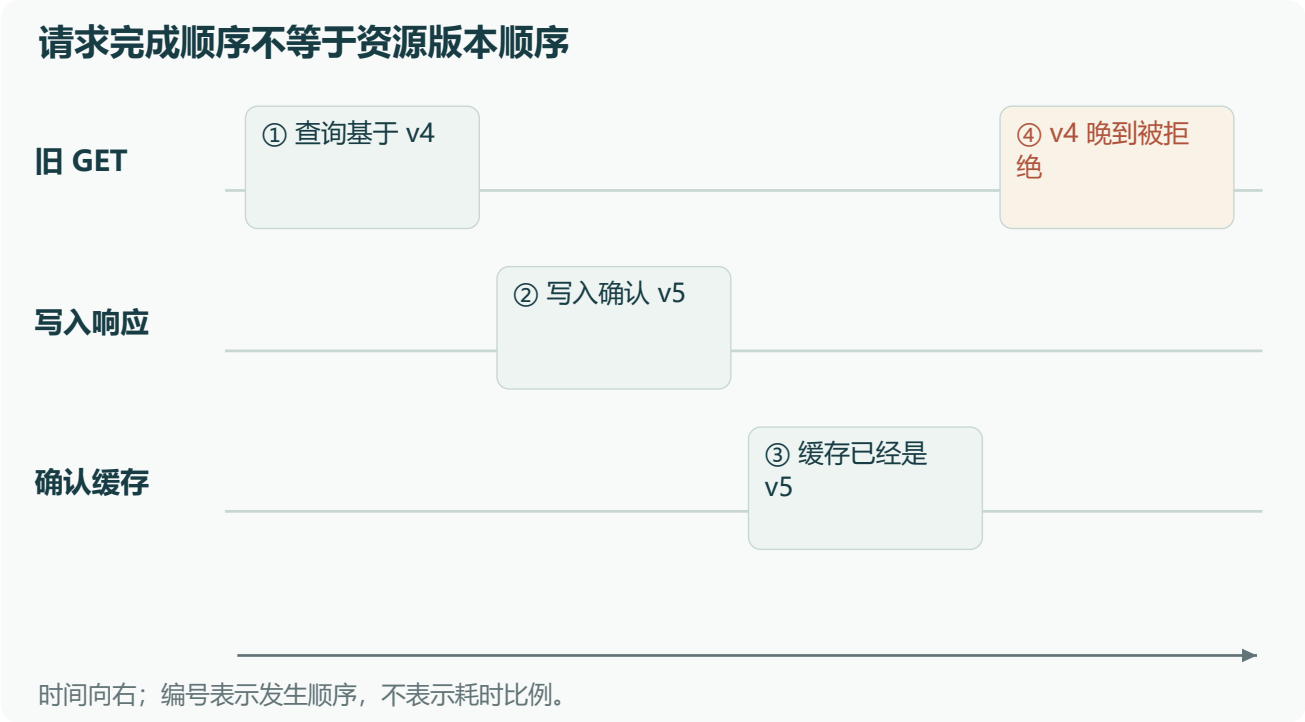


图 14.1 取消减少过期读取；latest 的版本比较还保护无法取消的晚到结果和旧回执。

旧 GET 可能在 mutation 开始前发出，结果却在新写成功后回来。如果直接覆盖，页面从版本 5 倒退到 4。取消旧查询能减少这个窗口，但取消需要底层真正使用 signal；即使取消生效，也还存在旧回执等无法用同一种方式消除的结果。

源码 frontend/src/workspace/useInvestigation.ts · L16-20

```
16  function latest(old: unknown, incoming: unknown) {
17    const previous = old as Session | undefined;
18    const next = incoming as Session;
19    return previous && previous.version > next.version ? previous : next;
20  }
```

latest 比较资源版本，保留更高版本。它作为 structuralSharing 处理查询结果，也用于 mutation 成功后的 setQueryData。这样，相同资源键中的确认版本不会因晚到旧结果而倒退。这个保证依赖版本在同一调查身份内单调前进，以及响应符合合同；它不是对任意内容冲突的通用合并算法。

版本相同时当前函数接受 incoming，没有逐字段证明两个同版快照内容一致。正常业务协议应让相同版本代表同一已确认状态；如果出现同版不同内容，应调查写路径是否绕过了版本推进，而不是让缓存临时猜测哪个字段更可信。

14.3 mutation 是一段生命周期



图 14.2 等待失效刷新让编辑器在看到确认状态后关闭；它仍不提供跨浏览器原子性。

源码 frontend/src/workspace/useInvestigation.ts · L40-55

```
40      onMutate: async () => {
41        await cache.cancelQueries({ queryKey });
42      },
43      onSuccess: (result) => {
44        cache.setQueryData(queryKey, (old: Session | undefined) =>
45          latest(old, result),
46        );
47      },
48      onSettled: async () => {
49        await Promise.all([
50          cache.invalidateQueries({ queryKey }),
51          cache.invalidateQueries({ queryKey: ["sessions"] }),
52        ]);
53      },
54      retry: false,
55    });
```

onMutate 先等待 cancelQueries，避免旧读取轻易覆盖后续确认。mutationFn 发原 Operation。onSuccess 把响应按版本单调合并到确认缓存。onSettled 再使当前调查和总览失效，并 await 两项刷新；mutation 仍保持 pending，直到这些回调的 Promise 完成。取消查询、写后失效

等待刷新有实际交互效果。LinkEditor 通过 mutateAsync 保存，在 Promise 完成后关闭；如果只“发起刷新”就立刻关闭，用户可能看到旧关系短暂出现，或在列表尚未确认时进行下一项操作。这里把完成时机扩展到必要的确认读取，降低这种界面歧义。

失效不表示所有缓存立即都被重写。活跃查询会按库的规则获取，无活跃观察者的查询也有各自生命周期。当前正在显示的调查具有观察者，因此它是关键确认路径；总览前缀失效则使对应用户列表在需要时重新读取。应按真实使用方式解释行为，避免把 invalidate 当成跨应用同步广播。

14.4 为什么不把草稿乐观写入缓存

在发送笔记时直接 setQueryData(notes = draft) 看起来响应迅速，却会让 Query 中同时存在“可能提交”和“已确认”的数据。失败回滚还可能覆盖用户在等待期间刚输入的新内容，尤其多个请求或 refetch 交错时，快照恢复的前提非常脆弱。

源码 frontend/src/workspace/NotesEditor.tsx · L30-33

```
30  const optimistic =
31    pending && operation?.route === "notes"
32      ? (operation.body as { notes: string }).notes
33      : undefined;
```

EchoCase 将正在保存的 operation.body 作为局部提示投影。输入框继续显示 draft.text，确认缓存仍是服务器事实。这保留了及时反馈，却不需要把未经确认的值伪装成持久结果。“乐观”因此不是只有改缓存一种实现方式，关键在于用户能否分辨事实与尝试。

14.5 读取失败与未知写入是不同状态

若轮询失败，页面可以保留最后确认快照并展示读取错误，不能据此说本地草稿已失效。若写入结果未知，blocked 则阻止产生新的不同写意图，并提供原样重试。查询的自动重试一次与 mutation 的 retry false 也因此具有不同理由。

一次普通读取可以重复，不会再推进领域版本；自动重复命令则必须保证同一意图。当前选择显式让用户重试未知写入，原 variables 不变。更复杂的离线队列可以另行设计，但不应该在没有持久化意图协议时，仅把 retry 设置为 true 就宣称可靠恢复。

14.6 确认缓存与草稿之间还差一层语义

即使 Query 永不倒退，也不能直接把每个新 Session.notes 填回输入框。用户可能有基于旧版本的脏草稿，或者在自己的保存等待期间继续编辑。缓存解决“已确认事实的先后”，草稿钩子解决“这个事实能否确认我手上的输入”。

资源版本 Version 追踪服务器，revision 追踪本地输入代次；只有同时理解它们，才能正确处理晚到成功与后台变化。进入草稿模型前，还需明确刷新失败与操作完成对缓存意味着什么。

14.7 失效刷新失败时，哪些事实仍然成立

命令已经得到成功响应，说明这次持久决定成立；随后 GET 失败，不会把该提交撤销。缓存可能暂时保留命令返回的确认快照，并显示读取错误。它仍是可信的历史事实，只是不保证此刻没有更新版本。

若成功结果来自旧回执，而当前缓存更高，latest 保留高版。刷新失败也不能成为倒退的理由。界面需要分别理解“原操作确认了”和“最新读取暂时失败”，避免把后者文字误写成原保存一定失败。

Query 的后台状态能说明请求是否正在获取，却不能回答草稿能否重设基线。这个判断继续由草稿钩子拥有。将所有 isFetching 都当作编辑禁用条件，会让三十秒轮询不断打断用户；将所有刷新结果都直接写输入框，则会破坏输入。

14.8 操作完成边界影响组件行为

LinkEditor 的 await 依赖 mutateAsync 返回 Promise，而 Promise 的完成包含实际回调生命周期。当前 onSettled 返回失效刷新的 Promise，因此表单直到必要确认结束才关闭。若未来把回调改成 void fire-and-forget，编辑器源码即使一行未变，用户看到的完成时机也会改变。

这说明可维护性不仅是类型签名。一个 Promise 在何时完成，是跨组件合同；取消是否向下传，是查询合同；版本是否可单调比较，是缓存合同。重构这些位置需要保留语义，并针对实际交互反例验证。

当前 useInvestigation 对同一工作台的一项 mutation 提供 blocked，减少重叠不同命令。它没有跨标签页协调本地锁，也没有取代数据库仲裁。浏览器层减少歧义，服务器层保证事实，两种职责仍保持清楚。

第 15 章

草稿基线、输入代次与显式冲突

保存按钮最容易作出的错误承诺，是收到任何成功响应就把当前输入标为已保存。服务器只确认发送出去的正文，不知道用户在等待时又输入了什么。EchoCase 用基线和输入代次，把“用户正在写什么”与“某次操作确认了什么”区分开来。

15.1 一份草稿需要记录哪些事实

源码 frontend/src/workspace/useNotesDraft.ts · L9-16

```
  const [draft, setDraft] = useState({
    text: current.notes,
    baseline: current.notes,
    version: current.version,
    revision: 0,
    dirty: false,
    saved: false,
  });
```

text 是当前输入；baseline 是开始编辑时的已确认笔记；version 是这份基线对应的调查版本；revision 是本地编辑代次；dirty 表示用户有待确认的编辑意图；saved 只表达当前可展示的保存确认。它们不是六份重复数据，而是回答六个不同问题。

dirty 使用显式布尔值。用户输入后又把文字删回原样，也可能仍在进行一次编辑会话；后台刷新不应因为字符串碰巧相等就偷偷改变她的基线。若产品希望“恢复原文自动视作未编辑”，可以另行定义，但必须一起考虑后台变化和代次确认，不能只替换成一个字符串比较。

源码 frontend/src/workspace/useNotesDraft.ts · L63-71

```
    conflict: draft.dirty && current.version > draft.version,
    edit: (text: string) =>
      setDraft((d) => ({
        ...d,
        text,
        dirty: true,
        saved: false,
        revision: d.revision + 1,
      })),
```

每次 edit 增加 revision，清除 saved，并保持 dirty。函数式更新让事件与异步确认排队时仍基于当前草稿前值。资源 Version 与 revision 互不替代：用户可能在服务器版本 4 上编辑十次，也可能一次未输入就收到版本 8。

15.2 发出保存时冻结哪一份东西

源码 frontend/src/workspace/useNotesDraft.ts · L81-90

```
81      operation: (confirmCurrent = false): Operation => ({
82        route: "notes",
83        method: "PUT",
84        body: { notes: draft.text },
85        key: crypto.randomUUID(),
86        version: confirmCurrent ? current.version : draft.version,
87        draftRevision: draft.revision,
88      }),
89    };
90  }
```

operation 捕获正文、基线版本、随机键和 draftRevision。请求发出后，这个对象不随输入框继续变化。正常保存使用 draft.version；用户显式确认冲突后才使用 current.version。两者都会建立新键，因为那是一次新决定，而原请求重试仍由 mutation.variables 原样保存。

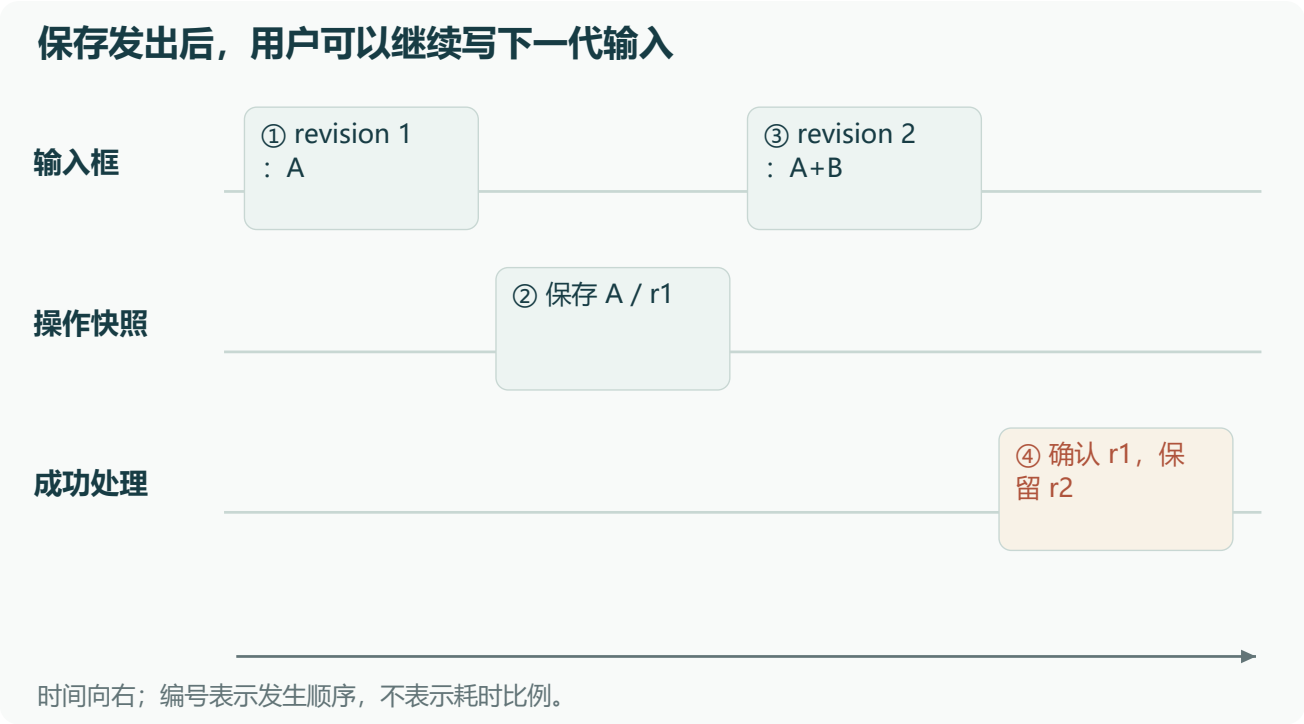


图 15.1 成功只确认被发送的代次；输入框当前文本不等于该次保存的请求正文。

把时间线展开：初始服务器版本 1、笔记为空；用户输入 A，revision 为 1；按保存发出 A/r1；等待期间继续输入 A+B，revision 为 2；服务器确认 A，版本为 2。正确结果是基线更新成 A/v2，输入框保持 A+B，dirty 仍为 true。

时刻	确认服务器	本地输入	草稿基线与状态
开始	v1 / 空	空	v1 / clean

时刻	确认服务器	本地输入	草稿基线与状态
第一次输入	v1 / 空	A / r1	v1 / dirty
保存发出	v1 / 空	A / r1	操作固定 A、v1、r1
等待中再输入	尚未确认	A+B / r2	仍基于 v1，保留新文本
A 保存确认	v2 / A	A+B / r2	基线变 A/v2，仍 dirty

15.3 成功确认必须对应代次

源码 frontend/src/workspace/useNotesDraft.ts · L38-59

```
38      const { result, operation } = success;
39      // A replay confirms that an earlier operation committed; it cannot say that
40      // those notes are still current. Keep the draft for an explicit decision.
41      if (result.version < current.version) return;
42      setDraft((d) =>
43        d.revision === operation.draftRevision
44          ? {
45              ...d,
46              text: result.notes,
47              baseline: result.notes,
48              version: result.version,
49              dirty: false,
50              saved: true,
51            }
52          : {
53              ...d,
54              baseline: result.notes,
55              version: result.version,
56              dirty: true,
57              saved: false,
58            },
59      );
```

当当前 revision 等于发送时 draftRevision，响应可以确认这份输入：使用服务器规范化的 notes，更新基线与版本，清 dirty，显示 saved。若代次已增长，响应只推进本次操作能够确认的基线，保留当前 text 和 dirty。于是用户能在保存期间继续工作，而无需锁死输入框。

此外，每个成功键只确认一次，acknowledged ref 防止后续渲染重复处理同一个操作。旧回执若 result.version 低于 current.version，会提前返回，不把过去成功当成当前内容。这里不需要让 ref 驱动渲染，它只是记录一项处理身份；可见状态仍在 state 中。



图 15.2 真实晚到保存结果之后的界面：服务器确认旧快照，输入框仍保留后来输入。

①文本包含“等待时继续记录的新线索”。

②有未保存修改；成功响应没有把新一代输入错误标为已保存。

真实运行中，服务器笔记为“先提交的观察”，输入框仍有“先提交的观察；等待时继续记录的新线索”。截图中“有未保存的修改”与可再次保存的按钮共同说明，新输入没有被晚到响应清掉。一个泛泛的成功 toast 无法表达这项区别。

15.4 后台刷新不能静默替换基线

源码 frontend/src/workspace/useNotesDraft.ts · L18-29

```
18  useEffect(() => {
19    setDraft((d) =>
20      d.dirty || (d.version === current.version && d.text === current.notes)
21        ? d
22        : {
23            ...d,
24            text: current.notes,
25            baseline: current.notes,
26            version: current.version,
27          },
28    );
29  }, [current.notes, current.version]);
```

干净草稿可以跟随 current.notes 与 current.version，因为用户没有尚未处理的编辑。脏草稿则保持 text、baseline 和 version，让后台新事实只进入确认缓存。否则用户本来基于版本 1 写出的判断，会在不知情时被包装成基于版本 2 的保存，绕过冲突确认。

conflict 的条件是 dirty 且 current.version 大于 draft.version。它可能由另一窗口修改笔记触发，也可能由同一调查的其他成员改变触发。由于版本属于整个聚合，界面保守要求核对。当前实现没有细分“只有关系改变所以笔记可自动合并”，这与后端根版本策略一致。



图 15.3 三方展示支持判断，没有实现自动文本合并；新确认也仍可能再次遇到并发。



图 15.4 真实脏草稿冲突：版本 1 的草稿保留，服务器已经是版本 2。

①输入框保留“本窗口的待核实推断”。

②比较区同时展示空基线、另一窗口的事实与本地草稿。

③采用服务器内容或显式确认草稿，是两种新的用户决定。

真实场景中，基线笔记为空且版本为 1；本地草稿是“本窗口的待核实推断”；另一窗口已经保存“另一窗口已经核实的事实”，当前为版本 2。比较区将三份内容并排解释，用户才知道发生了什么。只显示“有冲突，请刷新”，会让她无法判断刷新是否会删除输入。

15.5 两种确认是两种决定

“采用服务器内容”丢弃本地草稿，按当前快照重置 text、baseline、version，并增加本地代次，防止旧操作确认混入。它不需要发新写入，因为只是本地选择。

“确认保留草稿并基于当前版本保存”则建立新的 Operation，使用当前服务器版本和新键。它不是重试旧请求，而是用户看过差异之后作出的新决定。如果确认之后又有其他请求提交，仍可能再次得到 412；界面确认不能冻结世界。

当前比较没有实现自动文本合并，也没有识别段落级冲突。对于短笔记，显式选择更容易解释；若扩成长文协作，需要新的合并模型与用户体验，而不是把 baseline/version 逻辑悄悄移除。

15.6 输入可继续，不代表所有操作都能继续

未知写入期间可以继续编辑笔记，但 blocked 阻止新的保存和其他业务写入，直到原操作有明确结论。这样既保护工作连续性，也避免在未知结果之上叠加更多难以辨认的决定。LinkEditor 当前在 disabled 时会禁用自己的字段，与笔记策略并不完全相同，不能把笔记行为推广成全应用所有表单的保证。

草稿保存在工作台挂载生命周期中，关闭页面或切换调查会失去未保存内容。这是当前实现的实际边界。增加持久草稿时，还要决定用户隔离、版本过期、隐私与恢复提示；把一个字符串写进 localStorage 并不能自动解决这些问题。

从一次晚到响应看出去，可以发现前端一致性并非后端事务的附属。后端保护持久事实，前端保护尚未持久的用户意图。两者都正确，一次保存才值得用户信任。

15.7 把确认逻辑读成状态转移表

到达事件	当前前提	草稿应怎样变化
新服务器快照	clean	文本与基线跟随确认值
新服务器快照	dirty	保留文本与旧基线，比较版本
本次保存成功	同输入代次、结果不旧	接纳规范文本并清 dirty
本次保存成功	已有后续输入	只推进已确认基线，保留当前文本
历史回执成功	结果低于当前版本	不用它清除草稿

到达事件	当前前提	草稿应怎样变化
用户采用服务器	明确放弃本地意图	重置到当前快照，推进代次
用户确认保留	已看过当前差异	新键、新前提，形成新操作

表格不是另一套实现，而是把代码中的条件翻译成可审阅语义。某次修改如果让一个分支无法对应明确用户含义，就说明状态模型可能重新混淆了。尤其不能把 success 作为所有分支的共同清理入口。

15.8 自己保存成功，却又出现冲突

设本窗口保存 A/v4，服务器产生 v5；响应回来之前，另一窗口把调查推进到 v6；本窗口的确认刷新先取得 v6。随后原操作完成，result 仍为 v5。当前实现拒绝用这个旧成功清理草稿，因此即使原 A 确实保存过，用户仍需核对现在的事实。

这不是“服务器否认自己的成功”，而是两种结论同时成立：原操作已经提交，现在资源已经超越它。若界面只容纳一个 saved 布尔值，就很难表达这种关系。current、result、baseline 和 operation 各自保留来源，才能作出保守且可解释的选择。

用户继续输入也可能与保存规范化交错，例如服务器 Trim 去掉前后空格。只有同代次确认才将输入框替换为规范结果；有后续输入时，保留当前文本更重要。基线更新说明原操作已确认，不表示未经发送的新文本已被服务器规范化。

15.9 为什么不在 Effect 中自动“修复”冲突

自动把 draft.version 改成 current.version，表面上能解除冲突提示，却抹掉了用户尚未看过的新前提。自动把 draft.text 改成 current.notes，则直接放弃本地意图。两种更新都需要用户或明确业务策略授权，不能仅因为 Effect 可以运行就执行。

当前三方比较把决定交回用户。复杂协作编辑可以引入更细文本模型，但也必须定义合并冲突、撤销和身份语义。对短笔记，少量明确状态比一套看似自动、实际无法解释覆盖来源的同步逻辑更可靠。

第 16 章

React Flow：业务关系与本地布局

证据图很容易让开发者把 Node 和 Edge 当成业务模型。控件需要坐标、选中态、缩放和连接手势，业务则需要已调查证据、稳定关系身份和受约束的修订。EchoCase 通过明确映射让两套模型合作，避免拖一次节点就制造一个数据库版本。

16.1 控件模型服务于交互



图 16.1 source / target 是技术字段；领域按 ordinal 规范端点，避免 A-B 与 B-A 重复。

节点来自 session.evidence 中已调查的证据，Id 沿用证据标识；边来自 session.links，Id 沿用关系 Guid。这样，React Flow 的选中对象能稳定指向已有业务关系。节点坐标、缩放和本地选中态则没有出现在 SessionView 中。

源码 frontend/src/EvidenceGraph.tsx · L42-45

```
42          position: old.find((n) => n.id === e.id)?.position ?? {
43            x: 60 + (index % 3) * 260,
44            y: 70 + Math.floor(index / 3) * 190,
45          },
```

收到新 session.version 时，Effect 重新映射节点，先按 Id 保留旧位置，新增节点才使用网格默认位置。普通服务器刷新因此不会让用户刚整理的节点全部跳回初始坐标。节点被卸载后位置不再保留，跨页面共享布局也没有实现。

受控节点通过 onNodesChange 和 applyNodeChanges 更新本地数组，控件不直接写数据库。React Flow 交互模型

16.2 技术方向与业务方向分开

React Flow 的 Connection 有 source 和 target，表示手势从哪个 Handle 开始。业务关系却是无向的：门禁与领用册之间的矛盾，不因为拖动方向不同就变成另一条事实。根会规范端点次序，数据库唯一键阻止重复。

源码 frontend/src/EvidenceGraph.tsx · L60-72

```
60  const edges: Edge[] = session.links.map((l) => ({
61    id: l.id,
62    source: l.sourceId,
63    target: l.targetId,
64    label: l.kind === "Corroborates" ? "相互印证" : "存在矛盾",
65    type: "smoothstep",
66    animated: false,
67    style: {
68      stroke: l.kind === "Corroborates" ? "#4b8378" : "#b66848",
69      strokeWidth: selected === l.id ? 3 : 1.8,
70    },
71    labelStyle: { fill: "#163b42", fontSize: 12 },
72  }));
```

边用关系类型决定文字和颜色，没有添加方向箭头。选中状态只加粗线宽，不改变 Kind。图例和文字都保留类型信息，避免只凭颜色判断。表单式起终点选择与建联按钮提供另一条操作路径，减少对拖拽手势的依赖。



图 16.2 真实图谱：节点布局是本地呈现，证据关系另有稳定身份。

①连线表达关系类型，没有业务方向箭头。

②下面的关系列表提供可访问的修订与删除入口。

真实图中的关系列表与画布共享服务器快照。画布适合看整体联系，列表适合读解释与执行精确操作。图形不是唯一可访问入口；但当前实现也没有宣称完成所有键盘和辅助技术场景，复杂画布仍需持续验证。

16.3 修订编辑器冻结打开时版本

源码 frontend/src/workspace/LinkEditor.tsx · L33-47

```
        onClick={() =>
          setDraft({
            kind: link.kind,
            rationale: link.rationale ?? "",
            version,
          })
        }
      >
        修订关联
      </Button>
    );
  const length = Array.from(draft.rationale.trim()).length;
  const changed =
    draft.kind !== link.kind || draft.rationale.trim() !== link.rationale;
  const conflict = version !== draft.version;
```

打开编辑时，LinkEditor 把 kind、rationale 和当前调查版本放入本地 draft。之后如果父组件收到新版本，draft.version 不自动变化，conflict 会阻止直接保存。否则后台刷新可能悄悄把用户在旧事实下输入的解释改成新版本命令，与笔记章节讨论的静默 rebase 问题相同。

前端用 Array.from(trimmedText).length 统计标量值，与领域 EnumerateRunes 的业务单位对应。类型和值都没有变化时不允许保存，解释不足二十或超过五百也会禁用。服务端仍执行同样业务规则，因为 UI 不能保证所有调用都经过该控件。

16.4 等待实际完成，再关闭表单

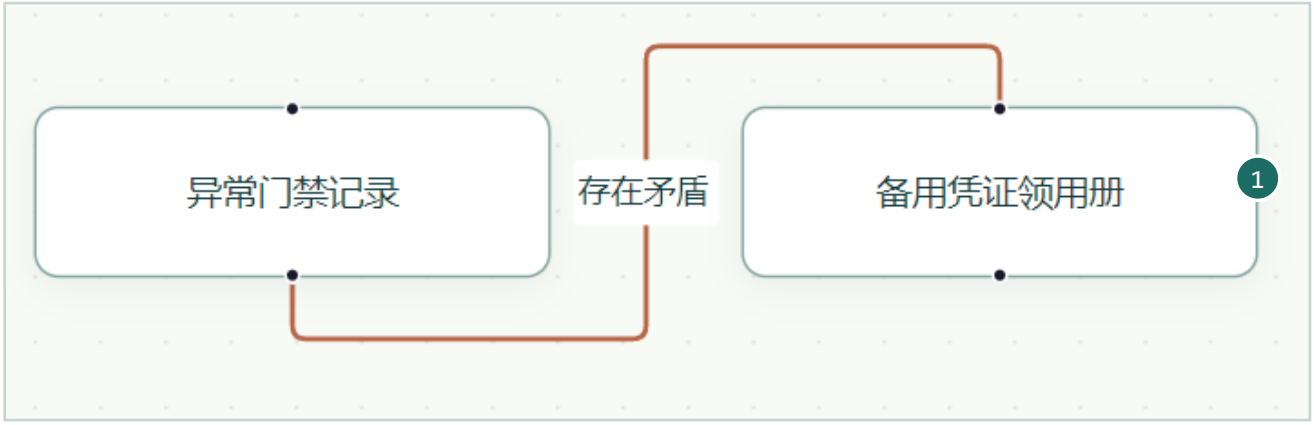
源码 frontend/src/workspace/LinkEditor.tsx · L81-92

```
        onClick={async () => {
          try {
            await onSave(
              link.id,
              { kind: draft.kind, rationale: draft.rationale },
              draft.version,
            );
            setDraft(null);
          } catch {
            /* Workspace displays the protocol error. Keep this draft. */
          }
        }}
```

onSave 返回 Promise，内部是 Workspace 的 mutateAsync。成功路径还会等待 Query 的 onSettled 失效刷新，之后才 setDraft(null)。异常被捕获后保留本地解释，由 Workspace 展示协议错误。错误处理因此保护的是用户劳动，而不仅是控制台没有未处理 Promise。

版本变化时，当前编辑器要求用户保留或复制解释、取消后核对当前关联，再打开修订。它没有笔记那样的三方比较和“确认当前版本”按钮。两个编辑器共享保护基线的原则，具体交互仍按当前复杂度选择，不能在教材中替它补写尚未存在的功能。

同一真实界面的画布局部



同一界面的关系列表解释



图 16.3 真实修订完成后的图谱：同一关联改为矛盾关系，解释显示在列表中。

①关系的端点未更换，类型改变后边的视觉编码同步改变。

②解释随服务器确认快照显示；修订时间保存在返回数据中。

真实修订结果中，原有两端保持，关系变成 Contradicts，图边的标签与颜色随确认快照变化，列表出现解释。修订没有创建替代 Guid，也没有把拖拽布局写回服务器。身份、类型、解释和位置分别拥有稳定含义。

16.5 窄屏与规模改变设计条件

原窄屏界面：上半部

原窄屏界面：下半部



图 16.4 真实窄屏图谱：工具栏换行，节点区域可缩放与拖动。

①导航与内容适应窄视口。

②图形手势之外仍需保留表单式建联入口。

窄屏工具栏换行，画布仍能缩放和拖动。这解决的是当前六节点案件的可用性，不能证明大规模图布局、自动聚类或多人布局同步。若未来关系数量大幅增长，默认网格、完整列表和一次加载全部图都需要重新评估。

选择图形库提供的是交互基础，领域约束仍由应用拥有。把边从画布拖出来，不代表业务关系已经成立；只有命令通过规则、版本、幂等和事务，新的确认快照才有权进入图。这个回路与普通表单完全相同，只是呈现方式不同。

16.6 派生边与局部节点采用不同更新方式

edges 每次从 session.links 派生，关系类型变更后自然得到新标签与颜色；nodes 需要保留用户拖动坐标，不能每次直接抛弃旧数组。因此一个是确认业务事实的投影，一个是带本地记忆的呈现模型。两者采用不同实现，是因为状态所有者不同。

若把 edges 也长期复制到另一份独立 state，却忘记在服务器修订后更新，就会出现列表已显示矛盾、画布仍显示印证的分叉。若把 nodes 完全从服务器快照重建，又会让普通刷新重置布局。正确的“单一来源”不是全页面只允许一个 state，而是每项事实都有明确的权威来源。

选中关系也只是本地引用。当删除请求失败，服务器关系仍存在，界面下一次确认仍应以服务器为准；本地 selected 清除不代表业务删除成功。用户交互状态可以先变化，但不能因此把数据库事实一并改写。

16.7 图形反馈要连接业务语言

绿色和橙色只是一种视觉编码，文字“相互印证”“存在矛盾”才把控件值接回业务。解释位于关系列表，帮助用户理解为什么改变类型。图形越直观，越容易让人把“已经画出来”理解成“已经保存”，所以 pending、禁用和确认刷新同样重要。

当前图谱选择有限节点、简单默认布局和表单备用入口，适合教学案件。自动布局、路径分析或关系置信度都可以成为未来需求，却应从具体用户问题出发。增加动画和算法不能替代现有关联身份与保存语义。

第 17 章

从实际 EF 查询读懂 PostgreSQL 执行计划

“增加索引会变快”不是足够的工程结论。需要先确定应用真正发出了什么 SQL，再解释索引改变了哪部分工作，最后限定测量结果能支持什么。EchoCase 的查询观察取自实际 Store.ListAsync，而非手写一条看起来相似的简化查询。

17.1 查询形状先决定问题

源码 backend/EchoCase.Infrastructure/PostgresInvestigationStore.cs · L20-21

```
20      public async Task<IReadOnlyList<SessionView>> ListAsync(string ownerId, CancellationToken
        ct) =>
21          (await Graph.AsNoTracking().Where(x => x.OwnerId == ownerId).OrderByDescending(x =>
        x.UpdatedAt).Take(50).ToListAsync(ct)).Select(SessionPresenter.Map).ToArray();
```

List 按 owner 筛选调查，按 UpdatedAt 倒序取前五十个，再加载发现和关系。实际 SQL 把根选择放在子查询中，外层 LEFT JOIN 两个集合，并为物化增加排序。下面仅摘出根选择部分，保留真实参数名。

运行摘录

```
FROM "Investigations" AS i
WHERE i."OwnerId" = @ownerId
ORDER BY i."UpdatedAt" DESC
LIMIT @p
```

该次参数为 owner probe-owner-17、limit 50。若把 LIMIT 放到连接后的行上，就可能截断同一根的子项，不能保证取到五十个完整调查。EF 的子查询形状正是为了保留根数量语义，因此比较性能必须使用这个真实结构。

映射已有 (OwnerId, UpdatedAt DESC) 复合索引。观测比较的是已有设计的机制，不是本轮给产品新加了优化。QueryProbe 通过拦截真实 DbCommand 取得 SQL 和参数，再交给 EXPLAIN；正文只保留解释所需的数值与图形。

源码 backend/EchoCase.Infrastructure/EchoDbContext.cs · L50-50

```
50          i.HasIndex(x => new { x.OwnerId, x.UpdatedAt }).IsDescending(false, true);
```

17.2 三种路径各做了什么

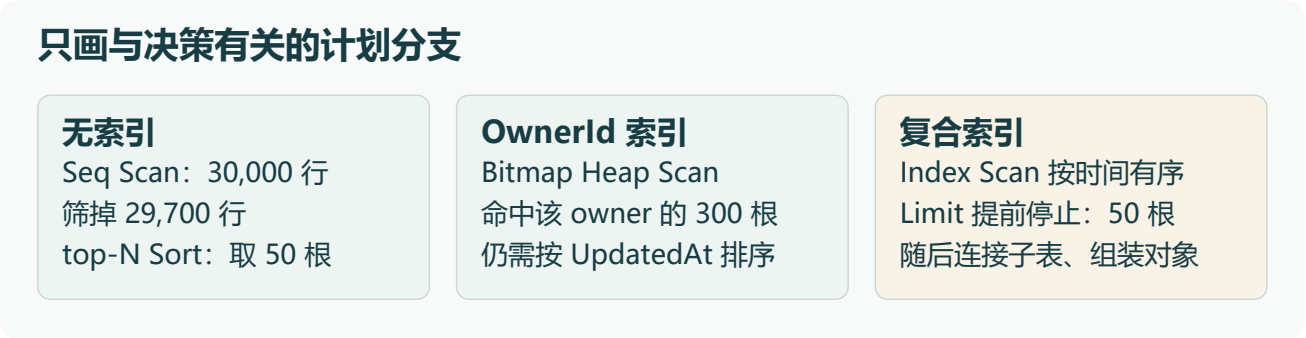


图 17.1 代表性真实计划的简化结构；根选择分支消除排序，不意味着整个查询再无排序。

无 owner 索引时，代表计划顺序扫描三万条根，过滤掉 29,700 条，留下该用户的三百条，再用 top-N heapsort 选出五十条。LIMIT 减少输出，不代表扫描器事先知道哪些是最新的五十条，候选仍需要检查。

只有 OwnerId 索引时，Bitmap Index Scan 与 Bitmap Heap Scan 能找到该 owner 的三百条，减少无关根的读取。但位图访问结果不按 UpdatedAt 排列，所以仍需排序，根数量较多时这部分工作不会凭索引存在而消失。

复合 B-tree 的前导 OwnerId 用于等值范围，后续 UpdatedAt 顺序直接满足该范围内的倒序需求。Limit 得到五十个根就能停止继续读取这个范围。它同时服务过滤、顺序和早停，比“分别建两个索引”更贴合这项查询。多列索引、索引与排序

17.3 根选择变快，外层工作仍存在

复合索引的代表计划中，根 Index Scan 输出五十行。Links 的索引访问执行五十次，每次一条；Discoveries 的索引访问也执行五十次，每次两条；最终连接输出一百行。这里的 Actual Rows 要结合 Actual Loops 阅读，不能把“每次一行”当成整个查询只处理一行。

外层仍出现 Incremental Sort，因为完整排序还包含根 Id 与发现键。根选择分支不再有独立的时间排序，并不意味着整个 SQL 树没有任何 Sort。性能解释应指向具体节点，避免把局部改善扩大为完整查询重构。

执行计划中的 cost 是规划器估算单位，不是毫秒；EXPLAIN ANALYZE 的 actual time 则来自实际执行。父节点覆盖子工作，不能把所有节点时间简单相加当成总时间；BUFFERS 也要注意层级和重复计数。阅读 EXPLAIN

17.4 数值只在测量条件中有意义

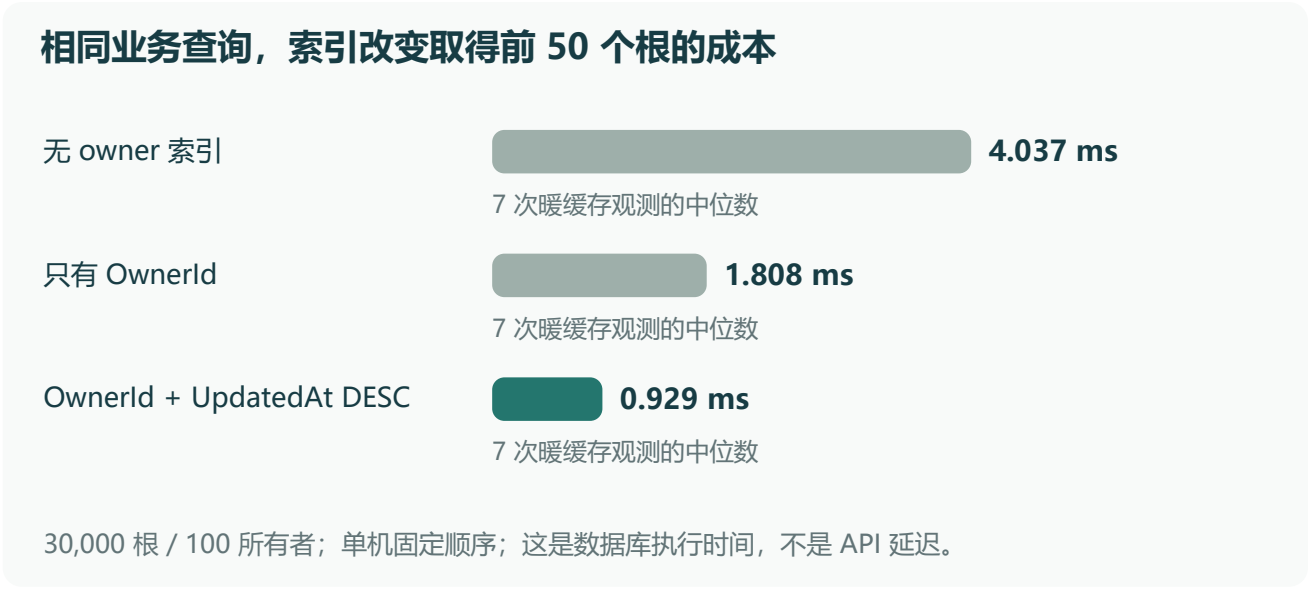


图 17.2 实际 Store.ListAsync 的参数化 SQL；复合索引减少根选择工作，连接仍有自己的成本。

索引条件	七次执行时间的中位数	可以解释的主要变化
无 owner 索引	4.037 ms	扫描大量无关根后选前五十
只有 OwnerId	1.808 ms	缩小候选，但仍需按时间排序
OwnerId + UpdatedAt DESC	0.929 ms	按用户和顺序提前取得五十根

数据来自 PostgreSQL 17.7 的一次真实应用表观察：三万根、一百用户、每用户三百根，总计六万条发现和三万条关系。每组先暖身一次，再取七次暖缓存数据，按固定顺序比较；没有并发写入，也没有真实生产分布。

这些数值支持“在此查询形状和样本中，复合索引减少工作”的解释，不能证明生产 P95、API 端到端延迟或最大吞吐。数据库执行时间没有包括完整网络往返、JSON 序列化、浏览器渲染和用户思考。一个漂亮比例若脱离范围，比不测量更容易误导决策。

17.5 为什么不继续增加所有可能的索引

索引需要存储和维护，写入时可能增加工作。当前 UpdatedAt 每次成功操作都改变，因此复合索引也随业务写入维护。是否值得，需要看列表读取收益与写入成本，而不是只看一次查询减少了几毫秒。

如果列表只需要标题、状态、计数和最后修改时间，未来可以引入专门摘要投影，减少正文和完整子集合。那会改变查询与前端合同，属于另一项可评审优化。当前章节没有把可能方案伪称成已实现成果。

性能研究的可靠输出是因果模型：哪些行必须访问，何处能提前停止，什么结构造成重复，哪项上限支持当前简化。当数据分布和需求改变，工程师可以用这个模型重新判断，而不是机械复制一组历史数字。

17.6 从索引次序推导早停条件

设索引按 owner 分组，每组再按 UpdatedAt 倒序排列。查询固定 owner 时，可以进入这一组，从最新位置依次取得五十条；不必先遍历其他 owner，也不必看完本组才知道前五十。这个顺序推理解释了为什么列次序要与等值筛选和排序一起考虑。

如果查询取消 owner 条件，现有复合索引并不天然提供所有用户共同的 UpdatedAt 顺序，因为第一关键字仍是 owner。若查询按另一个字段排序，也不能仅凭索引包含时间就期待相同收益。索引服务于具体访问模式，不是一个“更快存储”的通用标签。

如果大量记录拥有相同 UpdatedAt，当前列表对根的选择没有稳定分页次序。增加 Id 作为排序与游标组成部分，会同时改变查询和索引设计。性能与合同再次相遇：先明确结果次序，再决定如何高效取得它。

17.7 计划是解释，不是固定模板

规划器会依据数据统计、选择率和可用路径改变策略。某个样本中选择 Bitmap Heap Scan，不表示只有 owner 索引时永远只能走这一节点；数据更少或分布不同，可能出现其他合理计划。观测的价值在于能解释这一次为何减少候选和排序，而不是把节点名称背成规则。

暖缓存样本中读块多为命中，不代表没有存储访问成本，更不代表冷启动相同。固定比较顺序也可能引入缓存与环境偏差。若要支撑更广的性能决定，需要不同数据分布、交错测量或实际负载；当前教材停在应用查询机制，不把有限观察包装成容量规划。

第 18 章

用反例验证承诺，而非累积测试数量

测试最有价值的部分，是把容易被忽略的时间与边界变成可观察条件。一个测试通过，不代表系统所有性质都成立；一个有因果设计的反例，则能说明某项机制为什么必要。EchoCase 的独立测试分别观察领域决定、真实 PostgreSQL 事实和浏览器交互，不把它们混成一种“运行成功”。

18.1 先确定要推翻的错误假设

若假设“修订关系就是删除后重建”，测试应检查身份和端点保持；若假设“收到成功就能清 dirty”，测试应让用户在响应返回前输入下一代文本；若假设“版本 if 检查已经足够”，测试应强制双方都在数据库保存前读到旧版本。



图 18.1 测试层次说明能观察什么；目录或测试总数都不能替代反例的解释。

这样的测试由反例驱动，而不是把每个方法再调用一遍。它们需要明确的观察点：对象是否在失败后保持旧值，事务是否只留下成功回执，输入框是否仍有新文本。状态码和截图可以参与解释，却不能替代所有关键事实。

18.2 单元边界检验规则与身份

源码 test/unit/EchoCase.UnitTests/LinkRevisionTests.cs · L19-36

```
19      public void RevisionKeepsIdentityAndEndpointsAndAdvancesOnlyRootOnce()
20      {
21          var i = Ready(); var link = i.Links.First(); var version = i.Version;
22          var identity = (link.Id, link.InvestigationId, link.SourceId, link.TargetId);
23          i.ReviseLink(link.Id, LinkKind.Contradicts, "  " + new string('证', 20) + "  ",
    Now.AddMinutes(1));
24          Assert.Multiple(() =>
25          {
26              Assert.That((link.Id, link.InvestigationId, link.SourceId, link.TargetId),
    Is.EqualTo(identity));
27              Assert.That(link.Kind, Is.EqualTo(LinkKind.Contradicts));
28              Assert.That(link.Rationale, Is.EqualTo(new string('证', 20)));
29              Assert.That(link.RevisedAt, Is.EqualTo(Now.AddMinutes(1)));
30              Assert.That(i.Version, Is.EqualTo(version + 1));
31              Assert.That(i.Links.Count, Is.EqualTo(2));
32          });
33          var e = Assert.Throws<DomainRule>(() => i.ReviseLink(link.Id, link.Kind,
    link.Rationale!, Now));
34          Assert.That(e!.Code, Is.EqualTo("link_unchanged"));
35          Assert.That(i.Version, Is.EqualTo(version + 1));
36      }
```

这个实际测试先保存关系身份与版本，再修订，断言 Id、InvestigationId、SourceId、TargetId 不变，类型、解释和时间变化，根版本只推进一次。它还立即重复相同修订，要求得到 link_unchanged 并保持版本。这样同时区分有效变化与无意义重复。

源码 test/unit/EchoCase.UnitTests/LinkRevisionTests.cs · L37-44

```
37      [TestCase(19, false), TestCase(20, true), TestCase(500, true), TestCase(501, false)]
38      public void ExplanationUsesUnicodeScalarsMatchingPostgresCharacterLength(int count, bool
    valid)
39      {
40          var i = Ready(); var link = i.Links.First(); var version = i.Version;
41          var text = string.Concat(Enumerable.Repeat("🔑", count)); // Two UTF-16 code units,
    one scalar.
42          if (valid) { i.ReviseLink(link.Id, link.Kind, text, Now); Assert.That(i.Version,
    Is.EqualTo(version + 1)); }
43          else { Assert.Throws<DomainRule>(() => i.ReviseLink(link.Id, link.Kind, text, Now));
    Assert.That(i.Version, Is.EqualTo(version)); }
44      }
```

边界用例使用一个在 UTF-16 中占两个 code unit 的字符，覆盖十九、二十、五百和五百零一个标量值。若把 EnumerateRunes 改为 Length，测试会直接挑战业务单位，而非只证明普通汉字可通过。固定时间也让断言不依赖真实等待。

单元测试不能证明 PostgreSQL 的 CHECK 接受同样内容，也不能证明 EF 把子关系标为 Modified。它没有数据库，正因此快速且清晰；跨到持久机制的结论需要另一个边界。

18.3 集成验证必须触及真正的竞争窗口

源码 test/integration/EchoCase.IntegrationTests/PostgresApiTests.LinkRevision.cs · L31-37

```
31                  var result = await new PostgresInvestigationStore(db, TimeProvider.System)
    .WriteAsync(new(owner, keys[n], new string((char)('A' + n), 64), s.Id, s.Version,
    "revise-link"), i =>
32                  {
33                      isolations.Add(db.Database.CurrentTransaction!.GetDbTransaction()
    .IsolationLevel.ToString());
34                      if (!barrier.SignalAndWait(TimeSpan.FromSeconds(20))) throw new
    TimeoutException("Both real SQL reads must reach the race barrier.");
35                      i!.ReviseLink(s.Links.Single().Id, LinkKind.Contradicts, new string((char)
    ('甲' + n), 20), DateTimeOffset.UtcNow);
36                      return i;
37                  }, default);
```

每次 Attempt 使用独立上下文。屏障位于 Store 已加载聚合之后、领域变更之前，要求双方都走到同一个窗口。不同键排除同键重放的影响，再断言只有一个成功、失败为 412，并检查最终关系、版本、回执与审计。

这里使用真实 PostgreSQL，是因为唯一键等待、事务回滚、READ COMMITTED 与生成 SQL 都属于数据库行为。内存替身可以验证方法被调用，却不能证明这些竞争性质。WebApplicationFactory 可以替代 HTTP 宿主，不意味着数据库也应该换成内存提供者。

迁移验证同样需要旧列集中的真实数据，以及旧 JSON 回执。只在空库执行最新模型，会漏掉“旧行保持身份”和“缺少新字段仍可反序列化”两个承诺。验证对象应随兼容对象变化，而不是总从干净世界出发。

18.4 浏览器反例要控制因果，少猜时间

源码 test/e2e/specs/consistency.spec.ts · L71-76

```
71  await page.route("**/api/investigations/*/notes", async (route) => {
72    const response = await route.fetch(); // Real API commits before the response is held.
73    committed();
74    await hold;
75    await route.fulfill({ response });
76  });
```

route.fetch 先让真实 API 执行并提交，再延迟把响应交给页面。测试随后让输入框继续编辑，最后释放响应。这与直接伪造一个成功 JSON 不同：事务确实发生，只有交付时间被控制。另一项反例在真实提交后截断正文，用来研究成功状态与可解析结果之间的缺口。网络拦截能力

等待固定秒数只能猜测环境速度；等待明确的 committed Promise、版本标签或保存状态则描述事件关系。类似地，界面中某段文字可能同时出现在输入框、pending 提示和确认区域，断言“页面包含这段文字”无法证明它已经持久化。应定位正确区域，并在需要时读取真实服务器值。可等待断言

这不是把书变成浏览器脚本教程。关键思想是：让被研究的先后关系确定下来，再观察对应事实。若控制条件与结论无关，即使脚本跑得很稳，也没有验证想要的承诺。

18.5 测试依赖边界保护生产系统

Unit 只引用 Domain/Application，Integration 可以引用实际 API 与持久层，E2E 通过公开界面交互。生产项目不能引用测试项目或夹具；Playwright 安装在独立 test/e2e 包中，不进入前端生产依赖。应用解决方案不包含测试项目。

验证应从多个方向观察：项目引用描述显式结构，求值后的 MSBuild 项目图揭示条件导入，解析后的包图揭示传递依赖，实际发布目录和浏览器 bundle 揭示产物。它们各自能发现另一层看不到的问题，却不需要把实现这些检查的脚本印进教材。

data-testid 只是可测试属性，并不等于把测试运行器打进生产。反过来，没有 test 字样也不保证依赖正确。边界根据实际来源和依赖关系判断，避免把命名当成架构事实。

18.6 回归保护与设计演进

重构时优先保留外部承诺的测试：身份稳定、失败原子、草稿保留、显式冲突。若测试只断言私有函数调用顺序，内部拆分就会造成无意义失败；若测试完全不观察持久结果，破坏事务也可能悄悄通过。好的验证给合理重构留空间，同时对用户会感受到的回归敏感。

测试总数不进入本书的学习成绩。能解释每个反例推翻了哪项错误假设、还留下哪些未覆盖条件，比重复展示绿色记录更有价值。这些判断还需要独立的观察出口，避免结论和被测实现共享同一个错误。

18.7 有效反例需要独立的观察出口

若浏览器测试只看同一个输入框，就无法区分本地仍显示原文与服务器真的保存。独立 Get 读取或确认区域提供另一个观察出口；数据库竞争测试则用新上下文检查持久结果，避免直接相信失败上下文里的对象。

观察出口也必须符合研究问题。验证所有权时，应使用不同身份请求同一实际资源，不能只检查菜单是否隐藏；验证幂等时，应比较原键重放后的版本和持久审计，而不只是两个响应都 200；验证迁移时，应保存旧行身份并跨迁移读取，而不只查看新列存在。

这些做法不是让每个测试都跨越全部系统。它们让一项保证至少有一个不会被同一错误共同伪造的观察。单元规则用独立断言，数据库事实用真实重读，界面承诺用目标区域和服务器结果，各自保持清楚范围。

18.8 小而明确的持续验证

持续验证执行应用构建、独立测试、合同和依赖边界。它负责尽早发现应用回归，不拥有环境发布和基础设施运行。出版工具也不被塞进生产依赖；教材更新可以独立完成版式与编辑检查。

第 19 章

把“修订关联”作为一次完整需求演进

真实需求变化通常不会只落在一个文件。允许修订关系类型并补充解释，看似很小，却会影响身份、规则、数据库、回执、协议和前端编辑。维护能力来自识别这些必要影响，并复用已有保证，而不是通过一次变化引入一整套新框架。

19.1 先确定什么保持，什么改变

原系统已经能建立、删除关系。新需求要求修改同一关系的判断，因此 Id 与端点保持；Kind、Rationale 和 RevisedAt 变化；根版本前进一次。Active 才能执行，二十至五百个标量值的解释是必需内容，相同类型和解释应拒绝。

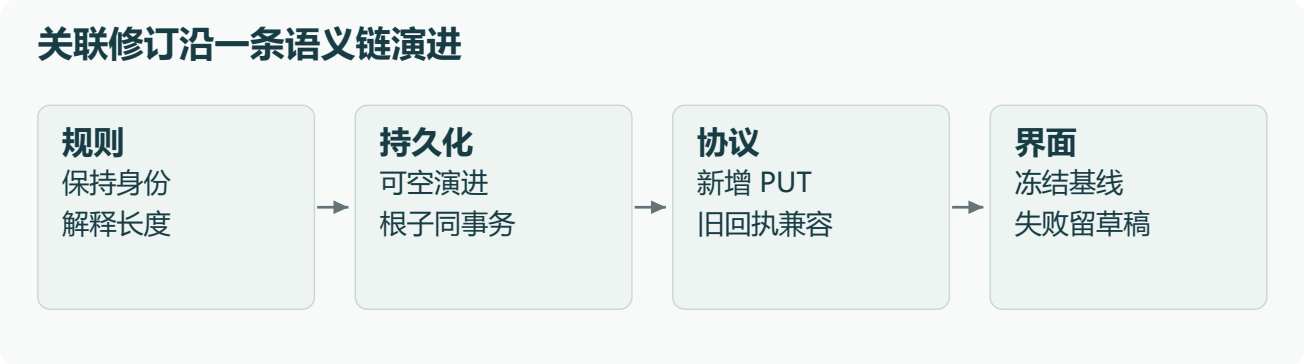


图 19.1 变化穿过必要边界；共享 Write 路径承担幂等、版本、审计，不再复制另一套。

这个定义排除了“删除再创建”和“直接改子表”两种看似快捷的实现。前者破坏身份，后者绕过根版本和共同事务。需求越清楚，需要触及的边界反而越少：新增一个根行为和对应端点，沿共享 Write 进入原有幂等、审计和并发路径。

19.2 领域新增行为，应用复用协议

ReviseLink 的校验都在赋值之前，子对象内部 Revise 只接收已经通过规则的值。Service 做精确枚举转换，将 linkId 和正文放进指纹。Store 无需再为这项操作复制一套锁、回执和事务逻辑，只记录新的 operation 名。

这种复用有具体前提：新行为仍然只改变一个调查聚合，仍是本地数据库副作用。若需求变为“修订后必须发送可靠外部通知”，共享 Write 还不够，不能为了减少代码而忽视副作用边界。抽象只能复用相同承诺，不能强行吞下不同承诺。

19.3 持久化兼容贯穿三条路径

新列可空，旧行不补虚构说明；新行为写成对值，数据库约束拒绝半对；旧 LinkView JSON 缺字段时使用默认 null。这里同时保护已有表行、新写入和旧回执。迁移文件成功只是其中一项事实。

真实跟踪观察中，旧关系的 Rationale 仍为 null；修订后子对象只有 Kind、Rationale、RevisedAt 变脏，根只有 Version、UpdatedAt 变脏。成功保存后版本从 4 到 5；旧版本尝试的子修改随事务失败回滚。这个结果让“新增行为没有绕开原保护”成为可以解释的结论。

19.4 新端点不改变旧端点的全部接受范围

PUT 使用已有关系 Id 和新请求类型，不接收新端点。精确枚举名称保证新合同清晰，旧 POST 的数值字符串兼容行为则没有顺便重写。需求演进中，统一与兼容经常冲突；若想收紧旧行为，应作为单独合同变化评估，而不是夹在“新增功能”里无声发生。

返回仍为完整 SessionView，ETag 使用新根版本，幂等指纹包含操作与 linkId。两个目标不同的修订不会被误认为同一请求，复用旧键表达新解释会得到 409，旧版本的新修订得到 412。它们继承的是原协议，而不是客户端碰巧没有重复点击。

19.5 前端拆分围绕变化原因

LinkEditor 拥有打开时版本和字段草稿；EvidenceGraph 负责从确认关系映射边；useInvestigation 负责请求和缓存；Workspace 负责接线与统一错误。每个模块都有一个可以用自然语言说清的状态所有者。

保存失败时解释保留，成功并完成必要刷新后表单关闭。后台新版本使旧编辑变成冲突，不能自动换上新版本再发。图布局不写入数据库，所以增加解释不会额外改变本地节点的业务含义。

这类拆分修复了大组件中容易隐藏的耦合：一个成功回调原本可能同时清草稿、写缓存、关表单和换页签。现在每项更新依据自己的前提发生。文件变短只是副产品，真正收益是可以分别推理。

19.6 评审应从用户承诺逆向阅读

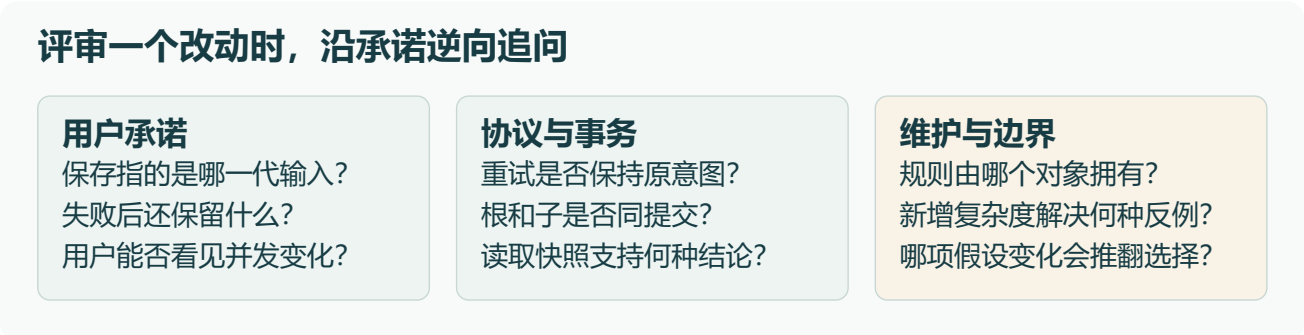


图 19.2 综合推理把需求、源码和运行含义连起来，而非用术语数量给架构评分。

从“同一关系被修订”回到 Guid 和端点；从“不会丢别人的修改”回到根版本与真实 UPDATE；从“网络失败可以恢复”回到指纹和历史回执；从“等待时可继续写”回到 revision 和确认条件。这种逆向追踪能发现某层遗漏，而不是只看每个文件是否符合样式。

必要验证也沿这些承诺展开：纯领域断言身份和计数单位，真实数据库断言迁移与回滚，浏览器断言编辑器和确认状态。无需为每个新增文件创造一个镜像测试，也无需把一次小需求升级成全面 QA 平台。

一次良好演进结束后，系统应更容易解释。新规则有唯一入口，旧数据意义没有被改写，已有协议继续承担相同职责，用户在错误和并发下仍能做出清楚决定。这样的维护质量可以从具体行为看出来，无需用复杂度装饰它。

19.7 变更影响图为什么比文件清单有用

文件清单告诉人哪里动过，却不解释为什么必须一起改变。关联修订的影响图从“身份保持”连到领域方法和 UI key，从“新解释”连到列与旧 JSON，从“旧窗口保护”连到根版本和编辑基线，从“结果未知”连到同一回执协议。每条边都对应一项承诺。

若某个新增文件无法连到承诺，可能是无必要复杂度；若某个承诺没有任何落点，可能是遗漏。这样的评审比逐文件看语法更容易发现跨层空隙，也能让不同专长的人讨论同一件事。数据库工程师和前端工程师不需要懂对方每个 API，仍能围绕“失败后解释不能丢”建立共同判断。

19.8 什么时候应该停止重构

当规则入口明确、已有协议能够复用、失败反例得到保护，继续把所有相似代码抽成通用框架未必增加价值。NotesEditor 与 LinkEditor 都有基线，却有不同确认交互；过早强行统一，可能把两种清晰行为塞进充满开关的通用编辑器。

停止的依据不是“代码已经最短”，而是当前变化可以清楚推理，剩余差异有实际理由，未来改变条件被记录。维护性包含克制：保留可解释的简单实现，并知道什么新需求会让它不再足够。

第 20 章

设计推理题与解析

本章的问题不要求先准备环境。每题先给出事实，请读者预测结果并说明理由，再对照解析。判断依据是前面各章建立的对象、快照、事务与意图模型；若需要深究实现，可从所列符号进入源码。

20.1 成功响应能否清掉新输入

事实：服务器笔记为空、版本 1。用户输入 A 后保存，操作带 revision 1。等待期间用户又输入 B，当前文本变为 A+B、revision 2。服务器成功返回 A、版本 2。有人建议“既然成功，就把 dirty 设为 false”。这是否正确?

解析：不正确。成功确认的是发送出去的 A/r1，不包含 B。正确状态是确认缓存进入 v2/A，草稿基线进入 v2/A，当前文本保持 A+B，dirty 为 true。若响应来自更旧回执且 current 已更高，还不能借它推进当前草稿确认。关键符号是 useNotesDraft 的 draftRevision 比较与旧版本提前返回。

把判断改成“响应 notes 与当前 text 不同就保留”仍不完整。用户可能删回相同文本，但编辑代次与意图已经变化；字符串相等不能表达全部时间关系。代次提供的是因果信息。

20.2 幂等重试为什么不能换新版本

事实：K 在版本 4 上成功，生成版本 5，但客户端断线。另一窗口又生成版本 6。客户端恢复后把原 body、原 K 与最新 If-Match 6 一起重发，认为这样更容易通过。会发生什么?

解析：版本进入指纹，原 K 对应的是版本 4 的请求，换成 6 后应得到 409。正确重试保留 K、4 和原 body，恢复版本 5 的历史成功；确认缓存保持已经取得的 6。若用户看过版本 6 后确实想再次保存，应建立新键的新意图，仍可能遇到后续竞争。

这题区分资源前提与操作身份。版本不是为了“找到能通过的数字”，而是表明用户作决定时看过什么。

20.3 只给子表加版本能否保持原语义

事实：为了减少冲突，开发者删掉根版本推进，只在 EvidenceLink 上增加独立版本。笔记修改和关系修订于是可以同时成功。是否可以宣布这是无风险优化?

解析：不可以。它改变了调查的决定边界。原来任何成员变化都要求旧窗口核对，现在关系判断可能在未见新笔记的情况下成立。若业务证明两类决定独立，可以设计更细版本；还需重新考虑复核状态、发现条件、审计版本与读模型。性能收益不能代替语义证明。

若只是对子表加版本，同时保留根版本，系统会更保守并增加复杂度，却未必获得更高并发。每个令牌都需要明确保护的事实。

20.4 DetectChanges 前 Unchanged 是否说明保存失效

事实：调用 ReviseLink 后，调试看到两个条目仍是 Unchanged，但 CLR 中 Version 已加一、Kind 已改变。有人准备调用 Update(root) 强行标记整图。该如何判断？

解析：先确定是否已经运行变更检测。默认快照跟踪不保证每次普通赋值后立即改变条目状态；真实观察中 DetectChanges 后只把相关属性标脏。强行 Update 整图可能掩盖真正问题，也可能把无需修改的字段一并发送。应比较当前值、原始值、生成策略与 SQL，再判断是否存在漏存。

若新子成员被标成 Modified 而应为 Added，则需要检查键生成配置。并发异常还可能由更新不存在的行造成，不能只凭异常名认定其他用户竞争。

20.5 复合索引消除了所有排序吗

事实：复合索引让根查询直接取五十行，计划外层仍有 Incremental Sort。有人认为索引没有生效，因为还能看到 Sort。如何解释？

解析：根选择的排序与外层为连接物化安排的完整排序不是同一节点。代表计划已显示根由有序 Index Scan 输出五十行，外层仍按根 Id、发现键等整理连接行。应沿树定位工作，结合 rows 和 loops，而不是搜索某个单词判成败。

中位数从 4.037 ms 到 0.929 ms 只说明给定暖缓存样本中的数据库执行变化，不能直接变成 API P95 承诺。测量单位与适用范围同样属于答案。

20.6 新解释列能否全部补成迁移时间

事实：旧关系没有修订解释和时间。有人为减少 null，把解释设为“历史关联”，时间设为迁移执行时刻。新界面因此不必处理空值。这合理吗？

解析：不合理。它把技术迁移伪装成业务修订。旧事实应继续用成对 null 表达，新修订写真实解释与业务时间，界面明确展示有无解释。还要保证旧回执 JSON 可恢复，而不是只让新表查询通过。

数据清洁不等于所有列非空。缺失事实有明确意义时，诚实保留缺失比制造完整表象更重要。

20.7 把图坐标放进 Version 会怎样

事实：为了跨窗口共享布局，开发者打算每次拖动都更新 Investigation.Version。会带来什么影响，是否一定不应该做？

解析：拖动会与笔记、关系、复核竞争同一版本，后台布局同步可能不断触发草稿冲突。高频手势还需要节流、提交和失败恢复。若共享布局确有业务价值，可以设计独立布局资源或明确纳入调查协议；不能只把局部 position 序列化后就忽略一致性成本。

答案不是禁止持久化坐标，而是要求先决定布局属于共享业务事实还是个人呈现。不同所有权需要不同版本和权限边界。

20.8 发送通知能否放进 SavingChanges

事实：修订成功后需要发邮件。开发者在保存前事件中发送，随后数据库根 UPDATE 发生冲突并回滚。用户收到邮件说修订成功。应怎样理解这个失败？

解析：外部邮件不在 PostgreSQL 事务里，不能随回滚撤回。当前本地事务协议已超出适用边界。应考虑在事务中持久化待交付事实，提交后由独立交付过程发送并处理重试、去重与状态；仅把调用移到另一个类，不会建立可靠性。

这些题可以继续延展，但应保持同一种推理习惯：先标出事实的时间和所有者，再判断哪个操作有权改变它。源码可以帮助检查机制，理解则来自因果关系。

附录 术语、源码地图与一手参考

本附录帮助定位概念，不作为环境安装或操作清单。源码以书前固定的应用提交为准；在线文档可能继续更新，应核对所读版本与项目锁定依赖。机制结论来自本书对实际实现和明确范围内观察的解释。

A.1 易混概念对照

术语	在本书中的准确含义
案件定义	固定材料目录，不保存某位用户的调查进度
调查实例	某 owner 的独立聚合，拥有稳定 Guid 和版本
聚合根	统一表达调查成员约束和状态变化的业务对象
逻辑 CQRS	读写意图与方法分离，当前共用数据库和视图
工作单元	一次有限加载、决定和保存中的上下文生命周期
身份解析	同一跟踪上下文中，同一实体键对应同一实例
原始值	EF 跟踪时保存的前值，用于差异与并发条件
当前值	此刻 CLR 对象中的值，尚不代表已经提交
DetectChanges	把属性与关系差异识别成跟踪状态的阶段
MVCC 行版本	数据库为并发快照管理的物理可见性机制
应用 Version	由调查根推进、公开给客户端的业务版本
If-Match	客户端本次新决定基于的已确认资源前提
幂等键	在 owner 范围内标识同一次意图，不是资源版本
指纹	当前应用对目标、版本、操作与正文建立的摘要
回执	某次成功操作当时的历史 SessionView
原子性	相关持久修改一起成立或一起不成立
结果未知	浏览器尚无法判断原写入是否成功的本地状态

术语	在本书中的准确含义
确认缓存	只包含服务器确认快照的 Query 数据
pending 投影	对正在发送操作的暂时呈现，不冒充已保存
baseline	草稿开始编辑时依赖的已确认内容和版本
revision	本地编辑代次，区别于服务器资源版本
dirty	尚需用户或保存结果处理的本地编辑意图
rebase	把草稿前提换到新事实；当前不静默执行
图布局	节点坐标和视口等呈现状态，当前不持久化

A.2 按问题进入源码

想理解的问题	主要符号与当前路径
状态与成员规则	Investigation；backend/EchoCase.Domain/Investigation.cs
用例与读模型	InvestigationService、SessionPresenter；backend/EchoCase.Application/Investigations.cs
原子写协议	WriteAsync；backend/EchoCase.Infrastructure/PostgresInvestigationStore.cs
映射与关系约束	OnModelCreating；backend/EchoCase.Infrastructure/EchoDbContext.cs
请求和身份边界	Program；backend/EchoCase.Api/Program.cs
网络分类	request；frontend/src/api/client.ts
确认缓存生命周期	useInvestigation；frontend/src/workspace/useInvestigation.ts
草稿确认与冲突	useNotesDraft；frontend/src/workspace/useNotesDraft.ts
字段交互	NotesEditor、LinkEditor；frontend/src/workspace/
业务图与局部布局	EvidenceGraph；frontend/src/EvidenceGraph.tsx
规则反例	LinkRevisionTests；test/unit/EchoCase.UnitTests/

想理解的问题	主要符号与当前路径
持久边界反例	PostgresApiTests、MigrationTests；test/integration/EchoCase.IntegrationTests/
浏览器时间反例	consistency.spec.ts；test/e2e/specs/

以上是研究入口，不要求按表逐项执行。章节中的源码框提供固定提交、准确行号和链接，可沿调用进入完整上下文。

A.3 框架与协议的一手资料

- ASP.NET Core 中间件：阅读委托嵌套、顺序与短路。
- ASP.NET Core 防伪：理解 Cookie 应用的请求令牌边界。
- 默认 ProblemDetails writer 固定源码：核对默认扩展与定制钩子的先后。
- EF Core 跟踪、变更检测、身份解析：分别理解状态、差异与实例身份。
- EF Core 并发、事务、生成值：核对 WHERE 原值、提交边界与键生成。
- EF 单查询与拆分：理解同级集合放大与读取一致性的取舍。
- PostgreSQL 17 MVCC、隔离级别、显式锁：区分快照、写竞争与事务级 advisory lock。
- PostgreSQL 时间类型、EXPLAIN、多列索引、排序索引：核对精度与真实访问路径。
- React 渲染与提交、状态快照、更新队列：理解闭包与函数式更新。
- React Effect、状态保留、StrictMode：理解组件身份与开发检查。
- TanStack Query 查询键、取消、写后失效：核对使用中 v5 的行为。
- React Flow 交互：研究受控节点与变化应用。
- Playwright 网络、断言：理解如何控制交付与等待事实。
- CleanArchitecture 固定上游：用于第二章的机制比较，不表示已运行上游完整测试。

A.4 关于真实性与适用范围

UI 图片来自真实 EchoCase 浏览器捕获，书页只裁切相关区域并叠加独立编号。机制图为本书原创矢量图；示意版本、事务与时间轴不伪装成采集数据。查询时间来自单机、合成应用数据、暖缓存的有限观察，不能当作生产容量结论。